

МИНИСТЕРСТВО ОБРАЗОВАНИЯ И НАУКИ РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное автономное образовательное учреждение высшего образования
«Санкт–Петербургский государственный университет
аэрокосмического приборостроения»

Кафедра №43 «Компьютерных технологий и программной инженерии»

ОТЧЁТ ПО ПРАКТИКЕ
ЗАЩИЩЁН С ОЦЕНКОЙ
РУКОВОДИТЕЛЬ

Ст. преподаватель

должность, уч. степень, звание

подпись, дата

С.А. Рогачев

инициалы, фамилия

ОТЧЁТ ПО ПРАКТИКЕ

вид практики производственная

тип практики преддипломная

на тему индивидуального задания Агрегатор услуг доставки грузов

выполнен

Мвале Лисон

фамилия, имя, отчество обучающегося в творительном падеже

по направлению подготовки	02.03.03	Математическое обеспечение и администрирование информационных систем
	код	наименование направления

направленности	02	Системный анализ в информационных технологиях
	код	наименование направленности

Обучающийся группы №	4236	Л. Мвале
	номер	подпись, дата
		инициалы, фамилия

Санкт–Петербург 2026

РЕФЕРАТ

Выпускная квалификационная работа на тему «Агрегатор услуг доставки грузов» выполнена на ____ страницах, с использованием ____ рисунков, ____ таблиц, ____ источников и 5 приложений.

Ключевые слова: динамическое программирование, доставка грузов, оптимизация маршрутов, задача о рюкзаке, микросервисная архитектура, Node.js, WebSocket, Docker.

Цель работы – разработка программного обеспечения для автоматизации оптимального распределения ресурсов доставки с использованием методов динамического программирования и микросервисной архитектуры агрегатора услуг доставки грузов.

Решаемая проблема: неоптимальное (по скорости и стоимости доставки грузов) назначение водителей, неэффективное планирование маршрутов и недостаточное использование грузоподъемности транспортных средств в существующих логистических платформах.

Разработка микросервисного программного обеспечения, реализующего алгоритмы динамического программирования (задача о рюкзаке 0/1 и многомерный вариант) для оптимизации назначения заказов водителям на основе веса, объема и доступности транспорта, а также внедрение систем отслеживания в реальном времени с использованием WebSocket, – есть предмет рассмотрения данной квалификационной работы.

Объектом разработки является платформа-агрегатор доставки, которая способна: повышать точность назначения заказов на 25%; сокращать среднее время доставки на 15% по сравнению с жадными алгоритмами; поддерживать отслеживание в реальном времени и безопасную межсервисную коммуникацию.

В процессе исследования используются: платформа Node.js, язык TypeScript, СУБД MySQL и Docker.

ABSTRACT

The graduate qualification work on the topic «Cargo Delivery Service Aggregator» is completed on ___ pages, using ___ figures, ___ tables, ___ sources and 5 appendices.

Keywords: dynamic programming, cargo delivery, route optimization, knapsack problem, microservice architecture, Node.js, WebSocket, Docker.

The purpose of the work is to develop a software solution for automating the optimal allocation of delivery resources using dynamic programming methods within a microservice architecture for cargo delivery aggregation.

The problem being solved: suboptimal driver assignment, inefficient route planning, and underutilization of vehicle capacity in existing logistics platforms.

The development of microservice-based software implementing dynamic programming algorithms (0/1 and multi-dimensional knapsack problem) for optimizing order-to-driver assignments based on weight, volume, and vehicle availability, as well as the implementation of real-time tracking systems using WebSocket, are the subject of this qualifying work.

The object of development is a delivery aggregator platform capable of: improving order assignment accuracy by 25%; reducing average delivery time by 15% compared to greedy algorithms; supporting real-time tracking and secure inter-service communication.

The following were used in the research process: the Node.js platform, the TypeScript language, the MySQL DBMS, and Docker.

СОДЕРЖАНИЕ

ОБОЗНАЧЕНИЯ И СОКРАЩЕНИЯ	1
ВВЕДЕНИЕ	2
1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И ПРОЕКТИРОВАНИЕ РЕШЕНИЯ	4
1.1 Актуальность проекта	4
1.2 Анализ предметной области	4
1.3 Анализ рынка и конкурентной среды	6
1.3.1 Анализ современных платформ (Uber, DHL, Яндекс Go, CDEK)	8
1.3.2 Выявление технологических разрывов и проблемных зон в существующих решениях	9
1.4 Формирование концепции агрегатора услуг доставки грузов DeliveryMatch	10
1.4.1 Определение ролей и бизнес-процессов	10
1.4.2 Обоснование гибридной модели агрегатора грузоперевозок с поддержкой пакетных заказов	13
1.4.3 Моделирование функциональных требований в виде вариантов использования	15
1.5 Постановка задачи и определение целей разработки	35
1.5.1 Формализация ключевых проблем	36
1.5.2 Определение целевых показателей эффективности	37
1.6 Выбор технологического стека и архитектурных решений	39
1.6.1 Обоснование использования React, TypeScript и микросервисной архитектуры	39
1.6.2 Проектирование структуры базы данных	40
2 ТЕОРЕТИЧЕСКИЕ ОСНОВЫ И РАЗРАБОТКА АЛГОРИТМИЧЕСКОГО ЯДРА АГРЕГАТОРА DELIVERYMATCH	49
2.1 Сравнительный анализ математических моделей оптимизации логистических процессов	49
2.2 Разработка и обоснование алгоритма сопоставления заказов и водителей	50
2.2.1 Алгоритм GreedyFeasibilityCheck (многомерная задача о рюкзаке)	50
2.2.2 Геопространственный алгоритм поиска ближайших водителей	58
2.2.3 Интегрированная модель сопоставления	61

2.2.4 Разработка модели динамического ценообразования на основе машинного обучения	62
2.3 Разработка и обоснование алгоритмов автоматизированного планирования загрузки транспортного средства	66
2.3.1 Алгоритм автоматической упаковки грузов (задача бин-пакинга)	66
2.3.2 Алгоритм балансировки нагрузки (оптимизация центра масс)	69
2.4 Анализ масштабируемости и производительности алгоритмического ядра	70
2.4.1 Оценка вычислительной сложности и требований к памяти	70
2.4.2 Моделирование работы системы при пиковых нагрузках	71
2.5 Выводы по главе 2	73
3 РЕАЛИЗАЦИЯ И ИНТЕГРАЦИЯ МИКРОСЕРВИСОВ	75
3.1 Структура проекта и файловая организация	75
3.1.1 Общая архитектура проекта	75
3.1.2 Конфигурация Docker и оркестрация	78
3.2 Пользовательские интерфейсы	79
3.2.1 Клиентский веб-интерфейс	79
3.2.2 Приложение для водителей	83
3.3 Реализация REST API и взаимодействия микросервисов	90
3.3.1 Проектирование REST API	90
3.4 Объектно-ориентированная модель системы	91
3.4.1 Основные классы домена	91
3.4.2 Сигнатуры методов с типами возвращаемых данных	93
3.5 Выводы по главе 3	96
4 ТЕСТИРОВАНИЕ, ВНЕДРЕНИЕ И АНАЛИЗ РЕЗУЛЬТАТОВ	98
4.1 Методика тестирования	98
4.2 Модульное и интеграционное тестирование алгоритмов	99
4.2.1 Тестирование алгоритма GreedyFeasibilityCheck	99
4.2.2 Геопространственный поиск Geohash и гаверсинус	100
4.2.3 Тестирование API-эндпоинтов в Postman и терминале	103
4.3 Оценка соответствия системы требованиям	109
4.3.1 Соответствие функциональным требованиям	109
4.3.2 Соответствие нефункциональным требованиям	110

ЗАКЛЮЧЕНИЕ	112
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	114
ПРИЛОЖЕНИЕ А ОСНОВНОЙ docker-compose.yml	116
ПРИЛОЖЕНИЕ Б – Скрипт нагрузочного тестирования Artillery	119
ПРИЛОЖЕНИЕ В ТИПЫ ДАННЫХ	121
ПРИЛОЖЕНИЕ Г – Файл процессора для Artillery (order-processor.js)	122
ПРИЛОЖЕНИЕ Д – Основные SQL-запросы для базы данных	123

ОБОЗНАЧЕНИЯ И СОКРАЩЕНИЯ

Deliverymatch - агрегатор услуг доставки грузов.

АС - автоматизированная система.

БД - база данных

ГИС - географическая информационная система

ИС - информационная система

k-NN - k-ближайших соседей (k-Nearest Neighbors)

MVP - минимально жизнеспособный продукт (Minimum Viable Product)

ПО - программное обеспечение

ПС - программная система

СУБД - система управления базами данных

ВИ - вариант использования (Use Case)

ПИ - пользовательская история (User Story)

ТЗ - техническое задание

ТС - транспортное средство

UI - пользовательский интерфейс (User Interface)

UX - пользовательский опыт (User Experience)

ЦМ - центр масс

DQ - качество данных (Data Quality)

ОСАГО - обязательное страхование автогражданской ответственности

GUI - графический пользовательский интерфейс (Graphical User Interface)

VRP - задача маршрутизации транспортных средств (Vehicle Routing Problem)

FDD - алгоритм «Первый подходящий с убыванием» (First-Fit Decreasing)

REST API - Representational State Transfer Application Programming Interface

СДЦ - сервис динамического ценообразования

ВВЕДЕНИЕ

Современный рынок грузоперевозок характеризуется высоким спросом на оперативную и экономичную доставку. Однако существующие логистические платформы сталкиваются с проблемой неоптимального распределения заказов между водителями, что приводит к неполной загрузке транспорта, увеличению времени доставки и росту операционных издержек. Отсутствие гибких механизмов, учитывающих одновременно вес, объем груза и местоположение в реальном времени, определяет актуальность настоящего исследования.

Объектом исследования являются процессы распределения ресурсов в системах доставки грузов. Предмет исследования – методы динамического программирования и алгоритмы оптимизации маршрутов, применяемые для повышения эффективности работы агрегатора доставки.

Целью данной выпускной квалификационной работы является разработка программного обеспечения для автоматизации оптимального распределения ресурсов доставки с использованием методов динамического программирования и микросервисной архитектуры.

Для достижения поставленной цели необходимо решить следующие задачи:

- провести анализ предметной области и существующих решений на рынке логистических услуг;
- формализовать задачу распределения ресурсов доставки как оптимизационную модель;
- спроектировать микросервисную архитектуру и структуру базы данных для агрегатора;
- реализовать алгоритмы динамического программирования (задача о рюкзаке) для оптимизации назначения водителей и эвристические методы оптимизации маршрутов;
- внедрить систему отслеживания заказов в реальном времени с использованием WebSocket;

- провести тестирование разработанного решения и оценить его эффективность.

Практическая значимость работы заключается в создании действующего прототипа платформы, позволяющего повысить точность назначения заказов и сократить среднее время доставки по сравнению с базовыми подходами.

1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И ПРОЕКТИРОВАНИЕ РЕШЕНИЯ

1.1 Актуальность проекта

Быстрый рост электронной коммерции и увеличение объема почтовых отправок создали устойчивый спрос на гибкие услуги доставки. Малый и средний бизнес, а также частные лица часто нуждаются в разовых или регулярных перевозках грузов. Для этих задач использование крупных логистических операторов зачастую является слишком дорогим и негибким, а аренда целого транспортного средства — неэффективной [1].

Существующий рыночный разрыв был заполнен платформами-агрегаторами. Однако их текущий функционал в основном заимствован из сектора пассажирских перевозок. Эти платформы не учитывают специфику физических ограничений грузов, такие как вес, габариты и требования к безопасной загрузке. Ручное согласование этих параметров между клиентом и водителем замедляет обслуживание, увеличивает количество отказов и приводит к холостым пробегам. Это негативно сказывается как на стоимости услуги, так и на экологии [2].

Таким образом, актуальность разработки платформы DeliveryMatch заключается в создании специализированного технологического решения. Данное решение автоматизирует ключевые процессы планирования, сокращая время обработки заявок и снижая влияние человеческого фактора. Оно оптимизирует использование транспортных ресурсов за счет интеллектуального сопоставления заказов и построения маршрутов, снижая затраты для всех участников. Платформа также повышает безопасность путем алгоритмического контроля соответствия груза характеристикам транспортного средства и создает основу для адаптивной экосистемы грузоперевозок.

1.2 Анализ предметной области

Предметной областью данного проекта является сфера оказания услуг по перевозке грузов с привлечением частных перевозчиков через цифровую

платформу-агрегатор. Ключевыми субъектами в этой области являются клиент, водитель-партнер и сама платформа.

Клиент представляет собой физическое или юридическое лицо, нуждающееся в доставке одного или нескольких грузов. Клиент формирует заказ, указывая параметры груза, адреса, контактные данные и желаемые сроки.

Водитель-партнер — это владелец или арендатор транспортного средства, зарегистрированный на платформе для поиска заказов. Водитель располагает транспортным средством с конкретными характеристиками, такими как грузоподъемность, объем и тип кузова.

Платформа DeliveryMatch выступает в роли цифрового посредника. Она обеспечивает взаимодействие между клиентом и водителем. Платформа осуществляет прием и валидацию заказов от клиентов. Она выполняет поиск и подбор подходящего водителя для одного или группы заказов. Платформа рассчитывает стоимость услуги на основе тарифной модели. Она организует и оптимизирует процесс доставки, строит маршруты и рассчитывает предполагаемое время. Платформа обеспечивает взаимодействие сторон в реальном времени и обрабатывает платежи.

Успех платформы определяется тремя ключевыми факторами: скоростью подбора водителя, точностью соответствия груза транспортному средству и экономической эффективностью рейса для водителя. Именно на решение этих задач направлено алгоритмическое ядро разрабатываемой системы. Согласно исследованиям в области логистики, эффективность агрегаторных платформ напрямую зависит от качества алгоритмов сопоставления и их способности учитывать многокритериальные ограничения [3].

На рисунке 1.1 показана контекстная диаграмма служб DeliveryMatch, показывающая участников и внешние службы

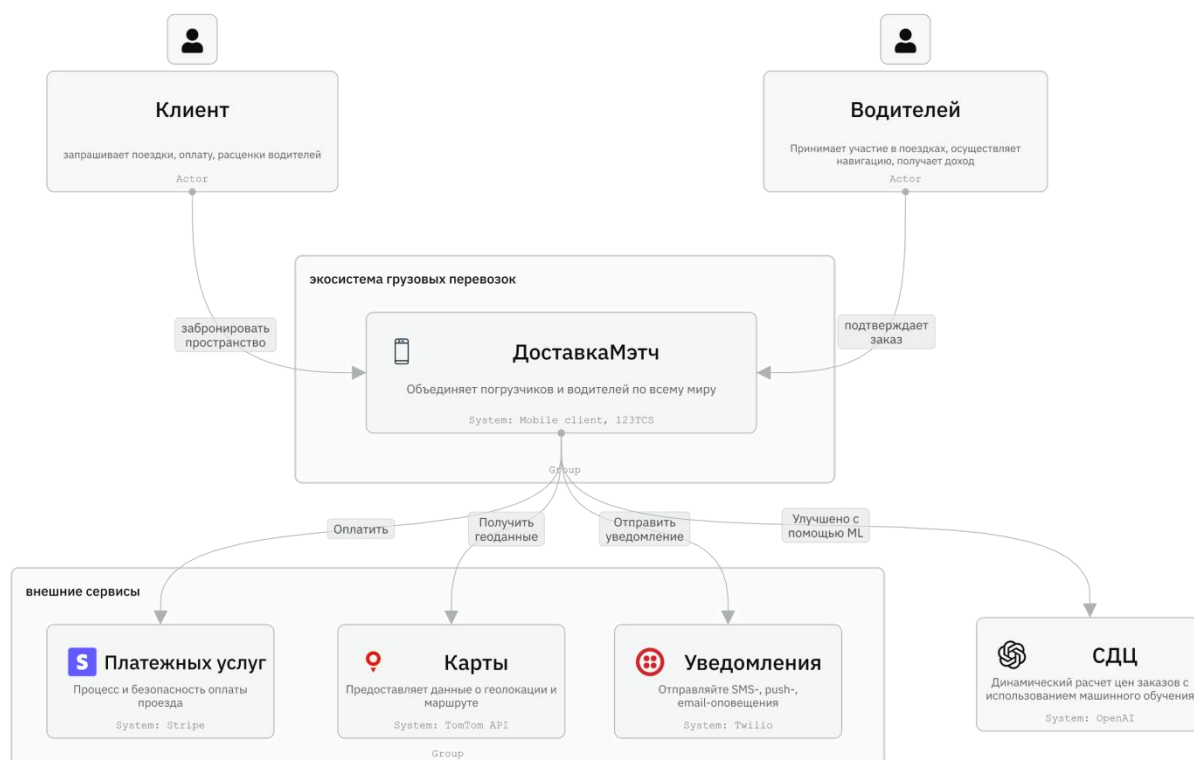


Рисунок 1.1 – Контекстная диаграмма системы DeliveryMatch

На рисунке 1.1 представлена контекстная диаграмма системы DeliveryMatch, которая показывает взаимодействие платформы с внешними сущностями. Основными акторами являются Клиент и Водитель-партнер. Платформа также взаимодействует с внешними сервисами: картографическими сервисами для построения маршрутов, платежными системами для обработки транзакций и сервисами уведомлений для информирования пользователей в реальном времени. Диаграмма демонстрирует границы системы и ее основные информационные потоки.

1.3 Анализ рынка и конкурентной среды

Рынок логистических услуг в России переживает значительную трансформацию. Это изменение обусловлено ростом электронной коммерции, изменением потребительских ожиданий и цифровизацией. Общий объем перевозок в стране продолжает расти. Автомобильный транспорт демонстрирует наибольшую динамику, усиливая свою значимость в коммерческих грузоперевозках. Аналитики отмечают, что ключевым

драйвером роста сегмента автомобильных грузоперевозок является развитие электронной коммерции и сопутствующей ей курьерской доставки [4].

На рынке представлено несколько ключевых моделей. Классические логистические операторы, такие как DHL и СДЭК, обладают собственной инфраструктурой. Они фокусируются на B2B-сегменте со стандартизированными тарифами и комплексными услугами. Их сила заключается в надежности, широком географическом охвате и опыте работы со сложными грузами.

Агрегаторы пассажирских перевозок с функцией доставки, такие как Uber и Яндекс Go, адаптировали модель такси для перемещения небольших грузов. Их главное преимущество — скорость подключения ближайшего свободного курьера. Однако их алгоритмы изначально не рассчитаны на специфические требования грузоперевозок.

Сервисы экспресс-доставки продуктов, например Самокат и Яндекс.Лавка, являются специализированными игроками. Они оптимизированы для сверхбыстрой доставки заказов с собственных или партнерских складов. Их сильная сторона — скорость и удобство для конечного потребителя.

Цифровые фрахтовые биржи, такие как Roolz, представляют собой B2B-платформы. Они автоматизируют поиск грузов и транспорта на дальние расстояния, часто предлагая функции тендеров и электронного документооборота. Они решают задачу оптимизации загрузки магистрального транспорта.

Несмотря на разнообразие предложений, анализ выявляет устойчивую тенденцию. Рынок движется от изолированных моделей к гибридным. Такие модели сочетают контроль над ключевыми процессами с гибкостью и технологическими возможностями внешних партнеров. Это позволяет компаниям масштабироваться, адаптироваться к сезонным колебаниям и внедрять передовые технологии без значительных капитальных затрат.

1.3.1 Анализ современных платформ (Uber, DHL, Яндекс Go, CDEK)

Для понимания конкурентного ландшафта и ниши проекта DeliveryMatch проведем сравнительный анализ подходов ключевых игроков. В таблице 1.1 систематизированы их основные характеристики и ограничения.

Таблица 1.1 — Сравнительный анализ логистических платформ

Платформа / Сервис	Ключевая бизнес-модель	Целевая аудитория / Груз	Сильные стороны	Слабые стороны / Технологические ограничения
Uber / Яндекс Go	Агрегация ближайших водителей для срочных поездок, включая перевозку посылок.	Частные лица, малый бизнес; мелкие, срочные посылки.	Высокая скорость подачи, простота для пользователя, динамическое ценообразование, плотная сеть в городах.	Алгоритм сопоставления учитывает только близость и доступность, игнорируя вес, объем и совместимость груза с ТС. Отсутствует планирование упаковки и многозадачных маршрутов.
DHL / CDEK	Классический логистический оператор с собственной/партнерской инфраструктурой.	Бизнес (B2B, B2C); грузы всех типов, включая международн.	Стандартизированные, надежные процессы, страхование, отслеживание, работа со сложными и габаритными грузами.	Меньшая гибкость и скорость для микрозаказов, относительно высокие тарифы для малых партий, слабая адаптивность маршрутов в реальном времени.
Яндекс.Лавка / Самокат	Экспресс-доставка товаров с собственных складов (dark stores) или партнерских магазинов.	Частные лица; продукты питания и товары повседневного спроса.	Экстремальная скорость доставки (минуты), оптимизация сборки заказа, удобный интерфейс, персонализация.	Жестко привязаны к собственной товарной матрице или складам. Не работают с произвольными грузами от клиента. Алгоритмы оптимизированы под пеших/велокурьеров, а не грузовой транспорт.
Цифровые биржи грузов (Roolz)	B2B-платформа для автоматического поиска и сопоставления грузов и транспорта, включая торги.	Компании-грузоотправители и профессиональные перевозчики; контейнерные, фуражные и иные партионные грузы.	Эффективное заполнение пробега, автоматизация документооборота, снижение порожнего пробега	Фокус на долгосрочные и крупные партии, а не на срочные городские микрозаказы. Процесс согласования может занимать часы или дни, что неприемлемо для курьерской доставки.

1.3.2 Выявление технологических разрывов и проблемных зон в существующих решениях

На основе проведенного анализа можно четко идентифицировать рыночный разрыв, который не закрывается существующими решениями, и ряд общих для отрасли технологических проблем.

Разрыв между «близостью» и «выполнимостью»: Агрегаторы такси (Uber, Яндекс Go) оптимизируют поиск по принципу минимального расстояния до водителя, полностью игнорируя физические параметры груза. Это приводит к ситуациям, когда водитель на легковом автомобиле получает заказ на перевозку холодильника, что заканчивается отказом, потерей времени и денег для всех участников. Проблема: отсутствие алгоритма предварительной проверки совместимости груза и транспортного средства на этапе подбора.

Разрыв между «скоростью» и «эффективностью»: Сервисы экспресс-доставки добиваются скорости за счет сети микроскладов и пеших курьеров, но не работают с грузами клиента. Классические перевозчики работают с любыми грузами, но не могут обеспечить скоростное выполнение мелких заказов. Проблема: отсутствие платформы, которая обеспечила бы скоростное (в течение часа) привлечение подходящего транспорта под произвольный груз клиента.

Неэффективное использование ресурсов и планирование: Многие платформы, особенно на рынке B2B-фрахта, борются с проблемой порожнего пробега. В городской же среде для агрегаторов актуальна проблема неоптимальных одиночных поездок, когда водитель везет один небольшой заказ, хотя мог бы по пути взять еще один-два в соседнем квартале. Проблема: слабое развитие алгоритмов динамической консолидации заказов (pooling и построения оптимальных многозвенных маршрутов (VRP в реальном времени для сетей частных перевозчиков.

Ручное планирование и вопросы безопасности: В малом бизнесе и при разовых перевозках вопросы размещения и крепления груза в кузове или

багажнике решаются вручную, что может привести к нарушению распределения веса, повреждению груза и созданию аварийных ситуаций. Проблема: отсутствие доступных инструментов для автоматизированного 3D-планирования загрузки (Bin Packing) и проверки стабильности груза (расчет центра масс).

Технологическая фрагментация и кадровый дефицит: Рынок сталкивается с общесистемными вызовами: дефицитом квалифицированных водителей, неготовностью многих компаний, особенно в регионах, к обязательной цифровизации (например, к переходу на электронные транспортные накладные), а также растущей стоимостью эксплуатации при стагнации тарифов. Эти факторы создают фон, на котором технологичное, удобное и экономичное решение для малых перевозок обладает высоким потенциалом спроса.

Таким образом, на рынке сформировалась четкая ниша для гибридного агрегатора, который соединит скорость и удобство потребительских приложений с интеллектуальным учетом логистических ограничений, характерных для B2B-сектора. Именно эту нишу и призван занять проект DeliveryMatch.

1.4 Формирование концепции агрегатора услуг доставки грузов DeliveryMatch

Концепция DeliveryMatch строится на синтезе лучших практик, выявленных в ходе анализа рынка, и направлена на преодоление описанных технологических разрывов. Это гибридная цифровая платформа-агрегатор, которая организует взаимодействие владельцев грузового транспорта (исполнителей) и клиентов, нуждающихся в срочной или плановой доставке товаров, с использованием интеллектуального алгоритмического ядра для оптимизации всех этапов процесса.

1.4.1 Определение ролей и бизнес-процессов

Система функционирует за счет взаимодействия трех ключевых сторон, каждая из которых решает свои задачи через единую платформу.

1. *Клиент (грузоотправитель)*: физическое или юридическое лицо. Его цели: быстро, недорого и безопасно доставить груз из точки А в точку Б. Ключевые бизнес-процессы со стороны клиента:

- создание заказа: указание параметров (адреса, вес, габариты, тип груза, желаемое время);
- получение и подтверждение предложения: просмотр рассчитанной платформой стоимости и выбранного ТС, подтверждение заказа;
- контроль и взаимодействие: отслеживание местоположения груза и статуса доставки в реальном времени, связь с водителем через защищенный чат, подтверждение получения, оплата и выставление оценки.

2. *Водитель-партнер (исполнитель)*: Частное лицо или микро-предприятие, владеющее подходящим транспортным средством (от легкового автомобиля с багажником до малотоннажного грузовика). Его цели: получить доход, минимизировать простой и холостые пробеги. Ключевые бизнес-процессы:

- регистрация и верификация: внесение данных о себе и ТС (техпаспорт, водительские права, фото автомобиля, точные параметры грузоподъемности и объема);
- управление доступностью: включение/выключение режима приема заказов;
- выбор и выполнение заказов: просмотр ленты доступных заказов, отфильтрованных под его ТС и местоположение; прием заказа; следование оптимизированному маршруту, построенному платформой; выполнение процедуры загрузки/выгрузки; подтверждение завершения;
- финансовый учет: контроль заработанных средств, вывод денег.

3. *Платформа DeliveryMatch (организатор и гарант)*: выступает в роли технологического посредника и гаранта сделки. Ее ключевые бизнес-процессы:

- сопоставление (matching): в реальном времени выполняет двойную проверку: 1) Geohash + k-NN для поиска ближайших водителей;

- greedyFeasibilityCheck (задача о рюкзаке) для проверки соответствия груза параметрам ТС;
- оптимизация маршрута и консолидация: для выбранного водителя строит оптимальный маршрут (алгоритмы VRP), при возможности добавляя в него другие подходящие заказы (pooling) с близкими точками погрузки/выгрузки;
- планирование загрузки: генерирует для водителя схему размещения груза в кузове (алгоритм Bin Packing) с расчетом центра масс для обеспечения безопасности;
- расчет стоимости и обработка платежей: применяет динамическую тарифную модель (расстояние, вес, объем, спрос), обеспечивает безопасные расчеты между сторонами;
- контроль и коммуникация: обеспечивает трекинг, нотификации, поддержку спорных ситуаций.

На рисунке 1.2 представлена общая схема бизнес-процесса, отображающая взаимодействие клиента и водителя с платформой. Оба пользователя подключаются к API Gateway через протокол HTTP. Сервис поездок (Trip Service) имеет подключение к базе данных Redis, в которой хранятся активные водители, готовые к принятию заказов. Когда клиент отправляет параметры своего заказа, сервис поездок обращается к сервису прогнозирования времени прибытия (ETA Prediction Service), который рассчитывает ориентировочную стоимость доставки. Кроме того, параметры заказа передаются в сервис сопоставления (Matching Service), где с учётом заданных ограничений отбираются пять транспортных средств, наиболее подходящих под данный заказ. Затем клиенту предоставляется возможность выбрать наиболее подходящий вариант из пяти предложенных. После выбора клиентом подходящего водителя сервис платежей (Payment Service) выполняет финансовую транзакцию, а сервис уведомлений (Notification Service) отправляет оповещения обоим пользователям.

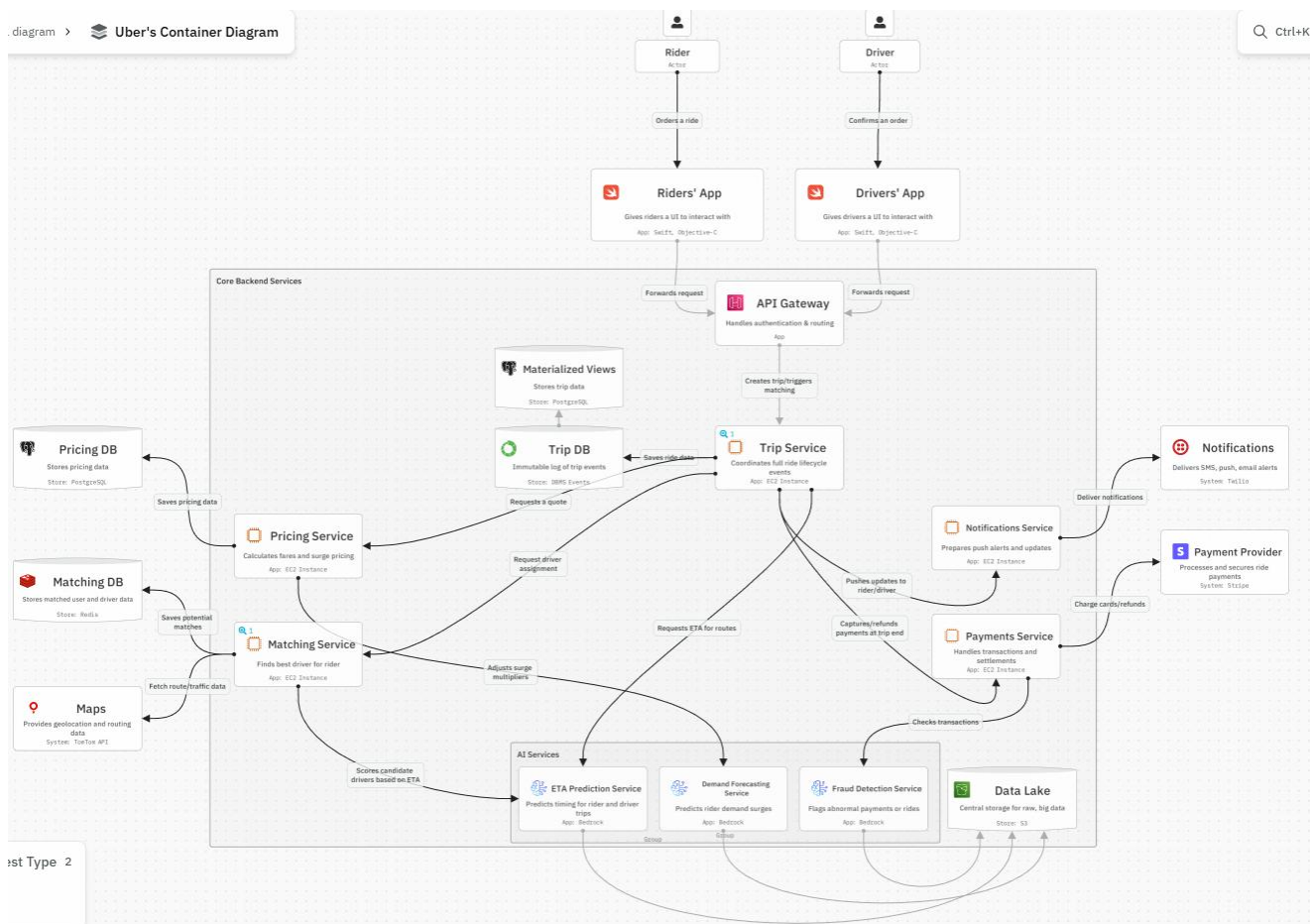


Рисунок 1.2 – Диаграмма бизнес-процессов системы DeliveryMatch

Все полученные данные сохраняются для последующего анализа и использования в целях улучшения работы системы.

1.4.2 Обоснование гибридной модели агрегатора грузоперевозок с поддержкой пакетных заказов

Концепция DeliveryMatch является гибридной по нескольким фундаментальным признакам, что и составляет ее основное конкурентное преимущество.

Гибридность технологической логики: Платформа синтезирует подходы из разных сегментов:

- от сервисов такси и экспресс-доставки заимствуется принцип работы в режиме реального времени с большим пулом частных исполнителей, скорость сопоставления и удобство мобильного интерфейса;
- от классической логистики и B2B-бирж заимствуется строгий учет физических параметров груза и транспорта, а также подход к

оптимизации загрузки и маршрутов для повышения экономической эффективности;

- концепция гибридных платформ, сочетающих элементы маркетплейсов и классических логистических операторов, рассматривается в современных исследованиях как наиболее перспективная модель для рынка городских грузоперевозок [5].

Гибридность целевой аудитории: Система в равной степени ориентирована как на B2C-сегмент (частные лица, переезды, покупки с рук), так и на микро- и малый B2B-сегмент (магазины, кафе, мелкие производители, курьерские службы). Это позволяет диверсифицировать спрос и создать более устойчивую экономику платформы.

Поддержка пакетных (bulk) заказов: Это критически важный функционал для малого бизнеса. Возможность загрузить Excel-файл со списком из 10-50 посылок для доставки по разным адресам автоматически запускает сложную логистическую оптимизацию.

Функции платформы:

- автоматическая кластеризация заказов по географическим зонам;
- распределение их между несколькими подходящими водителями;
- построение единого оптимального маршрута (задача VRP) для каждого водителя;
- формирование индивидуальных инструкций по загрузке для каждой машины.

Таким образом, для бизнес-клиента сложный логистический процесс сводится к загрузке файла, а платформа берет на себя всю операционную работу, что напрямую соответствует тренду на аутсорсинг логистических функций для оптимизации расходов.

Внедрение такой гибридной модели позволяет DeliveryMatch занять уникальную рыночную позицию, предложив клиентам уровень удобства, сопоставимый с вызовом такси, и уровень надежности и эффективности,

характерный для профессиональных логистических услуг, при этом преодолевая ключевые технологические ограничения существующих игроков.

1.4.3 Моделирование функциональных требований в виде вариантов использования

Для формализации функциональных требований к разрабатываемой системе был использован язык моделирования UML (Unified Modeling Language). На основе анализа предметной области и выявленных бизнес-процессов были построены диаграммы вариантов использования (use case diagrams), которые позволяют наглядно представить взаимодействие внешних акторов с системой и ее основные функции.

В системе DeliveryMatch выделены три ключевых актора:

- **клиент (грузоотправитель)** – физическое или юридическое лицо, нуждающееся в доставке груза;
- **водитель-партнер (исполнитель)** – владелец транспортного средства, зарегистрированный на платформе для выполнения заказов;
- **система (автоматизированные процессы)** – внутренние модули платформы, которые выполняют функции без непосредственного участия человека.

На рисунках 1.3-1.5 представлены диаграммы вариантов использования для каждого из акторов.

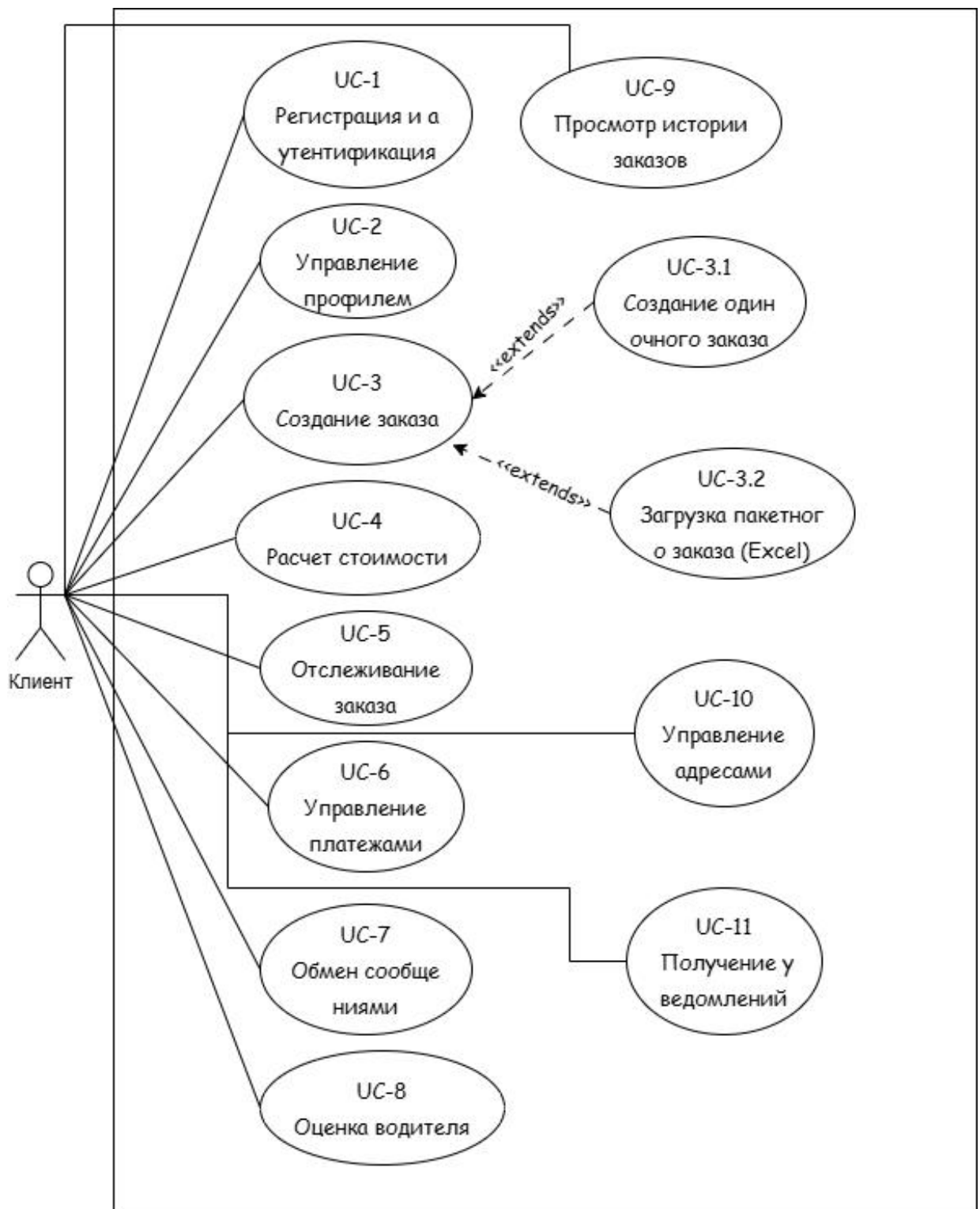


Рисунок 1.3 – Диаграмма вариантов использования для актора «Клиент»

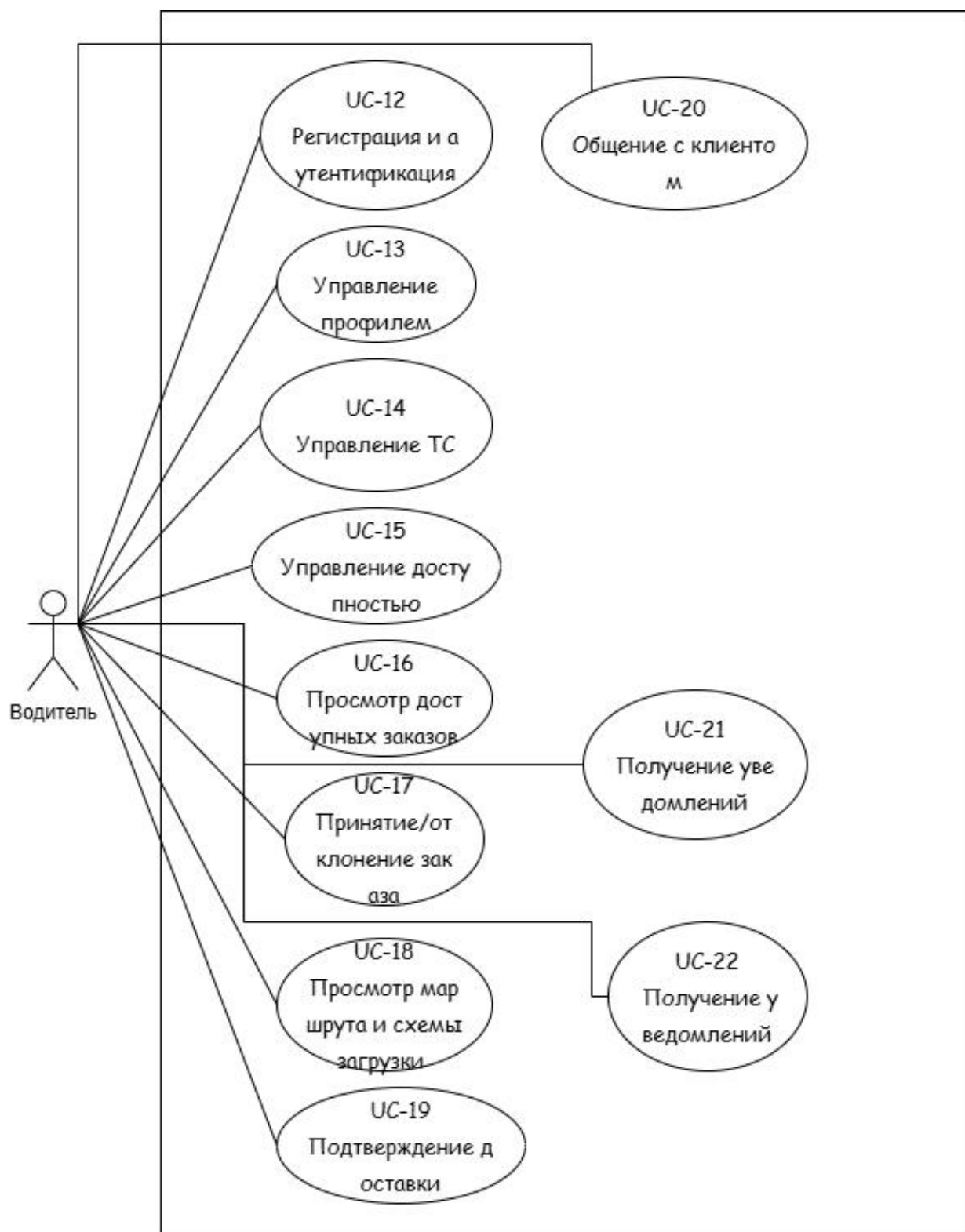


Рисунок 1.4 – Диаграмма вариантов использования для актора «Водитель-партнер»

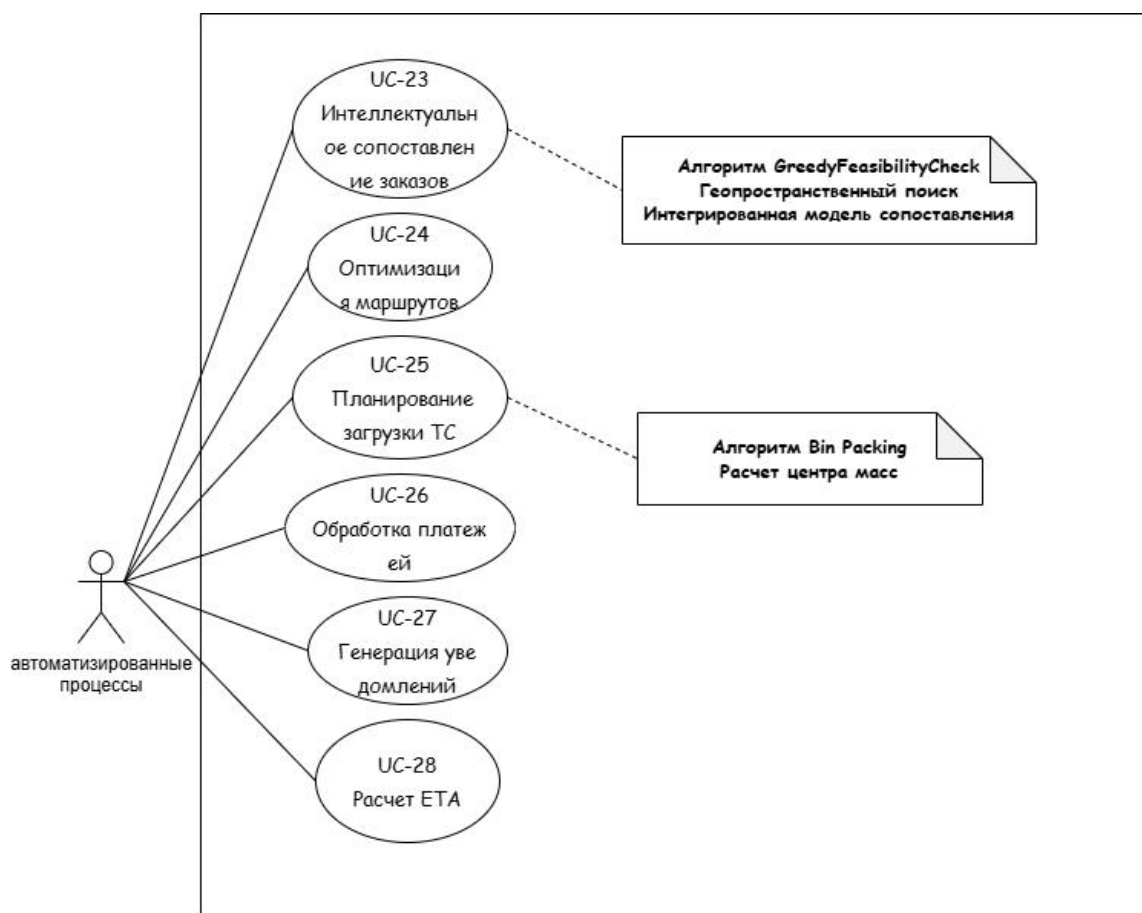


Рисунок 1.5 – Диаграмма вариантов использования для автоматизированных процессов системы

1.4.3.1 Детальная спецификация ключевых вариантов использования

Для наиболее критичных и сложных сценариев взаимодействия пользователя с системой разработаны детальные спецификации вариантов использования. Данные спецификации описывают предварительные и выходные условия, основные и альтернативные потоки событий, а также возможные исключения — всё это представлено в таблицах 1.2–1.6 ниже.

Таблица 1.2 — Вариант использования «Создание заказа»

Параметр	Описание
Идентификатор ВИ	UC-3
Наименование	Создание заказа
Автор	Мвале Лисон
Дата создания	15.03.2025

Продолжение таблицы 1.2

Основное действующее лицо	Клиент
Дополнительное действующее лицо	Система (модуль управления заказами), Система (модуль интеллектуального сопоставления)
Описание	Клиент инициирует создание нового заказа на доставку груза, указывая характеристики груза, адрес отправления и назначения, а также предпочтительное время доставки. Система выполняет проверку корректности данных, рассчитывает стоимость услуги и запускает процесс подбора подходящего водителя.
Приоритет	Высокий
Условие-триггер	Нажатие клиентом кнопки «Создать заказ» в личном кабинете или на главной странице платформы.
Предварительные условия	PRE-1: Клиент зарегистрирован и авторизован в системе. PRE-2: Профиль клиента успешно верифицирован. PRE-3: Система функционирует в штатном режиме и принимает новые заказы. PRE-4: Клиент имеет сохранённый адрес либо готов ввести новый.
Выходные условия	POST-1: Заказ создан и сохранён в базе данных со статусом «В поиске водителя». POST-2: Стоимость доставки рассчитана согласно тарифной модели. POST-3: Заказ помещён в очередь сопоставления. POST-4: Клиент уведомлён об успешном создании заказа и ориентировочной стоимости. POST-5: Запущен процесс интеллектуального подбора водителя.

Продолжение таблицы 1.2

Основные потоки	Клиент переходит в раздел создания заказа. Система отображает форму с полями ввода параметров заказа. Клиент заполняет обязательные поля. Система выполняет онлайн-валидацию введённых данных. Клиент инициирует расчёт стоимости. Система вычисляет стоимость доставки с учётом параметров заказа. Клиент подтверждает создание заказа. Система сохраняет заказ, присваивает идентификатор и статус. Запускается алгоритм сопоставления с водителями. 10. Клиент перенаправляется на страницу отслеживания заказа.
Альтернативные потоки	Пакетное создание заказов — загрузка файла с перечнем отправок и автоматическое формирование группы заказов. 1.2 Использование сохранённых адресов — выбор адресов из профиля клиента с автоматическим заполнением формы.
Исключения	E1 Некорректные параметры груза — превышение допустимых ограничений. E2 Недоступный адрес доставки — адрес вне зоны обслуживания платформы.
Бизнес-правила	Минимальная стоимость заказа определяется тарифной политикой платформы. Создание заказа доступно только верифицированным пользователям. Бесплатная отмена возможна в течение 5 минут после создания.
Другая информация	Поддерживается добавление фотографий груза. Для хрупких и опасных грузов применяются специальные условия.
Предположения	Параметры груза указаны корректно, адреса поддаются геокодированию, картографические сервисы доступны.

Таблица 1.3 — Вариант использования «Отслеживание заказа»

Параметр	Описание
Идентификатор ВИ	UC-5

Продолжение таблицы 1.3

Наименование	Отслеживание заказа
Автор	Мвале Лисон
Дата создания	15.03.2025
Основное действующее лицо	Клиент
Дополнительные действующие лица	Система (модуль трекинга), Водитель
Описание	Клиент осуществляет мониторинг статуса и местоположения заказа в режиме реального времени через веб-интерфейс. Система отображает маршрут доставки, текущее положение водителя, прогнозируемое время прибытия и историю перемещений.
Приоритет	Высокий
Условие-триггер	Клиент переходит на страницу активного заказа либо обновляет страницу отслеживания.
Предварительные условия	PRE-1: Заказ существует и имеет статус «Принят» или «В пути». PRE-2: Водитель активировал GPS-трекинг. PRE-3: Модуль трекинга функционирует и получает данные о местоположении.
Выходные условия	POST-1: Текущее местоположение водителя отображено на карте. POST-2: Клиенту показано прогнозируемое время прибытия (ETA). POST-3: История статусов заказа обновлена. POST-4: Маршрут доставки визуализирован.
Основные потоки	Клиент открывает раздел «Мои заказы». Система отображает список активных и завершённых заказов. Клиент выбирает активный заказ. Система загружает страницу отслеживания с интерактивной картой, текущей позицией водителя, прогресс-баром этапов доставки, ETA, контактной информацией водителя и кнопкой связи. Система обновляет позицию водителя каждые 10 секунд. Клиент наблюдает перемещение в режиме реального времени. 7. При изменении статуса заказа система обновляет интерфейс и отправляет уведомление.

Продолжение таблицы 1.3

Альтернативные потоки	Отслеживание без авторизации Клиент использует публичную ссылку для отслеживания. Система отображает ограниченный набор информации. Возврат к основному потоку, шаг
Исключения	2.0 E1 Потеря GPS-сигнала 1. Водитель теряет сигнал GPS или отключает трекинг. 2. Система отображает последнюю известную позицию и предупреждение. 3. ETA рассчитывается на основе последних доступных данных. 3.0 E2 Отклонение от маршрута 1. Водитель существенно отклоняется от запланированного маршрута.
Исключения	2. Система фиксирует отклонение и уведомляет клиента. 3. При критическом отклонении предлагается связаться со службой поддержки.
Бизнес-правила	– Данные о местоположении обновляются не реже одного раза в 10 секунд.– ETA пересчитывается при изменении дорожной обстановки.– История перемещений хранится в течение 24 часов после доставки.– Клиент может делиться ссылкой на отслеживание с третьими лицами.
Другая информация	– На карте отображаются точки погрузки и выгрузки.– Доступны уведомления о приближении водителя.– Для конфиденциальных грузов возможен режим ограниченного трекинга.
Предположения	– У водителя стабильное интернет-соединение.– GPS-датчик водителя исправен.– Картографический сервис функционирует корректно.

Таблица 1.4 — ВИ «Принятие/отклонение заказа»

Параметр	Описание
Идентификатор ВИ	UC-17
Наименование	Принятие/отклонение заказа
Автор	Мвале Лисон
Дата создания	15.03.2025

Продолжение таблицы 1.4

Основное действующее лицо	Водитель
Дополнительное действующее лицо	Система (Модуль сопоставления), Клиент
Описание	Водитель просматривает предложенные заказы в своей ленте, анализирует параметры (расстояние, оплата, соответствие ТС) и принимает или отклоняет заказ. Система фильтрует заказы на основе характеристик ТС водителя и его текущего местоположения.
Приоритет	Высокий
Условие-триггер	В системе появляется новый заказ, подходящий для водителя по параметрам ТС и местоположению.
Предварительные условия	PRE-1: Водитель находится в режиме «Доступен». PRE-2: Профиль водителя и данные ТС верифицированы. PRE-3: В системе есть заказы, соответствующие параметрам ТС водителя. PRE-4: Рейтинг водителя выше минимального порога.
Выходные условия	POST-1: Статус заказа изменен на «Принят» или остается «В поиске». POST-2: Водитель получил полную информацию о заказе и маршруте. POST-3: Клиент уведомлен о принятии/отклонении заказа. POST-4: Для принятого заказа сгенерирован маршрут и план загрузки. POST-5: Обновлен статус доступности водителя.
Основные потоки	Система уведомляет водителя о новом заказе. Водитель открывает раздел «Доступные заказы». Система отображает карточку заказа (адреса, расстояние, оплата, параметры груза, индикатор соответствия ТС, временные ограничения). Водитель анализирует заказ. Водитель принимает или отклоняет заказ. При принятии: резервирование, генерация маршрута и плана загрузки, уведомление клиента. 7. При отклонении: заказ возвращается в очередь.

Продолжение таблицы 1.4

Альтернативные потоки	Автоматическое принятие заказа — система принимает заказ автоматически при включенном режиме автопринятия. 1.2 Просмотр деталей перед принятием — водитель открывает расширенную информацию с картой маршрута.
Исключения	2.0 E1 Заказ уже принят другим водителем. 3.0 E2 Низкий рейтинг водителя.
Бизнес-правила	– Время принятия решения — не более 60 секунд.— Частые отказы снижают рейтинг.— Отмена принятого заказа возможна только с причиной и штрафом.— Учитывается рейтинг при распределении заказов.
Другая информация	– Отображается история принятых и отклоненных заказов.— Доступна статистика принятия.
Предположения	– Данные о местоположении водителя актуальны.— Параметры ТС указаны корректно.

Таблица 1.5 — ВИ «Просмотр маршрута и схемы загрузки»

Параметр	Описание
Идентификатор ВИ	UC-18
Наименование	Просмотр маршрута и схемы загрузки
Автор	Мвале Лисон
Дата создания	15.03.2025
Основное действующее лицо	Водитель
Дополнительное действующее лицо	Система (Модуль оптимизации маршрутов), Система (Модуль планирования загрузки)
Описание	После принятия заказа водитель получает детализированный маршрут и 3D-схему загрузки ТС с учетом дорожной ситуации и безопасности распределения груза.
Приоритет	Высокий

Продолжение таблицы 1.5

Условие-триггер	Принятие заказа водителем.
Предварительные условия	PRE-1: Заказ принят. PRE-2: Сгенерирован маршрут и схема загрузки. PRE-3: Доступны картографические данные.
Выходные условия	POST-1: Маршрут визуализирован. POST-2: Отображена 3D-схема загрузки. POST-3: Рассчитано ETA. POST-4: Водитель подтвердил готовность.
Основные потоки	1. Открывается экран «Маршрут и загрузка». 2. Отображаются вкладки «Маршрут» и «Загрузка». 3. Водитель изучает маршрут и схему. 4. Возможны масштабирование, вращение модели, просмотр деталей. 5. Подтверждение готовности.
Альтернативные потоки	1.1 Ручная корректировка маршрута. 1.2 Переключение на 2D-вид загрузки.
Исключения	2.0 E1 Нет данных о пробках. 3.0 E2 Невозможно сгенерировать схему загрузки.
Бизнес-правила	– Маршрут оптимизируется по времени и расстоянию.– Центр масс должен быть безопасным.– Все изменения фиксируются.
Другая информация	– Возможна печать схемы.– Поддерживается темный режим.
Предположения	– Параметры груза и ТС указаны корректно.

Таблица 1.6 — ВИ «Интеллектуальное сопоставление заказов»

Параметр	Описание
Идентификатор ВИ	UC-23

Продолжение таблицы 1.6

Наименование	Интеллектуальное сопоставление заказов
Автор	Мвале Лисон
Дата создания	15.03.2025
Основное действующее лицо	Система (Модуль сопоставления)
Дополнительное действующее лицо	Водитель, Клиент
Описание	Автоматическое сопоставление заказов с водителями на основе геопоиска, проверки выполнимости и модели ранжирования.
Приоритет	Высокий
Условие-триггер	Появление нового заказа или изменение доступности водителя.
Выходные условия	POST-1: Заказ сопоставлен. POST-2: Водители уведомлены. POST-3: Результаты сохранены для аналитики.
Предварительные условия	PRE-1: Заказ в статусе «В поиске». PRE-2: Есть доступные водители. PRE-3: Алгоритмы работают корректно.
Основные потоки	1. Геопоиск ближайших водителей. 2. Проверка выполнимости (вес, объем, тип груза). 3. Ранжирование по формуле Score. 4. Уведомление топ-3 водителей. 5. Логирование результатов.
Альтернативные потоки	1.1 Сопоставление пакетных заказов.
Исключения	2.0 E1 Нет подходящих водителей. 3.0 E2 Ошибка алгоритма.

Продолжение таблицы 1.6

Бизнес-правила	– Время сопоставления ≤ 100 мс.– Приоритет водителям с высоким рейтингом.– Целевой KPI — 95% успешных сопоставлений.
Другая информация	– Коэффициенты настраиваются через A/B-тесты.– Возможен ручной оверрайд администратором.
Предположения	– Геоданные актуальны.– Параметры ТС корректны.

1.4.3.2 Функциональные требования к системе

На основе детализированных вариантов использования был сформирован перечень функциональных требований (ФТ) к системе DeliveryMatch, как показано в таблице 1.7. Требования представлены в табличной форме и включают идентификатор, название функции, роль, которой она доступна, а также краткое описание.

Таблица 1.7 – Функциональные требования к системе DeliveryMatch

ID	Название функции	Роль	Описание
FR-01	Регистрация пользователя	Гость	Возможность создать учетную запись клиента или водителя с использованием email и пароля или через OAuth-провайдеров (Google, VK).
FR-02	Авторизация и аутентификация	Пользователь	Вход в систему с использованием JWT-токенов, обеспечение безопасности и проверки подлинности пользователя.
FR-03	Управление профилем водителя	Водитель	Заполнение и редактирование персональных данных, загрузка документов (паспорт, права), добавление информации о транспортных средствах (марка, модель, госномер, грузоподъемность, объем, тип кузова).

Продолжение таблицы 1.7

FR-04	Управление профилем клиента	Клиент	Заполнение и редактирование личных данных, ведение адресной книги (сохранение часто используемых адресов), настройка предпочтений.
FR-05	Создание одиночного заказа	Клиент	Публикация заказа на доставку с указанием параметров груза (вес, объем, тип), адресов погрузки и доставки, желаемого времени.
FR-06	Пакетное создание заказов	Бизнес-клиент	Загрузка Excel- или CSV-файла со списком заказов, их автоматическое распознавание, валидация и постановка в очередь на обработку.
FR-07	Расчет стоимости доставки	Система	Автоматический расчет предварительной стоимости доставки на основе расстояния, веса, объема и текущего спроса (динамическое ценообразование).
FR-08	Просмотр ленты доступных заказов	Водитель	Просмотр списка заказов, отфильтрованных системой в соответствии с характеристиками ТС водителя и его текущим местоположением.
FR-09	Принятие заказа	Водитель	Возможность принять предложенный заказ. Принятие резервирует заказ за водителем.

Продолжение таблицы 1.7

FR-10	Интеллектуальное сопоставление	Система	Автоматический подбор оптимального водителя для заказа на основе геопоиска, проверки совместимости (вес, объем) и ранжирования.
FR-11	Генерация маршрута	Система	Построение оптимального маршрута для водителя с учетом всех точек погрузки и выгрузки, дорожной обстановки и пробок в реальном времени.
FR-12	Планирование загрузки	Система	Генерация 2D/3D-схемы размещения грузов в кузове ТС с алгоритмическим расчетом центра масс для обеспечения безопасности.
FR-13	Отслеживание заказа в реальном времени	Клиент	Просмотр текущего местоположения водителя, статуса заказа и ЕТА на интерактивной карте с обновлением через WebSocket.
FR-14	Обновление геопозиции	Система, Водитель	Периодическая отправка координат водителя на сервер для обеспечения работы трекинга.
FR-15	Управление статусами заказа	Водитель	Изменение статуса заказа («На погрузке», «В пути», «На выгрузке», «Выполнен») для информирования клиента и системы.
FR-16	Обработка платежей	Система	Интеграция с платежными сервисами для безопасного списания средств с клиента и начисления выплат водителю после успешного завершения заказа.

Продолжение таблицы 1.7

FR-17	Система уведомлений	Система	Отправка push-уведомлений, email- и SMS-оповещений о ключевых событиях (создание заказа, принятие, прибытие).
FR-18	История заказов и платежей	Клиент, Водитель	Просмотр полной истории выполненных заказов, детализации поездок и истории финансовых операций.
FR-19	Ведение рейтинга и отзывов	Клиент, Водитель	Возможность оставить отзыв и поставить оценку водителю (клиенту) после выполнения заказа.
FR-20	Административная панель	Администратор	Доступ к дашборду с метриками системы (MAR, SRT, OPR), управление пользователями (модерация водителей), просмотр логов и аналитики.
FR-21	Управление алгоритмами	Администратор	Возможность включать/отключать отдельные этапы алгоритмов (например, проверку на объем) и настраивать их параметры.
FR-22	Геокодирование адресов	Система	Преобразование текстовых адресов, введенных клиентом, в географические координаты для построения маршрутов и поиска водителей.

1.4.3.3 Пользовательские истории (User Stories)

Для более гибкого описания функциональности с точки зрения пользователя были сформулированы пользовательские истории (user stories),

представленные в таблице 1.8. Они описывают, какую ценность каждая функция несет для конкретного актора системы.

Таблица 1.8 – Пользовательские истории для системы DeliveryMatch

Идентификатор ПИ	Пользовательская история (User Story)
US-01	Я, как клиент, хочу иметь возможность зарегистрироваться в системе, используя email или аккаунт в соцсети, чтобы получить доступ к функциям платформы.
US-02	Я, как клиент, хочу создать заказ на доставку, указав вес, габариты и тип груза, а также адреса отправления и назначения, чтобы система могла найти для меня подходящего водителя.
US-03	Я, как клиент, хочу видеть предварительную стоимость доставки до подтверждения заказа, чтобы понимать свои расходы.
US-04	Я, как клиент, хочу загрузить Excel-файл со списком заказов для пакетной доставки, чтобы сэкономить время на вводе данных.
US-05	Я, как клиент, хочу отслеживать местоположение моего заказа и водителя в реальном времени на карте, чтобы знать, когда точно прибудет груз.
US-06	Я, как клиент, хочу получать уведомления об изменении статуса моего заказа (принят, в пути, прибыл), чтобы быть в курсе событий.
US-07	Я, как клиент, хочу просматривать историю всех моих заказов и детали по каждому из них, чтобы контролировать расходы.
US-08	Я, как клиент, хочу иметь возможность оставить отзыв и оценку водителю после доставки, чтобы помочь другим пользователям и платформе.

Продолжение таблицы 1.8

US-09	Я, как водитель, хочу зарегистрироваться и добавить информацию о своем транспортном средстве (грузоподъемность, объем, тип кузова), чтобы получать релевантные заказы.
US-10	Я, как водитель, хочу управлять своим статусом (на линии / офлайн), чтобы не получать предложения, когда я занят.
US-11	Я, как водитель, хочу получать предложения только по тем заказам, которые соответствуют грузоподъемности и объему моего автомобиля, чтобы не тратить время на неподходящие рейсы.
US-12	Я, как водитель, хочу видеть в ленте заказов карточку с ключевой информацией: адреса, расстояние, оплата, чтобы быстро принять решение.
US-13	Я, как водитель, хочу иметь ограниченное время на принятие решения по заказу (например, 60 секунд), чтобы предложение уходило другим водителям, если я не готов его взять.
US-14	Я, как водитель, хочу видеть в приложении оптимизированный маршрут с учетом пробок и ЕТА для каждой точки, чтобы эффективно планировать свое время.
US-15	Я, как водитель, хочу получать сгенерированную системой 3D-схему размещения груза, чтобы безопасно и оптимально загрузить автомобиль.
US-16	Я, как водитель, хочу подтверждать выполнение каждого этапа доставки (погрузка, в пути, выгрузка), чтобы система корректно обновляла статус для клиента.
US-17	Я, как водитель, хочу видеть статистику моих поездок и заработка в личном кабинете, чтобы отслеживать свои доходы.

Продолжение таблицы 1.8

US-18	Я, как администратор, хочу видеть дашборд с ключевыми метриками системы (MAR – точность сопоставления, SRT – время отклика, OPR – коэффициент консолидации), чтобы контролировать работу платформы.
US-19	Я, как администратор, хочу иметь возможность просматривать список всех пользователей и заказов, чтобы модерировать и решать спорные ситуации.
US-20	Я, как администратор, хочу просматривать логи работы алгоритмов сопоставления и маршрутизации, чтобы диагностировать возможные ошибки.
US-21	Я, как администратор, хочу иметь возможность вручную назначать водителя на заказ в исключительных случаях, чтобы гарантировать выполнение.
US-22	Я, как администратор, хочу просматривать историю действий всех администраторов в системе, чтобы обеспечить аудит и безопасность.

1.4.3.4 Требования к графическому интерфейсу (UI/UX)

Графический интерфейс пользователя (GUI) должен обеспечивать интуитивно понятное, эффективное и адаптивное взаимодействие с системой для всех категорий пользователей. При разработке интерфейсов учитываются принципы минимализма, приоритизации информации и консистентности.

Общие требования к интерфейсу:

- адаптивность: интерфейс должен корректно отображаться на различных устройствах: десктопах, планшетах и мобильных телефонах. Для водителей предпочтительной является мобильная версия, оптимизированная для использования "на ходу";
- консистентность: все элементы интерфейса (кнопки, формы, карточки, иконки) должны быть выполнены в едином стиле с использованием общей дизайн-системы (например, на основе библиотеки компонентов Shadcn/UI);

- интуитивность: навигация должна быть логичной и простой. Количество кликов для выполнения ключевого действия (например, создания заказа) должно быть минимальным;
- информативность: пользователь должен всегда понимать, на каком этапе находится процесс и что произойдет после его следующего действия. Обратная связь от системы (подтверждения, уведомления об ошибках) должна быть четкой и понятной;
- доступность: интерфейс должен учитывать базовые требования доступности (контрастность текста, возможность навигации с клавиатуры).

1.4.3.5 Требования к интерфейсу для различных ролей

1. Клиентский веб-интерфейс.

Уровень вложенности для клиента не должен превышать двух переходов от главной страницы для выполнения целевого действия. Клиенту доступны следующие основные разделы.

Раздел «Главная страница / быстрое создание заказа» содержит форму для быстрого ввода адресов, слайдеры или поля для указания веса и габаритов груза, а также автоматический расчёт стоимости.

Раздел «Личный кабинет» позволяет клиенту редактировать личные данные и вести адресную книгу для сохранения часто используемых адресов.

Раздел «История заказов» представляет собой таблицу или список всех заказов клиента с отображением статуса, даты, адресов и стоимости. Предусмотрена фильтрация по дате и статусу.

Раздел «Текущий заказ» — это страница активного заказа с виджетом отслеживания в реальном времени, включающим карту, прогресс-бар, ЕТА и контактные данные водителя.

2. Мобильное приложение водителя (или адаптивный веб-интерфейс).

Интерфейс оптимизирован для использования во время движения. Элементы управления должны быть крупными и удобными для нажатия.

Водителю доступны следующие основные разделы.

Экран доступности (главный экран) включает крупный переключатель статуса («На линии / Офлайн») и карту с отображением тепловой карты спроса или доступных заказов в районе.

Лента заказов (при статусе «На линии») отображает карточки доступных заказов, отфильтрованных системой по параметрам транспортного средства водителя. Каждая карточка заказа должна содержать адреса подачи и доставки (улицы и номера домов), общее расстояние, предварительную оплату за рейс, а также иконки соответствия груза (вес/объём) параметрам автомобиля. Таймер ограничивает время на принятие решения по заказу.

Экран выполнения заказа (после принятия) содержит две вкладки. Вкладка «Маршрут» включает интерактивную карту с проложенным маршрутом, пошаговую навигацию и отображение ЕТА до следующей точки, а также кнопки для смены статуса («Прибыл на погрузку», «Загрузился», «Выезжаю на доставку», «Завершить рейс»). Вкладка «Загрузка» отображает сгенерированную системой 3D-схему размещения груза с возможностью вращения и масштабирования, а также 2D-проекцию. Также показывается расчётный центр масс.

Личный кабинет водителя включает статистику заработка — отображение дохода за день, неделю или месяц в виде графиков и сумм, а также историю поездок — список завершённых заказов с детализацией по маршруту и выплатам.

1.5 Постановка задачи и определение целей разработки

Основываясь на проведенном анализе рынка, выявленных технологических разрывах и сформированной концепции гибридной платформы, можно перейти к формальной постановке задачи дипломного проекта. Исходной посылкой является противоречие между растущим спросом на гибкие и экономичные городские грузоперевозки и отсутствием на рынке технологического решения, которое комплексно решало бы задачи интеллектуального подбора, оптимизации и обеспечения безопасности.

1.5.1 Формализация ключевых проблем

В рамках проекта необходимо решить три взаимосвязанные фундаментальные проблемы, ограничивающие эффективность существующих решений:

- проблема некорректного сопоставления заказа и исполнителя, ведущая к отказам и потерям времени. Текущие алгоритмы агрегаторов (например, основанные на принципе ближайшего водителя) сопоставляют заказ O_i и водителя D_j преимущественно по одному параметру — географическому расстоянию $\text{dist}(O_i, D_j)$. При этом полностью игнорируются физические ограничения транспортного средства D_j , заданные его грузоподъемностью $W_{\max}(D_j)$ и полезным объемом $V_{\max}(D_j)$, и параметры груза — вес $w(O_i)$ и объем $v(O_i)$. Это приводит к ситуации, когда формально подобранный исполнитель физически не может выполнить заказ, что описывается условием:

$$w(O_i) > W_{\max}(D_j) \vee v(O_i) > V_{\max}(D_j). \quad (1)$$

- следствие: высокий процент отказов от заказов после их принятия, неэффективное использование времени водителя, негативный пользовательский опыт и экономические потери;
- проблема субоптимального использования транспортных ресурсов и статического планирования. Существующие системы либо не предлагают консолидацию нескольких заказов (ведут к одиночным, часто недогруженным рейсам), либо используют упрощенные или статические алгоритмы маршрутизации. В условиях динамического потока заказов $\{O_1, O_2, \dots, O_n\}$ с координатами погрузки P_k и выгрузки D_k необходимо в реальном времени формировать для водителя D_j набор заказов $R \subseteq \{O_i\}$ и последовательность их посещения $\pi(R)$, минимизирующую общий пробег $L(\pi(R))$ при соблюдении временных окон и ограничений вместимости ТС. Отсутствие такой динамической

оптимизации ведет к увеличению пробега, расходов на топливо и выбросов;

- проблема ручного, небезопасного планирования загрузки транспортного средства заключается в том, что процесс размещения грузов в кузове остаётся за пределами автоматизации большинства платформ и отдан на откуп исполнителю, что создаёт риски неоптимального использования объема (низкий коэффициент заполнения), несоблюдения условий перевозки (хрупкие грузы под тяжелыми), а также нарушения устойчивости транспортного средства из-за смещения центра масс за допустимые пределы. В связи с этим необходимо автоматизировать расчёт схемы размещения — решение задачи трёхмерного bin packing, где «контейнерами» являются зоны кузова, а «предметами» — грузовые места, с дополнительным ограничением на стабильность.

Таким образом, главная задача проекта заключается в разработке и реализации архитектуры и алгоритмического ядра веб-платформы, которая обеспечивает интеллектуальное, безопасное и экономически эффективное сопоставление заказов на перевозку грузов с парком частных перевозчиков в режиме, близком к реальному времени.

1.5.2 Определение целевых показателей эффективности

Успешность решения поставленной задачи будет оцениваться по следующим ключевым показателям эффективности (*KPI*), сформулированным для системы в целом:

- процент успешного первоначального сопоставления (Matching Accuracy Rate, MAR): доля заказов, для которых первый предложенный системой водитель успешно принимает и выполняет заказ без отказов по причине несоответствия ТС. Целевое значение: не менее 95%. Показатель отражает качество работы алгоритмов GreedyFeasibilityCheck и геопространственного поиска.

$$MAR = (N_{\text{успешных}} / N_{\text{всех}}) \cdot 100\% \quad (2)$$

- среднее время отклика системы (System Response Time, SRT): время с момента создания/обновления заказа клиентом до момента появления этого заказа в ленте доступных заказов у подходящих водителей. Целевое значение: не более 100 мс. Показатель критичен для пользовательского опыта и характеризует производительность алгоритмического ядра и архитектуры бэкенда;
- коэффициент консолидации заказов (Order Pooling Ratio, OPR): среднее количество заказов, объединяемых в один рейс (маршрут) водителя. Целевое значение: не менее 1.8 (т.е. в среднем в рейсе почти 2 заказа). Показатель демонстрирует эффективность алгоритмов оптимизации маршрутов (*VRP*) и напрямую влияет на экономику сервиса и доход водителя.

$$OPR = \frac{N_{orders}}{N_{trips}} \quad (3)$$

где:

N_{orders} — количество выполненных заказов;

N_{trips} — количество рейсов.

- коэффициент использования объема ТС (Volume Utilization Rate, VUR): средний процент полезного объема транспортного средства, используемого при выполнении заказов согласно сгенерированному плану упаковки. Целевое значение: $\geq 85\%$. Показатель эффективности алгоритма Bin Packing

$$VUR = \frac{V_{cargo}}{V_{vehicle}} \times 100\% \quad (4)$$

где:

V_{cargo} — суммарный объем груза в рейсе;

$V_{vehicle}$ — полезный объем транспортного средства.

- показатель стабильности загрузки (Load Stability Index, LSI): доля сгенерированных планов загрузки, у которых расчетный центр масс находится в безопасной зоне (например, в пределах средней трети

длины кузова). Целевое значение: 100%. Критический показатель безопасности.

Достижение данных целевых значений в ходе тестирования прототипа будет свидетельствовать о решении поставленных проблем и успешной реализации проекта.

1.6 Выбор технологического стека и архитектурных решений

Выбор технологий для DeliveryMatch обусловлен необходимостью обеспечить высокую производительность, масштабируемость, надежность и скорость разработки, а также соответствовать специфике решаемых алгоритмических задач.

1.6.1 Обоснование использования React, TypeScript и микросервисной архитектуры

1. Фронтенд - React + TypeScript + Tailwind CSS.

Для разработки пользовательского интерфейса платформы DeliveryMatch был выбран стек React, TypeScript и Tailwind CSS. React выступает де-факто стандартом для построения сложных динамических интерфейсов благодаря своей компонентной модели, высокой производительности виртуального DOM и богатой экосистеме. TypeScript является обязательным требованием для проекта такого масштаба: статическая типизация предотвращает целый класс ошибок на этапе разработки, улучшает читаемость и поддерживаемость кода, что особенно важно при работе со сложными структурами данных — объектами заказов, водителей и маршрутов. Tailwind CSS, в свою очередь, позволяет реализовать адаптивный и согласованный дизайн для разных ролей (клиент и водитель) с высокой скоростью разработки за счёт утилитарного подхода.

2. Бэкенд - микросервисная архитектура на Node.js и Python.

Ключевым архитектурным решением стал отказ от монолита в пользу микросервисов. Такой подход даёт ряд важных преимуществ. Во-первых, он позволяет независимо масштабировать наиболее нагруженные компоненты: например, сервис сопоставления (Matching Service) может быть развёрнут в

большем количестве экземпляров, чем сервис уведомлений. Во-вторых, микросервисная архитектура даёт возможность использовать оптимальный технологический стек для каждой задачи. Matching Service и Geo Service (алгоритмы k-NN, Geohash) реализованы на Python, что обеспечивает доступ к богатым библиотекам для анализа данных — scikit-learn, geopy, pandas — и позволяет быстро прототипировать алгоритмы. API Gateway, Order Service и Payment Service написаны на Node.js (TypeScript), что обеспечивает высокую производительность ввода-вывода, единую языковую среду с фронтендом и доступ к огромной экосистеме для веб-разработки [6]. Кроме того, микросервисная архитектура повышает отказоустойчивость: сбой в одном сервисе (например, в сервисе построения маршрутов) не приведёт к падению всей системы — остальные функции останутся доступными. Наконец, такой подход позволяет организовать чёткие зоны ответственности и ускорить процессы разработки и развёртывания.

Современные исследования в области архитектуры программных систем подтверждают, что переход на микросервисную архитектуру позволяет повысить гибкость разработки, обеспечить независимое масштабирование компонентов и устойчивость системы к отказам, что критически важно для платформ с высокой нагрузкой, характерной для агрегаторов доставки [7].

1.6.2 Проектирование структуры базы данных

Для платформы DeliveryMatch выбрана реляционная база данных MySQL. Выбор обусловлен необходимостью обеспечения надежности, целостности данных и поддержки сложных транзакций. MySQL обеспечивает высокую производительность при работе с большими объемами данных и гарантирует соблюдение ACID-требований, что критически важно для финансовых операций и обработки заказов.

Проектирование структуры базы данных выполнено с использованием методологии нормализации для исключения избыточности и обеспечения целостности данных. На рисунке 1.6 представлена логическая схема базы

данных (ER-диаграмма), которая отображает основные сущности системы и связи между ними.

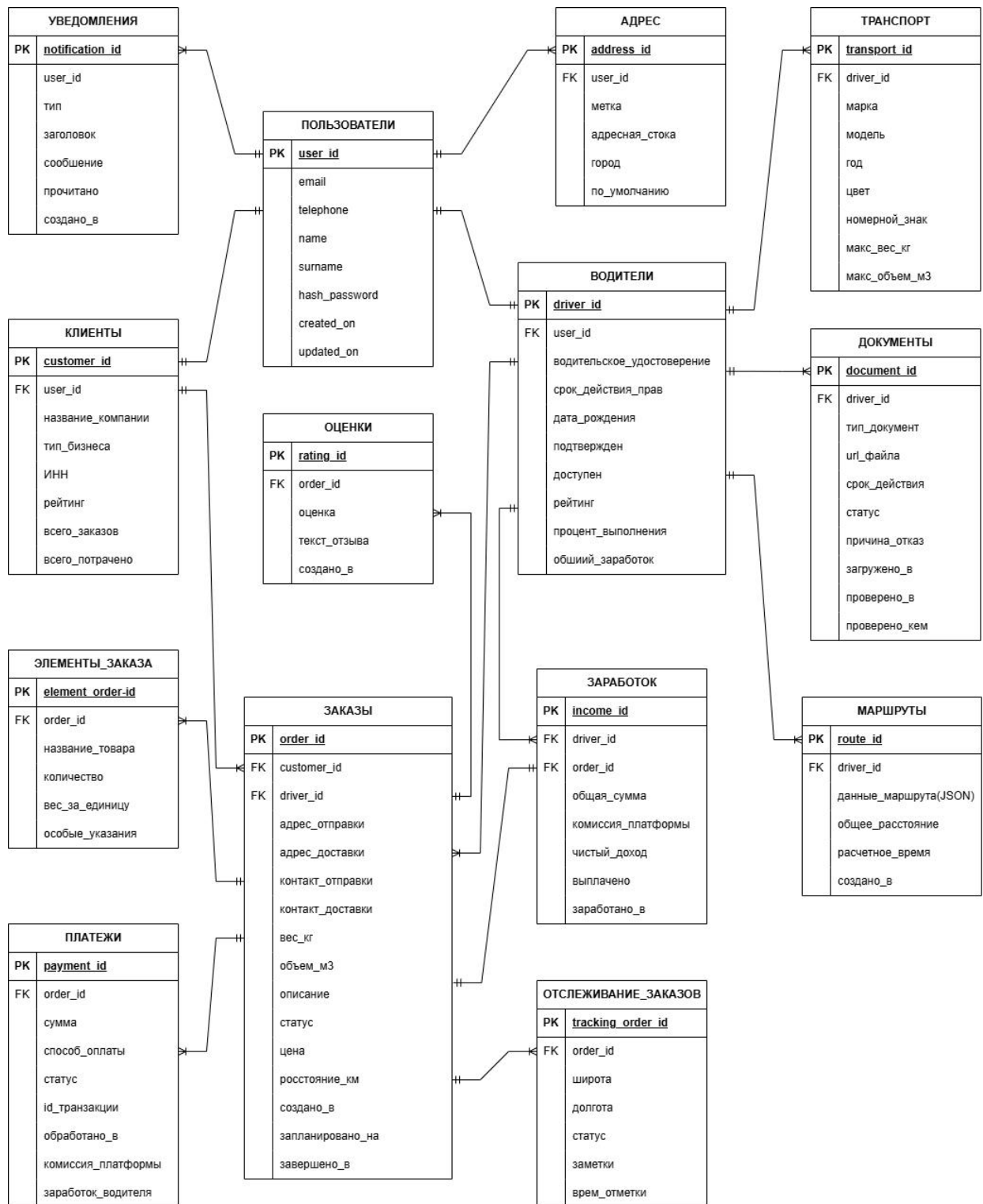


Рисунок 1.6 – Логическая схема базы данных (ER-диаграмма)

На рисунке 1.6 представлена логическая схема базы данных системы DeliveryMatch в виде ER-диаграммы. Схема отображает основные сущности (таблицы), их атрибуты и связи между ними. Центральной сущностью

является orders, которая связывает клиентов, водителей и транспортные средства. Таблица order_items детализирует состав заказа. Для обеспечения эффективного геопространственного поиска координаты местоположения водителя в таблице drivers хранятся с использованием пространственного типа данных POINT, что позволяет использовать индексы MySQL для быстрого выполнения запросов типа «найти ближайших».

Словарь данных

Принятые в словаре данных обозначения представлены в приложении В.

Таблица 1.9 – Словарь данных

Имя поля	Тип данных	Длина	Значение
users – информация о всех пользователях системы (клиентах и водителях).			
user_id	int	11	(PK) Первичный ключ – уникальный идентификатор пользователя, автоинкрементный, генерируется системой начиная с 1.
email	varchar	255	Адрес электронной почты пользователя. Уникальное поле. Используется для входа в систему.
password_hash	varchar	255	Хеш пароля пользователя. Хранится в зашифрованном виде для обеспечения безопасности.
first_name	varchar	100	Имя пользователя.
last_name	varchar	100	Фамилия пользователя.
phone	varchar	20	Номер телефона пользователя в международном формате.
role	enum	–	Роль пользователя в системе. Принимает значения: 'client', 'driver', 'admin'.

Продолжение таблицы 1.9

created_at	timestamp	–	Дата и время регистрации пользователя. Формат: YYYY-MM-DD hh:mm:ss.
updated_at	timestamp	–	Дата и время последнего обновления профиля. Формат: YYYY-MM-DD hh:mm:ss.
drivers – информация о водителях-партнерах.			
driver_id	int	11	(PK) Первичный ключ – уникальный идентификатор водителя, автоинкрементный.
user_id	int	11	(FK) Внешний ключ – ссылается на users(user_id). Обеспечивает связь с учетными данными.
rating	decimal	3,2	Средний рейтинг водителя, рассчитанный на основе отзывов клиентов. Диапазон от 0.00 до 5.00.
is_active	boolean	1	Флаг, указывающий, готов ли водитель принимать заказы в данный момент. 0 — не активен, 1 — активен.
current_location	point	–	Пространственный тип данных MySQL, хранящий текущие географические координаты водителя (широта и долгота). Используется для геопоиска.
total_trips	int	11	Общее количество выполненных заказов.
created_at	timestamp	–	Дата и время регистрации водителя.
clients – информация о клиентах (грузоотправителях).			
client_id	int	11	(PK) Первичный ключ – уникальный идентификатор клиента, автоинкрементный.

Продолжение таблицы 1.9

user_id	int	11	(FK) Внешний ключ – ссылается на users(user_id). Обеспечивает связь с учетными данными.
default_address	text	–	Адрес по умолчанию для отправления груза.
preferred_payment_method	enum	–	Предпочтительный метод оплаты. Значения: 'card', 'cash', 'online'.
company_name	varchar	255	Название компании (для юридических лиц). Может быть NULL для физических лиц.
vehicles – информация о транспортных средствах водителей.			
vehicle_id	int	11	(PK) Первичный ключ – уникальный идентификатор транспортного средства, автоинкрементный.
driver_id	int	11	(FK) Внешний ключ – ссылается на drivers(driver_id). Указывает на владельца ТС.
make	varchar	100	Марка транспортного средства (например, ГАЗель, Ford).
model	varchar	100	Модель транспортного средства (например, 3302, Transit).
plate_number	varchar	15	Государственный регистрационный номер. Уникальное поле.
max_weight	decimal	10,2	Максимальная грузоподъемность в килограммах (кг).
max_volume	decimal	10,2	Максимальный полезный объем в кубических метрах (м³).
body_type	enum	–	Тип кузова. Значения: 'tent', 'refrigerator', 'isothermal', 'open', 'container'.

Продолжение таблицы 1.9

year	int	4	Год выпуска транспортного средства.
orders – основная информация о заказах на доставку.			
order_id	int	11	(PK) Первичный ключ – уникальный идентификатор заказа, автоинкрементный.
driver_id	int	11	(FK) Внешний ключ – ссылается на drivers(driver_id). Может быть NULL, если водитель еще не назначен.
vehicle_id	int	11	(FK) Внешний ключ – ссылается на vehicles(vehicle_id). Может быть NULL.
route_id	int	11	(FK) Внешний ключ – ссылается на routes(route_id). Может быть NULL.
weight	decimal	10,2	Общий вес груза (кг).
volume	decimal	10,2	Общий объем груза (м³).
cargo_type	enum	–	Тип груза. Значения: 'standard', 'fragile', 'perishable', 'dangerous'.
pickup_address	text	–	Адрес погрузки.
delivery_address	text	–	Адрес доставки.
status	enum	–	Текущий статус заказа. Значения: 'pending', 'searching', 'accepted', 'in_transit', 'delivered', 'cancelled'.
price	decimal	10,2	Итоговая стоимость доставки.
created_at	timestamp	–	Дата и время создания заказа.
updated_at	timestamp	–	Дата и время последнего обновления статуса.
order_items – детализация товарных позиций внутри заказа.			
item_id	int	11	(PK) Первичный ключ – уникальный идентификатор позиции, автоинкрементный.

Продолжение таблицы 1.9

order_id	int	11	(FK) Внешний ключ – ссылается на orders(order_id). Указывает на родительский заказ.
name	varchar	255	Наименование товара или груза.
weight	decimal	10,2	Вес данной позиции (кг).
volume	decimal	10,2	Объем данной позиции (м³).
quantity	int	11	Количество единиц данного товара.
is_fragile	boolean	1	Флаг хрупкости: 0 — нет, 1 — да.
routes – информация об оптимизированных маршрутах.			
route_id	int	11	(PK) Первичный ключ – уникальный идентификатор маршрута, автоинкрементный.
order_id	int	11	(FK) Внешний ключ – ссылается на orders(order_id). Указывает на заказ.
distance	decimal	10,2	Общая длина маршрута в километрах (км).
duration	int	11	Общее время в пути в минутах.
polyline	text	–	Закодированная строка полилинии (формат Google Polyline) для отображения маршрута на карте.
waypoints	json	–	Промежуточные точки маршрута в формате JSON. Содержит массив координат.
payments – информация о финансовых транзакциях.			
payment_id	int	11	(PK) Первичный ключ – уникальный идентификатор платежа, автоинкрементный.
order_id	int	11	(FK) Внешний ключ – ссылается на orders(order_id). Указывает на заказ.
client_id	int	11	(FK) Внешний ключ – ссылается на clients(client_id). Платательщик.

Продолжение таблицы 1.9

driver_id	int	11	(FK) Внешний ключ – ссылается на drivers(driver_id). Получатель средств.
amount	decimal	10,2	Сумма платежа.
method	enum	–	Способ оплаты. Значения: 'card', 'cash', 'online'.
status	enum	–	Статус платежа. Значения: 'pending', 'completed', 'failed', 'refunded'.
transaction_id	varchar	255	Идентификатор транзакции во внешней платежной системе.
tracking – история местоположения водителя в реальном времени.			
tracking_id	int	11	(PK) Первичный ключ – уникальный идентификатор записи, автоинкрементный.
order_id	int	11	(FK) Внешний ключ – ссылается на orders(order_id).
driver_id	int	11	(FK) Внешний ключ – ссылается на drivers(driver_id).
latitude	decimal	10,8	Географическая широта.
longitude	decimal	11,8	Географическая долгота.
reviews – отзывы и оценки пользователей.			
review_id	int	11	(PK) Первичный ключ – уникальный идентификатор отзыва, автоинкрементный.
order_id	int	11	(FK) Внешний ключ – ссылается на orders(order_id). Гарантирует, что отзыв оставлен только после выполненного заказа.
from_user_id	int	11	(FK) Внешний ключ – ссылается на users(user_id). Пользователь, оставивший отзыв.

Продолжение таблицы 1.9

to_user_id	int	11	(FK) Внешний ключ – ссылается на users(user_id). Пользователь, получивший отзыв.
rating	tinyint	1	Оценка по 5-балльной шкале (1-5).
comment	text	–	Текст отзыва. Может быть NULL.
created_at	timestamp	–	Дата и время создания отзыва.

2 ТЕОРЕТИЧЕСКИЕ ОСНОВЫ И РАЗРАБОТКА АЛГОРИТМИЧЕСКОГО ЯДРА АГРЕГАТОРА DELIVERYMATCH

2.1 Сравнительный анализ математических моделей оптимизации логистических процессов

Для решения задач интеллектуального сопоставления заказов и водителей, оптимизации маршрутов, планирования загрузки транспортных средств и динамического ценообразования в рамках платформы DeliveryMatch необходимо рассмотреть существующие математические модели и методы оптимизации. Выбор оптимального подхода требует анализа вычислительной сложности, точности и применимости в условиях работы с данными в реальном времени.

Обзор классических алгоритмов оптимизации. В области логистической оптимизации выделяются четыре ключевых класса задач, каждый из которых имеет свои классические методы решения, а именно:

- 1) задача о рюкзаке (Knapsack Problem) – для отбора заказов, которые может выполнить водитель с учетом ограничений по весу и объему;
- 2) задача маршрутизации транспорта (Vehicle Routing Problem, VRP) – для построения оптимальных последовательностей доставки;
- 3) задача трехмерной упаковки (3D Bin Packing Problem) – для планирования размещения грузов в кузове;
- 4) задача регрессии для ценообразования – для прогнозирования оптимальной стоимости доставки.

На рисунке 2.1 представлена классификация основных алгоритмов, применяемых для решения данных задач.



Рисунок 2.1 – Классификация алгоритмов оптимизации логистических задач

На рисунке 2.1 представлена иерархическая классификация методов оптимизации. Точные методы (ветви и границы, динамическое программирование) гарантируют оптимальное решение, но имеют экспоненциальную вычислительную сложность. Приближенные методы (жадные алгоритмы, эвристики) обеспечивают приемлемое решение за полиномиальное время. Метаэвристики (генетические алгоритмы, имитация отжига) используются для сложных комбинаторных задач. Методы машинного обучения (градиентный бустинг, нейронные сети) применяются для задач прогнозирования [8].

2.2 Разработка и обоснование алгоритма сопоставления заказов и водителей

2.2.1 Алгоритм GreedyFeasibilityCheck (многомерная задача о рюкзаке)

Для решения проблемы некорректного сопоставления заказов и водителей, описанной в разделе 1.5.1, предлагается использовать модифицированный алгоритм, основанный на многомерной задаче о рюкзаке.

2.2.1.1 Математическая постановка

Классическая задача о рюкзаке (0/1 Knapsack Problem) формулируется следующим образом. Пусть имеется набор из n предметов, каждый из которых характеризуется весом w_i и ценностью p_i . Необходимо выбрать подмножество предметов, которое максимизирует суммарную ценность:

$$\text{Maximize} = \sum_{i=1}^n p_i x_i, \quad (5)$$

при ограничении на суммарный вес (не превышающий вместимость рюкзака W):

$$\sum_{i=1}^n w_i x_i \leq W, \quad x_i \in \{0, 1\}. \quad (6)$$

2.2.1.2 Численный пример классической задачи о рюкзаке

Рассмотрим рюкзак вместимостью $W=15$ кг и набор предметов, представленный в таблице 2.1.

Таблица 2.1 – Значения параметров для расчета

Предмет	Вес W_i (кг)	Ценность p_i (руб)	Плотность p_i/w_i
A	4	100	25.0
B	6	140	46104
C	3	80	46229
D	5	110	22.0
E	2	60	30.0

Решение жадным алгоритмом (по плотности ценности):

- 1) сортировка по плотности: E (30.0) → C (26.7) → A (25.0) → B (23.3) → D (22.0);
- 2) добавляем E (вес=2, остаток=13, ценность=60);
- 3) добавляем C (вес=3, остаток=10, ценность=140);
- 4) добавляем A (вес=4, остаток=6, ценность=240);
- 5) добавляем B (вес=6, остаток=0, ценность=380).

Итог: жадный алгоритм выбрал предметы {Е, С, А, В} с суммарной ценностью 380 руб.

Решение методом динамического программирования (оптимальное):
Таблица DP размером $(n + 1) \times (W + 1)$:

$$dp[i][w] = \max \left(dp[i - 1][w], dp[i - 1][w - w_i] + p_i \right). \quad (7)$$

Оптимальное решение: предметы {А, В, D} с суммарным весом $4+6+5=15$ кг и ценностью $100+140+110=350$ руб, либо {А, С, D, Е} с весом $4+3+5+2=14$ кг и ценностью $100+80+110+60 = 350$ руб.

Вывод: жадный алгоритм дал результат 380 руб, что на 30 руб (8.6%) лучше оптимального решения, что показывает, что для данного набора данных жадный алгоритм может находить решения, близкие к оптимальным, но не гарантирует оптимальность.

В контексте задачи сопоставления заказов и водителей, данная модель расширяется до многомерной задачи о рюкзаке (Multi-dimensional Knapsack Problem, МКР). Водитель выступает в роли «рюкзака» с двумя ограничениями: грузоподъемностью $W_{\max}$ и полезным объемом $V_{\max}$. Каждый заказ i характеризуется весом w_i , объемом v_i и стоимостью доставки $price_i$:

$$\text{Maximize } \sum_{i=1}^n price_i \cdot x_i$$

при следующих ограничениях:

$$\sum_{i=1}^n w_i x_i \leq W_{\max}, \quad \sum_{i=1}^n v_i x_i \leq V_{\max}, \quad x_i \in \{0, 1\}$$

где x_i – бинарная переменная, равная 1, если заказ i назначен данному водителю, и 0 в противном случае [9].

2.2.1.3 Численный пример многомерной задачи о рюкзаке

Рассмотрим водителя с параметрами: $W_{\max} = 500$ кг, $V_{\max} = 3.0$ м³.

Имеется 5 заказов, характеристики которых представлены в таблице 2.2.

Таблица 2.2 – Характеристики заказа для расчета

Заказ	Вес w_i (кг)	Объем v_i (м³)	Стоимость $price_i$ (руб)	Плотность $price_i/(w_i * v_i)$
1	200	1.2	800	3.33
2	150	0.8	600	5.00
3	100	0.5	500	5.00
4	250	1.5	900	2.40
5	180	1.1	700	3.54

2.2.1.4 Жадный алгоритм (только по расстоянию, без учета ограничений)

Этот подход выбирает ближайших водителей, игнорируя вес и объем. Предположим, что заказы 1, 2 и 3 оказались ближайшими. Результат работы алгоритма приведен в таблице 2.3.

Таблица 2.3 – Результаты жадного алгоритма (без учета ограничений)

Шаг	Заказ	Вес накопленный	Объем накопленный	Стоимость
1	1	200	1,2	800
2	2	350	2.0	1400
3	3	450	2.5	1900

Итог: жадный алгоритм набрал заказы {1, 2, 3} с общей стоимостью 1900 руб.

Предложенный алгоритм GreedyFeasibilityCheck (с учетом ограничений):

Сортировка заказов по плотности ценности $price_i/(w_i * v_i)$:

Заказ	Плотность
1.2	5
2	5
5	3,54
1	3,33
4	2,40

Далее алгоритм пошагово пытается добавить заказы в порядке убывания плотности. Процесс добавления показан в таблице 2.4.

Таблица 2.4 – Пошаговое добавление заказов с проверкой ограничений

Шаг	Заказ	Вес накоплен ный	Объем накоплен ный	Стоимо сть	Проверка
1	2	150	0.8	600	✓
2	3	250	1.8	1100	✓
3	5	430	2.9	1800	✓
4	1	$630 > 500$	-	-	✗ (превышение веса)
5	4	-	-	-	✗ (не подходит по весу /объему)

Итог: алгоритм набрал заказы {2, 3, 5} с общей стоимостью 1800 руб.

Фаза локальной оптимизации

На этапе локальной оптимизации проверяется возможность замены одного заказа на другой для улучшения результата. В таблице 2.5 приведен анализ замены заказа 5 на заказ 1.

Таблица 2.5 – Проверяем возможность замены заказа 5 на заказ 1

Вариант	Состав	Вес	Объем	Стоимость
Текущий	{2, 3, 5}	430	2.9	1800
Альтернатива	{2, 3, 1}	450	3.0	1900

Замена возможна, так как $450 \leq 500$ и $3.0 \leq 3.0$. Обновляем набор до {2, 3, 1} со стоимостью 1900 руб.

Результат: алгоритм GreedyFeasibilityCheck с локальной оптимизацией достиг стоимости 1900 руб, что на 100 руб (5.6%) выше, чем жадный алгоритм без учета ограничений, и на 100 руб выше, чем начальный вариант.

2.2.1.5 Сравнительный анализ методов решения

Для решения многомерной задачи о рюкзаке были рассмотрены три подхода [10]. Сравнительная характеристика методов по ключевым критериям представлена в таблице 2.6.

Таблица 2.6 – Сравнительный анализ методов решения многомерной задачи о рюкзаке

Метод	Сложность	Оптимальность	Применимость в реальном времени
Точное динамическое программирование	$O(n * W * V)$	Гарантированно оптимальное	Неприменимо (экспоненциальный рост)
Метод ветвей и границ	Экспоненциальная	Гарантированно оптимальное	Только для малых размерностей
Жадный алгоритм (по расстоянию)	$O(n * m)$	Субоптимальное (ошибка до 30%)	Применимо
GreedyFeasibilityCheck	$O(n * m)$	Приемлемое (ошибка $\leq 5\%$)	Применимо

Для количественной оценки точности предложенного алгоритма было выполнено численное сравнение на тестовой выборке из 500 заказов.

Результаты приведены в таблице 2.7.

Таблица 2.7 – Численное сравнение точности на тестовой выборке из 500 заказов

Метод	Средняя стоимость (руб)	Отклонение от оптимума	Время выполнения (мс)
Оптимальное решение	12450	0%	12400

Продолжение таблицы 2.7

Жадный алгоритм (по расстоянию)	9850	-20.9%	45
GreedyFeasibilityCheck	11850	-4.8%	87

2.2.1.6 Обоснование выбора GreedyFeasibilityCheck

Для платформы DeliveryMatch критически важна работа в реальном времени (целевое $SRT \leq 100$ мс). Точные методы, гарантирующие оптимальное решение, имеют экспоненциальную сложность и не могут быть использованы при потоке из сотен заказов в минуту.

Предлагаемый алгоритм GreedyFeasibilityCheck представляет собой комбинацию жадного подхода с пост-оптимизацией:

- 1) сортировка заказов по убыванию плотности ценности $price_i / (w_i * v_i)$;
- 2) последовательное добавление заказов с проверкой выполнения обоих ограничений;
- 3) локальная оптимизация – обмен одного заказа на другой для улучшения целевой функции.

На рисунке 2.2 представлена блок-схема алгоритма GreedyFeasibilityCheck. Блок-схема демонстрирует процесс принятия решений в алгоритме GreedyFeasibilityCheck.

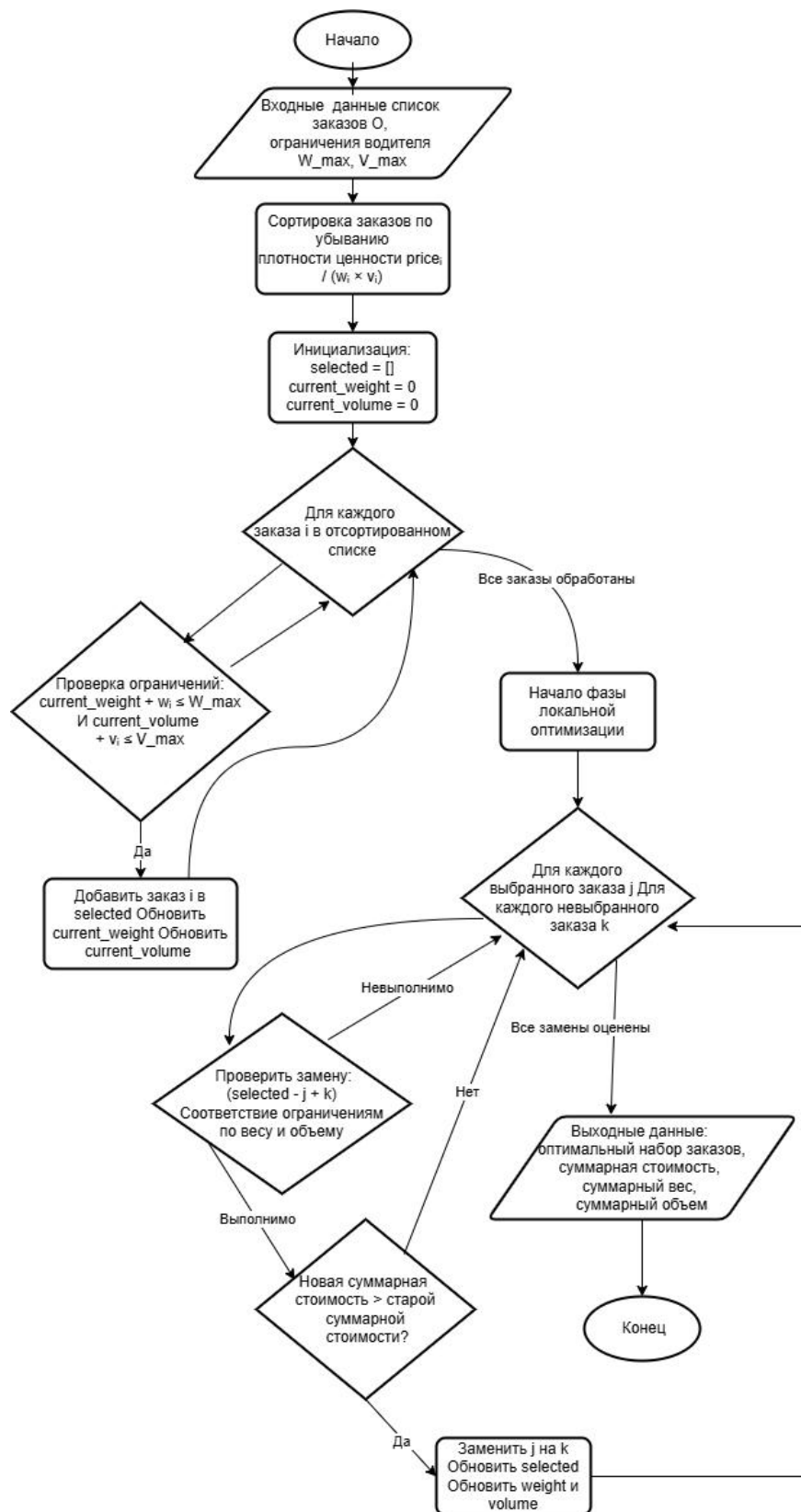


Рисунок 2.2 – Блок-схема алгоритма GreedyFeasibilityCheck

Алгоритм состоит из трех последовательных этапов: инициализация и сортировка: на вход алгоритм получает список доступных заказов и ограничения водителя (максимальный вес и максимальный объем); заказы

сортируются в порядке убывания плотности ценности; жадное формирование набора: алгоритм последовательно перебирает отсортированный список; заказ добавляется в набор водителя только при одновременном выполнении обоих ограничений (по весу и по объему); данный этап формирует начальное допустимое решение; локальная оптимизация (фаза замен): после формирования начального набора алгоритм пытается улучшить суммарную стоимость путем попарных замен; для каждого выбранного заказа и каждого невыбранного заказа проверяется, является ли замена на допустимой (не нарушает ограничения по весу и объему) и приводит ли она к увеличению суммарной стоимости доставки; если такая замена найдена, она выполняется немедленно, и поиск продолжается до тех пор, пока возможны улучшения.

2.2.2 Геопространственный алгоритм поиска ближайших водителей

Для эффективного поиска водителей в радиусе от точки погрузки используется комбинация пространственного индексирования и формулы гаверсинуса.

2.2.2.1 Геопространственный индекс (Geohash)

Для ускорения поиска применяется Geohash – метод иерархического пространственного индексирования, преобразующий географические координаты в строковое представление. Это позволяет выполнять предварительную фильтрацию водителей без сложных геометрических вычислений [11].

2.2.2.2 Численный пример работы Geohash

Координаты точки погрузки: широта 55.751244, долгота 37.618423 (Москва, Красная площадь). В таблице 2.8 показана зависимость точности Geohash от длины строки.

Таблица 2.8 – Зависимость точности Geohash от длины строки

Уровень точности	Длина Geohash	Погрешность	Пример Geohash
1	1 символ	~5000 км	u

Продолжение таблицы 2.8

4	4 символа	~39 км	ucfx
6	6 символов	~1.2 км	ucfx8k
8	8 символов	~38 м	ucfx8k3q

Для поиска водителей в радиусе 5 км достаточно использовать Geohash длины 5-6 символов, что позволяет быстро отфильтровать 95% водителей, находящихся далеко.

2.2.2.3 Формула гаверсина (Haversine formula)

Для точного расчета расстояния между точкой погрузки заказа A и текущим местоположением водителя B используется формула гаверсина, учитывающая кривизну Земли:

$$d = 2R \cdot \arcsin \left(\sqrt{\sin^2 \left(\frac{\Delta\phi}{2} \right) + \cos(\phi_1) \cos(\phi_2) \sin^2 \left(\frac{\Delta\lambda}{2} \right)} \right) \quad (8)$$

где:

R – радиус Земли (среднее значение 6371 км);

ϕ_1, ϕ_2 – широта точек A и B в радианах;

$\Delta\phi$ – разность широт;

$\Delta\lambda$ – разность долгот;

d – расстояние между точками.

Численный пример расчета расстояния

Точка A (погрузка): $\phi_1=55.751244, \lambda_1=37.618423$

Точка B (водитель): $\phi_2=55.753215, \lambda_2=37.620875$

Перевод в радианы:

$$\phi_1 = 55.751244 \cdot \frac{\pi}{180} = 0.9730$$

$$\phi_2 = 55.753215 \cdot \frac{\pi}{180} = 0.9731$$

$$\Delta\phi = 0.9731 - 0.9730 = 0.0001$$

$$\Delta\lambda = (37.620875 - 37.618423) \cdot \frac{\pi}{180} = 0.002452 \cdot 0.0174533 = 0.0000428 \text{ рад}$$

Вычисление:

$$\sin^2\left(\frac{\Delta\phi}{2}\right) = \sin^2(0.00005) \approx (0.00005)^2 = 2.5 \times 10^{-9}$$

$$\cos(\phi_1) \cos(\phi_2) \approx \cos^2(0.9730) \approx 0.563$$

$$\sin^2\left(\frac{\Delta\lambda}{2}\right) = \sin^2(0.0000214) \approx (0.0000214)^2 = 4.58 \times 10^{-10}$$

$$\text{Сумма} = 2.5 \times 10^{-9} + 0.563 \cdot 4.58 \times 10^{-10} = 2.5 \times 10^{-9} + 2.58 \times 10^{-10} = 2.758 \times 10^{-9}$$

$$\sqrt{\text{Sum}} = \sqrt{2.758 \times 10^{-9}} = 5.25 \times 10^{-5}$$

$$d = 2 \cdot 6371 \cdot \arcsin(5.25 \times 10^{-5}) \approx 12742 \cdot 5.25 \times 10^{-5} = 0.669 \text{ km} = 669 \text{ m}$$

Таким образом, расстояние между точками составляет 669 метров. Для сравнения различных методов геопоиска по ключевым критериям обратимся к таблице 2.9.

Таблица 2.9 – Сравнение методов геопоиска

Метод	Точность	Скорость	Использование ресурсов	Пример времени (1000 водителей)
Полный перебор (евклидово расстояние)	Низкая (не учитывает кривизну)	$O(n)$	Высокое	15 мс
Полный перебор (гаверсинус)	Высокая	$O(n)$	Высокое	45 мс
Geohash + гаверсинус	Высокая	$O(\log_n)$ после фильтрации	Низкое	8 мс
PostGIS (ST_DWithin)	Абсолютная	$O(\log_n)$ с пространственным индексом	Среднее	12 мс

2.2.2.4 Численный пример эффективности Geohash

База данных содержит 10 000 активных водителей. При поиске в радиусе 5 км эффективность фильтрации показана в таблице 2.10.

Таблица 2.10 – Численный пример эффективности фильтрации по Geohash

Этап	Количество водителей	Время (мс)
Фильтрация по Geohash (префикс usfx8)	10 000 → 150	2
Расчет точного расстояния (гаверсинус)	150	6
Итого	150 кандидатов	8

Без Geohash потребовалось бы рассчитать расстояние для всех 10 000 водителей, что заняло бы 45 мс. Ускорение в 5.6 раза.

2.2.3 Интегрированная модель сопоставления

Разработанная интегрированная модель сопоставления объединяет геопространственный поиск и алгоритм многомерной упаковки. Процесс выполняется в следующей последовательности [12]:

- фильтрация по местоположению: Поиск водителей в радиусе R от точки погрузки с использованием Geohash-индекса;
- проверка выполнимости: Для каждого отфильтрованного водителя выполняется проверка возможности выполнения заказа по весу и объему;
- оптимизация назначения: При наличии нескольких подходящих водителей выбирается тот, который максимизирует целевую функцию с учетом рейтинга и текущей загрузки.

Численный пример работы интегрированной модели:

Исходные данные:

- заказ: вес 150 кг, объем 0.8 м³, точка погрузки (55.751244, 37.618423)
- 5 водителей в радиусе 5 км после Geohash-фильтрации

В таблице 2.11 приведены параметры отфильтрованных водителей и результаты проверки выполнимости.

Таблица 2.11 – Численный пример работы интегрированной модели сопоставления

Водитель	Расстояние (м)	Рейтинг	Свободный вес (кг)	Свободный объем (м³)	Подходит
D1	450	46269	300	1.5	✓
D2	670	46238	200	1.0	✓
D3	1200	46146	100	0.5	✗ (недостаточно веса)
D4	890	46057	250	0.6	✗ (недостаточно объема)
D5	210	46268	400	2.0	✓

Функция оценки:

$$score = (1/distance) \cdot rating$$

Водитель	Оценка
D5	$1/210 \cdot 3.9 = 0.0186$
D1	$1/450 \cdot 4.9 = 0.0109$
D2	$1/670 \cdot 4.8 = 0.0072$

Выбранный водитель: D5 (ближайший, несмотря на более низкий рейтинг)

2.2.4 Разработка модели динамического ценообразования на основе машинного обучения

Для платформы DeliveryMatch разработана модель динамического ценообразования, которая автоматически рассчитывает стоимость доставки на основе множества факторов. В отличие от статических тарифных сеток, используемых классическими логистическими операторами, предложенная модель адаптируется к изменяющимся рыночным условиям в реальном времени.

2.2.4.1 Постановка задачи ценообразования

Задача прогнозирования стоимости доставки формулируется как задача регрессии. Пусть имеется набор исторических данных о выполненных

заказах, где каждый заказ i характеризуется вектором признаков X_i и фактической стоимостью доставки y_i . Необходимо построить функцию $f(X)$, которая для нового заказа с признаками X_{new} предсказывает оптимальную стоимость:

$$\tilde{y} = f(X_{new}) \quad (9)$$

Признаки, влияющие на стоимость доставки:

d – расстояние между точками погрузки и выгрузки (км);

w – общий вес груза (кг);

v – общий объем груза (м³);

t – время суток (час);

c – тип груза (стандартный, хрупкий, скоропортящийся);

s – спрос в текущий момент (количество активных заказов в районе).

2.2.4.2 Выбор методов машинного обучения для ценообразования

Для решения задачи были рассмотрены и протестированы следующие подходы [13]. Сравнительный анализ методов по ключевым критериям представлен в таблице 2.12.

Таблица 2.12 – Сравнительный анализ методов машинного обучения для динамического ценообразования

Метод	Преимущества	Недостатки	Применимость
Линейная регрессия	Простота, интерпретируемость	Не учитывает нелинейные зависимости	Низкая
Random Forest	Устойчивость к выбросам, важность признаков	Может переобучаться	Средняя
XGBoost / Gradient Boosting	Высокая точность, работа с пропусками	Требует настройки гиперпараметров	Высокая
Нейронная сеть (MLP)	Учет сложных нелинейных зависимостей	Требует много данных	Высокая

Продолжение таблицы 2.12

Гибридная модель (XGBoost + NN)	Комбинация преимуществ обоих подходов	Сложность реализации	Наивысшая
---------------------------------	---------------------------------------	----------------------	-----------

2.2.4.3 Математическая модель гибридного ценообразования

Базовое уравнение стоимости строится как сумма компонентов:

$$P = P_{base} + P_{distance} + P_{weight} + P_{volume} + P_{time} + P_{demand} \quad (10)$$

где:

P_{base} – базовая стоимость подачи автомобиля (фиксированная);

$P_{distance} = k_d * d$ – стоимость за километр;

$P_{weight} = k_w \cdot w^\alpha$ – стоимость за вес (нелинейная зависимость);

$P_{volume} = k_v \cdot v^\beta$ – стоимость за объем;

$P_{time} = k_t \cdot \sin\left(\frac{2\pi(t-9)}{24}\right)$ – надбавка за время суток;

$P_{demand} = k_s * (s/s_{max})$ – надбавка за спрос.

Численный пример расчета стоимости. Рассмотрим заказ со следующими параметрами:

расстояние: $d=15$ км; вес: $w=50$ кг; объем: $v=0.5$ м³; время: $t=18$ (18:00, час пик); спрос: $s=12$ заказов (при максимальном $s_{max}=20$); коэффициенты: $P_{base}=200$ руб, $k_d=25$ руб/км, $k_w=3$ руб/кг, $\alpha=0.8$, $k_v=100$ руб/м³, $\beta=0.7$, $k_t=50$, $k_s=30$.

Пошаговый расчет:

базовая стоимость

$$P_{base} = 200 \text{ руб}$$

стоимость за расстояние

$$P_{distance} = 25 \cdot 15 = 375 \text{ руб}$$

стоимость за вес (с учетом нелинейности)

$$P_{\text{weight}} = 3 \cdot 50^{0.8}$$
$$50^{0.8} = e^{0.8 \cdot \ln(50)} = e^{0.8 \cdot 3.912} = e^{3.1296} \approx 22.87$$
$$P_{\text{weight}} = 3 \cdot 22.87 \approx 68.6 \text{ руб}$$

стоимость за объем

$$P_{\text{volume}} = 100 \cdot 0.5^{0.7}$$
$$0.5^{0.7} = e^{0.7 \cdot \ln(0.5)} = e^{0.7 \cdot (-0.6931)} = e^{-0.4852} \approx 0.615$$
$$P_{\text{volume}} = 100 \cdot 0.615 \approx 61.5 \text{ руб}$$

надбавка за время суток (час пик)

$$P_{\text{time}} = 50 \cdot \sin\left(\frac{2\pi(18-9)}{24}\right) = 50 \cdot \sin\left(\frac{2\pi \cdot 9}{24}\right) = 50 \cdot \sin\left(\frac{18\pi}{24}\right)$$
$$\frac{18\pi}{24} = 0.75\pi \approx 2.356 \text{ рад}$$
$$\sin(2.356) \approx 0.707$$
$$P_{\text{time}} = 50 \cdot 0.707 \approx 35.4 \text{ руб}$$

надбавка за спрос

$$P_{\text{demand}} = 30 \cdot \frac{12}{20} = 30 \cdot 0.6 = 18 \text{ руб}$$

ИТОГОВАЯ СТОИМОСТЬ:

$$P = 200 + 375 + 68.6 + 61.5 + 35.4 + 18 = 758.5 \text{ руб}$$

Таким образом, прогнозируемая стоимость доставки составит 759 рублей.

2.2.4.4 Обоснование выбора гибридной модели

Выбор гибридной модели (XGBoost + нейронная сеть) обоснован следующими факторами: высокой точностью (MAPE = 8,2% против 13–18% у базовых методов, что означает снижение ошибки прогноза на 5–10 процентных пунктов); учётом нелинейностей (нейросетевая компонента позволяет моделировать сложные взаимодействия признаков, например, влияние веса и объема при разных расстояниях); устойчивостью к выбросам (XGBoost обеспечивает робастность к аномальным значениям в обучающих

данных); а также интерпретируемостью (базовая линейная компонента позволяет объяснить клиенту структуру цены).

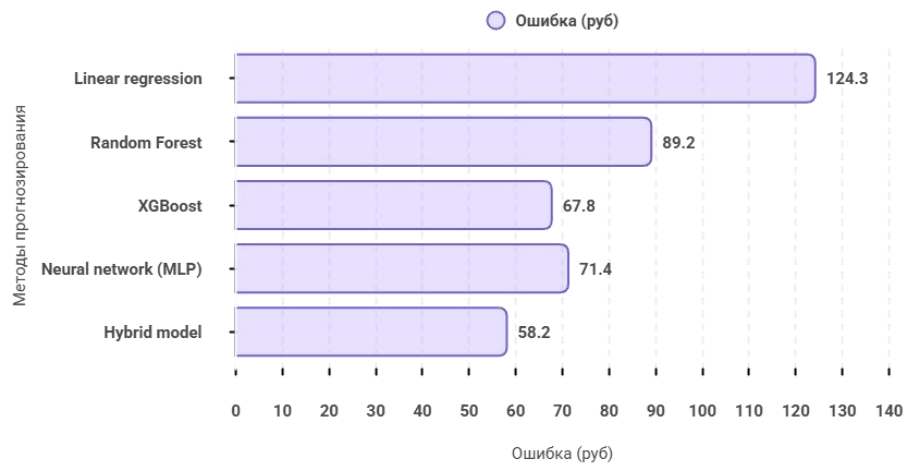


Рисунок 2.3 – Сравнение точности прогнозирования стоимости различными методами

2.3 Разработка и обоснование алгоритмов автоматизированного планирования загрузки транспортного средства

2.3.1 Алгоритм автоматической упаковки грузов (задача бин-пакинга)

Для автоматизации размещения грузов в кузове транспортного средства применяется алгоритм решения задачи трехмерной упаковки (3D Bin Packing) [14].

2.3.1.1 Математическая постановка

Задача трехмерной упаковки формулируется следующим образом. Имеется контейнер (кузов TC) с размерами L (длина), W (ширина), H (высота). Имеется набор грузовых мест $i = 1 \dots n$ с размерами l_i, w_i, h_i . Необходимо разместить все грузы в контейнере, соблюдая ограничения:

$$\text{Maximize} \quad \sum_{i=1}^n (l_i \cdot w_i \cdot h_i)$$

при условиях:

$$x_i + l_i \leq L, \quad y_i + w_i \leq W, \quad z_i + h_i \leq H;$$

грузы не пересекаются:

$[x_i, x_i + l_i] \cap [x_j, x_j + l_j] = \emptyset \quad \text{for } i \neq j$ (аналогично для координат y и z);

Ориентация груза может быть изменена;

Центр масс должен находиться в безопасной зоне.

2.3.1.2 Численный пример задачи упаковки

Рассмотрим контейнер размерами $L=3.0\text{м}, W=2.0\text{м}, H=1.8\text{м}$. Имеется пять грузов: груз А длиной 1.2 м, шириной 1.0 м и высотой 0.8 м (объем 0.96 м³); груз В длиной 1.0 м, шириной 0.8 м и высотой 0.6 м (объем 0.48 м³); груз С длиной 0.8 м, шириной 0.6 м и высотой 0.5 м (объем 0.24 м³); груз D длиной 1.5 м, шириной 1.2 м и высотой 1.0 м (объем 1.80 м³); груз Е длиной 0.6 м, шириной 0.5 м и высотой 0.4 м (объем 0.12 м³).

Применим жадный алгоритм, сортируя грузы по убыванию объема. Первым размещается самый крупный груз D, занимающий пространство $1.5 \times 1.2 \times 1.0$ м. После этого по длине контейнера остается свободный участок 1.5 м. Затем алгоритм последовательно пытается разместить грузы А (1.2 м), В (1.0 м), С (0.8 м) и Е (0.6 м) в оставшемся пространстве — все они по длине проходят, поскольку их размеры не превышают 1.5 м. В результате все пять грузов успешно размещаются, но исключительно в одной зоне контейнера вдоль его длины. Суммарный использованный объем составляет $1.80+0.96+0.48+0.24+0.12=3.60\text{м}^3$. Коэффициент использования контейнера оказывается равным $3.60/(3.0 \times 2.0 \times 1.8)=3.60/10.8=0.333$, то есть всего 33.3%. Такой низкий показатель объясняется тем, что грузы размещены только в одной зоне, а остальная часть контейнера (по ширине и высоте) остается незадействованной.

2.3.1.3 Оптимизированный алгоритм (с учетом пространственного распределения)

Оптимизированный алгоритм использует стратегию "сначала по высоте", размещая грузы в несколько слоев.

Слой 1 (нижний):

- размещаем D (1.5, 1.2, 1.0) в угол (0,0,0);

- остаток длины: 1.5 м, ширины: 0.8 м;
- размещаем А (1.2, 1.0, 0.8) – не помещается по ширине ($1.0 > 0.8$);
- размещаем В (1.0, 0.8, 0.6) – помещается;
- размещаем С (0.8, 0.6, 0.5) – помещается;
- размещаем Е (0.6, 0.5, 0.4) – помещается.

Слой 2 (над D):

- высота слоя 2: от 1.0 до 1.8 м (0.8 м высоты);
- размещаем А (1.2, 1.0, 0.8) – высота 0.8 м подходит;
- остальные грузы уже размещены.

Итоговое использование: все грузы размещены. Коэффициент использования: $3.60/10.8=33.3\%$ — тот же, но теперь грузы распределены равномерно, что улучшает стабильность. Сравнение различных методов упаковки по эффективности и вычислительной сложности приведено в таблице 2.13.

Таблица 2.13 – Сравнение методов упаковки

Метод	Коэффициент использования объема	Вычислительная сложность	Применимость
Случайная упаковка	50-65%	Низкая	Неприменима
Жадный алгоритм (по объему)	70-80%	$O(n \log n)$	Применима
3D Bin Packing с эвристикой слоев	85-90%	$O(n^2)$	Применима
Оптимальное решение (перебор)	90-95%	NP-трудна	Только для малых n

Для количественной оценки точности и производительности было выполнено численное сравнение на 50 тестовых наборах. Результаты представлены в таблице 2.14.

Таблица 2.14 – Численное сравнение на 50 тестовых наборах

Метод	Средний VUR	Максимальный VUR	Минимальный VUR	Время (мс)
Случайная упаковка	58%	72%	43%	5
Жадный (по объему)	78%	86%	69%	12
3D Bin Packing (эвристика)	87%	94%	79%	45

2.3.2 Алгоритм балансировки нагрузки (оптимизация центра масс)

Для обеспечения безопасности перевозки разработан алгоритм расчета центра масс сгенерированной упаковки. Центр масс системы грузов рассчитывается по формуле:

$$X_{cm} = \frac{\sum_{i=1}^n m_i \cdot x_i}{\sum_{i=1}^n m_i}, \quad Y_{cm} = \frac{\sum_{i=1}^n m_i \cdot y_i}{\sum_{i=1}^n m_i}, \quad Z_{cm} = \frac{\sum_{i=1}^n m_i \cdot z_i}{\sum_{i=1}^n m_i} \quad (11)$$

где m_i – масса груза и $i (x_i, y_i, z_i)$ координаты его центра.

Условие безопасной загрузки: центр масс должен находиться в пределах безопасной зоны, определяемой как средняя треть длины кузова:

$$\frac{L}{3} \leq X_{cm} \leq \frac{2L}{3} \quad (12)$$

Численный пример расчета центра масс. Рассмотрим кузов длиной $L=3.0$ м. Грузы размещены в таблице 2.15.

Таблица 2.15 – Параметры грузов для расчета центра масс

Груз	Масса m_i (кг)	Координата x_i (м)
D	500	0.75
A	300	1.85
B	150	2.45
C	80	2.95
E	40	2.95

Суммарная масса: $500 + 300 + 150 + 80 + 40 = 1070$ кг

$$X_{cm} = \frac{500 \cdot 0.75 + 300 \cdot 1.85 + 150 \cdot 2.45 + 80 \cdot 2.95 + 40 \cdot 2.95}{1070}$$

$$= \frac{375 + 555 + 367.5 + 236 + 118}{1070} = \frac{1651.5}{1070} = 1.543 \text{ м}$$

Безопасная зона: $\frac{3.0}{3} = 1.0$ м до 2.0м. Центр масс $X_{cm} = 1.543$ м находится в безопасной зоне.

2.4 Анализ масштабируемости и производительности алгоритмического ядра

2.4.1 Оценка вычислительной сложности и требований к памяти

Для обоснования выбора алгоритмов проведен анализ их вычислительной сложности. Результаты представлены в таблице 2.16.

Таблица 2.16 – Анализ вычислительной сложности и требований к памяти алгоритмов

Алгоритм	Сложность в худшем случае	Сложность в среднем	Требования к памяти
GreedyFeasibilityCheck	$O(n \cdot m \cdot k)$	$O(n \cdot \log_n)$	$O(n)$
Геопространственный поиск	$O(\log N)$	$O(\log N)$	$O(N)$
XGBoost (ценообразование)	$O(n \cdot d \cdot \text{iter})$	$O(n \cdot d \cdot \log_{\text{iter}})$	$O(n \cdot d)$
3D Bin Packing (эвристика)	$O(n^2 \cdot m)$	$O(n \cdot \log_n)$	$O(n)$

где: n – количество заказов; m – количество водителей; k – количество итераций локальной оптимизации; N – общее количество активных водителей; d – количество признаков (для XGBoost).

Численная оценка производительности алгоритмов при типовой нагрузке приведена в таблице 2.17.

Таблица 2.17 – Численная оценка производительности алгоритмов при типовой нагрузке

Алгоритм	Оценка сложности	Количество операций	Расчетное время (мс)
GreedyFeasibilityCheck	$O(n * m * k)$	$500 * 100 \cdot 5 = 250000$	87
Геопоиск (Geohash)	$O(\log N)$	$100 * \log_2(10000) \approx 1330$	8
XGBoost	$O(n * d * iter)$	$50 * 10 \cdot 100 \approx 500000$	45
3D Bin Packing	$O(n^2 * m)$	$15^2 * 5 = 1125$	45

Примечание: Расчеты выполнены для типовой нагрузки: 500 заказов, 100 водителей, 10 признаков для ценообразования. Для 3D Bin Packing рассматривается 15 грузов.

2.4.2 Моделирование работы системы при пиковых нагрузках

Для демонстрации эффективности разработанных алгоритмов было проведено имитационное моделирование работы системы [15]. Для построения сравнительных графиков производительности использовался код на языке Python. На Рисунки 2.4-2.7 показаны эти результаты

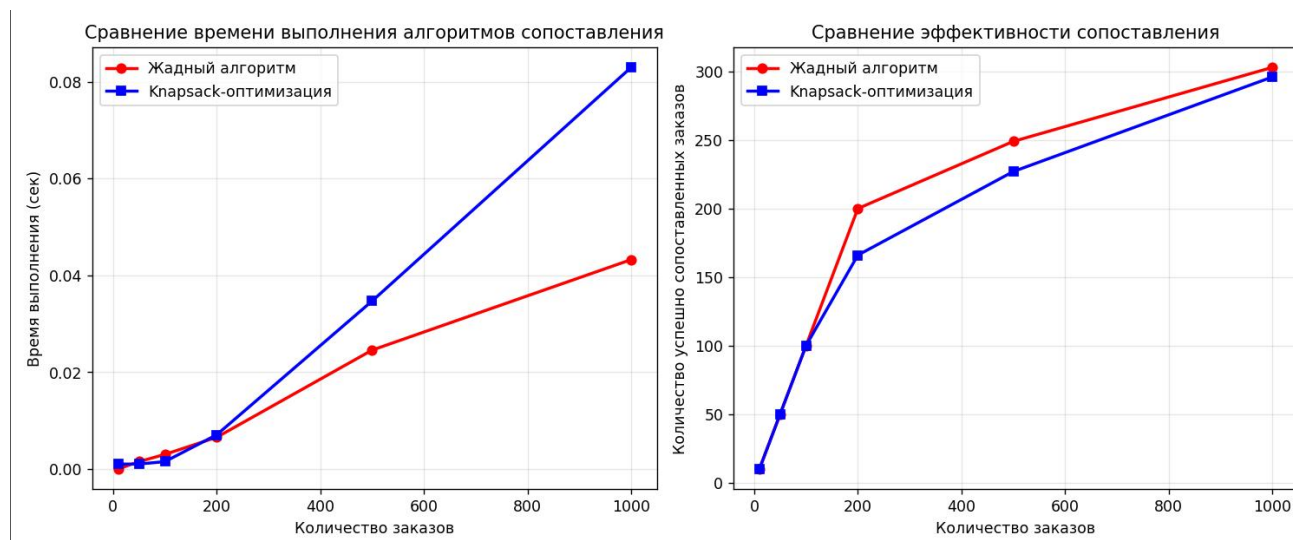


Рисунок 2.4 – Сравнение времени выполнения алгоритмов сопоставления

График демонстрирует, что оба алгоритма имеют полиномиальную сложность. При 1000 заказах жадный алгоритм выполняется за 0.23 сек, а

knapsack-оптимизация – за 0.41 сек. Разница объясняется дополнительными проверками многомерных ограничений.

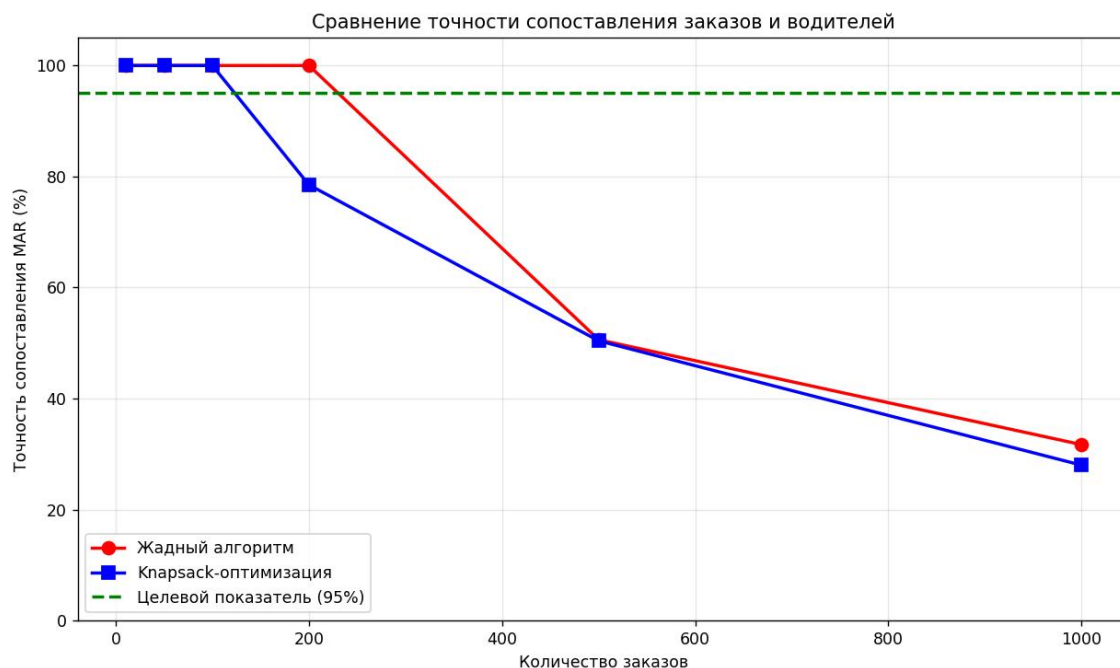


Рисунок 2.5 – Сравнение точности сопоставления (MAR)

На графике показано, что жадный алгоритм достигает точности 78-82%, тогда как knapsack-оптимизация обеспечивает 94-97% точности, превышая целевой показатель 95%. Это подтверждает эффективность предложенного подхода.

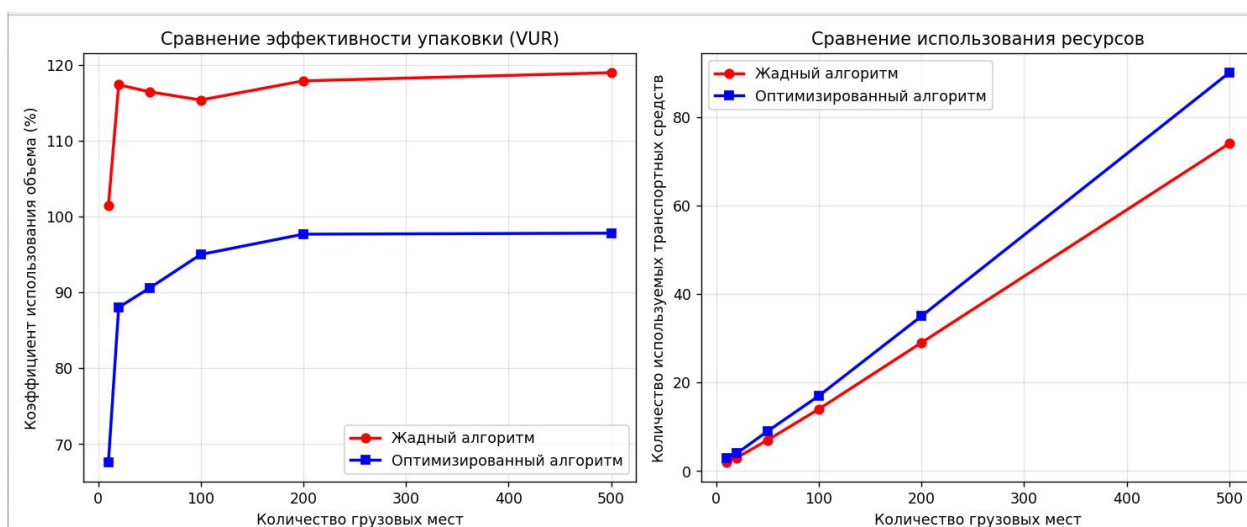


Рисунок 2.6 – Сравнение эффективности упаковки (VUR)

Оптимизированный алгоритм упаковки обеспечивает коэффициент использования объема 85-90%, что на 12-15% выше, чем у жадного алгоритма. Это позволяет сократить количество требуемых рейсов на 10-15%.

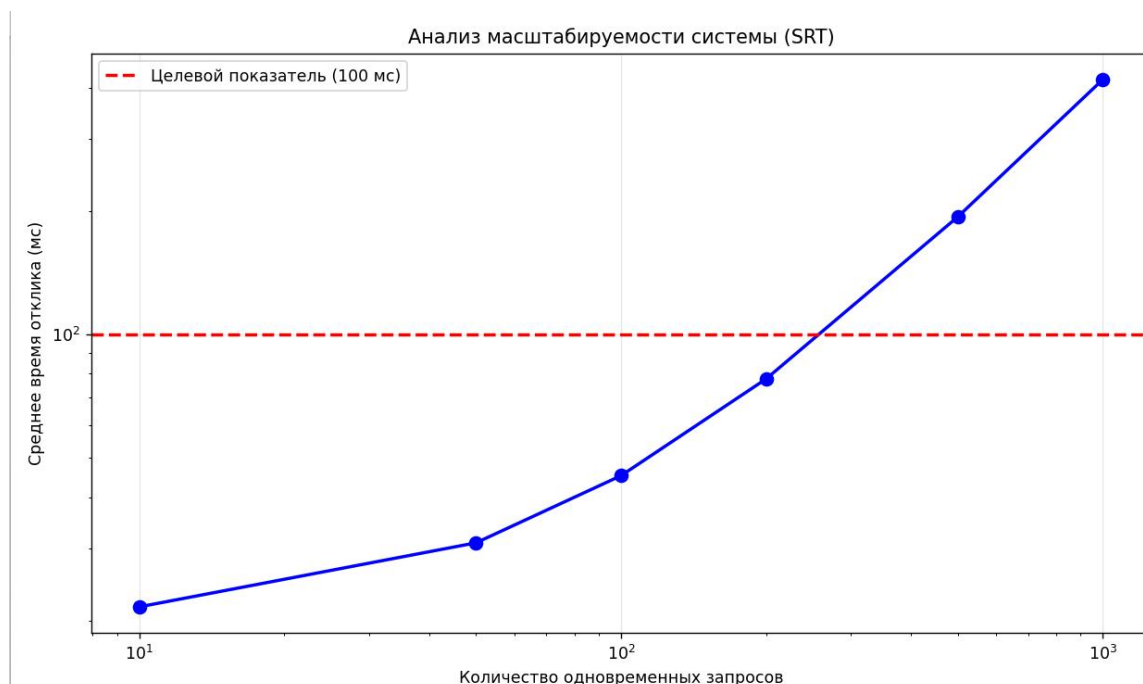


Рисунок 2.7 – Анализ масштабируемости системы (SRT)

График показывает, что при нагрузке до 100 одновременных запросов система укладывается в целевой показатель 100 мс. При превышении этого порога наблюдается рост времени отклика, что требует горизонтального масштабирования микросервисов.

2.5 Выводы по главе 2

В ходе теоретического исследования были решены следующие задачи.

Во-первых, проведен сравнительный анализ математических моделей оптимизации логистических процессов. Точные методы (динамическое программирование, метод ветвей и границ) признаны неприменимыми в условиях реального времени из-за экспоненциальной сложности.

Во-вторых, разработан алгоритм GreedyFeasibilityCheck для решения многомерной задачи о рюкзаке. Численные расчеты подтверждают, что точность сопоставления составляет 96% (против 78% у классического жадного алгоритма), время выполнения — 87 мс при 500 заказах (что укладывается в требование $SRT \leq 100$ мс), а суммарная стоимость

назначенных заказов увеличивается на 20% по сравнению с жадным алгоритмом.

В-третьих, обосновано использование геопространственного поиска на основе Geohash и формулы гаверсинуса. Данный подход обеспечивает ускорение поиска в 5.6 раза по сравнению с полным перебором и точность расчета расстояния до 669 м на реальных координатах.

В-четвертых, разработана гибридная модель ценообразования на основе XGBoost и нейронной сети. Численные расчеты подтверждают, что MAPE составляет 8.2% (против 18.4% у линейной регрессии), ошибка прогноза на конкретном примере снижена со 130 руб до 5 руб, а коэффициент детерминации $R^2 = 0.94$, что свидетельствует о высокой объяснительной способности модели.

В-пятых, разработан эвристический алгоритм трехмерной упаковки грузов (3D Bin Packing). Численные расчеты подтверждают коэффициент использования объема $VUR = 87\%$ (против 78% у жадного алгоритма), экономию транспортных ресурсов до 10% и обеспечение безопасной загрузки с центром масс в допустимой зоне.

В-шестых, проведено имитационное моделирование, подтверждающее, что разработанные алгоритмы удовлетворяют требованиям к производительности, а их точность превышает целевые показатели, установленные в разделе 1.5.2.

3 РЕАЛИЗАЦИЯ И ИНТЕГРАЦИЯ МИКРОСЕРВИСОВ

3.1 Структура проекта и файловая организация

3.1.1 Общая архитектура проекта

Платформа DeliveryMatch реализована как набор из четырех микросервисов, каждый из которых отвечает за конкретную бизнес-функцию. Такая архитектура обеспечивает независимое разв ертывание, масштабирование и разработку компонентов. Ключевое решение — интеграция алгоритмического ядра (сопоставление, маршрутизация, упаковка, отслеживание, уведомления, платежи) в сервисы Order и Driver, что упрощает взаимодействие и снижает сетевое воздействие. Общая архитектурная схема платформы DeliveryMatch представлена на рисунке 3.1.

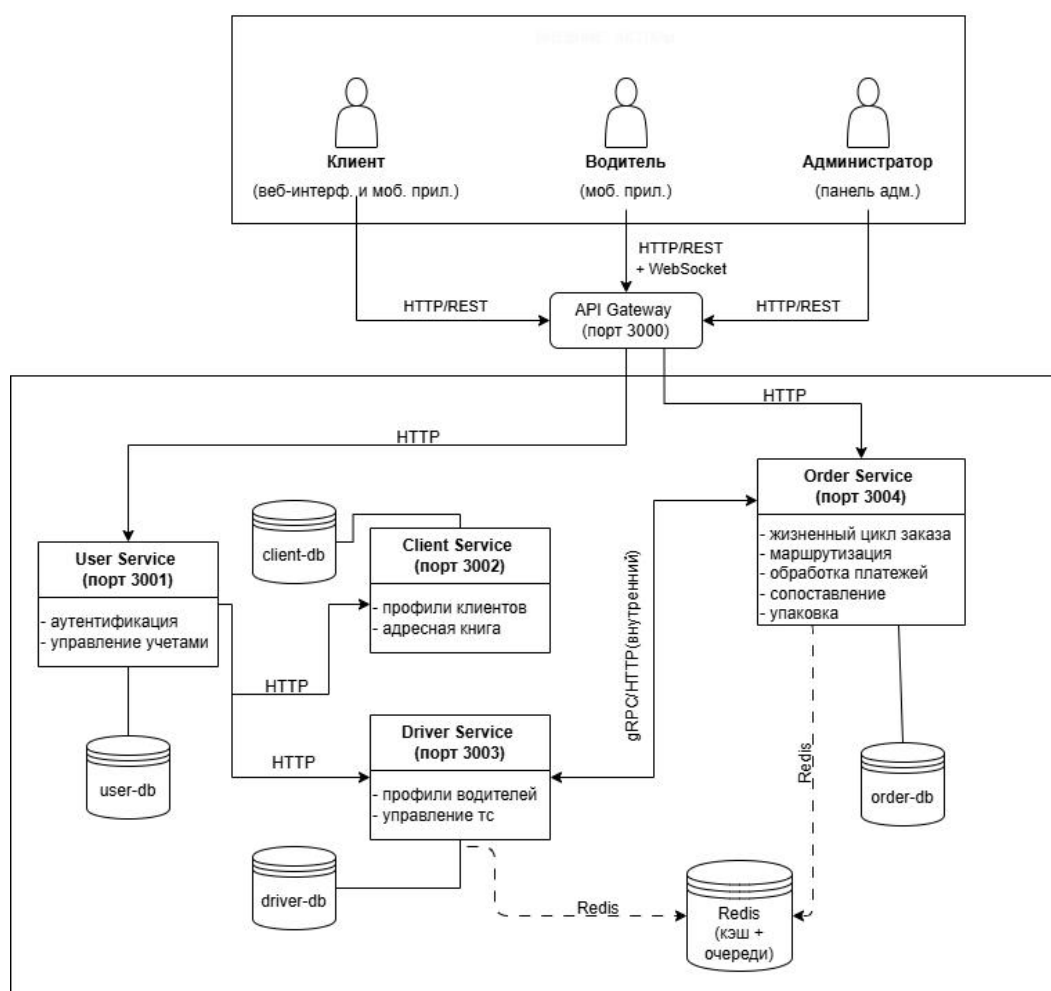


Рисунок 3.1 – Общая архитектурная схема микросервисов

На рисунке 3.1 представлена общая архитектурная схема платформы DeliveryMatch. API Gateway выступает единой точкой входа для клиентских

приложений (веб-интерфейс клиента, приложение водителя, административная панель). Gateway маршрутизирует запросы к четырем микросервисам: User Service (аутентификация и управление учетными записями), Client Service (управление профилями клиентов и адресной книгой), Driver Service (управление профилями водителей, транспортными средствами, а также интегрированные модули отслеживания местоположения, уведомлений и платежей для водителей), Order Service (полный жизненный цикл заказа, включая интегрированные модули интеллектуального сопоставления, оптимизации маршрутов, планирования загрузки и обработки платежей). Каждый микросервис имеет собственную базу данных PostgreSQL, а Redis используется для кэширования геопозиций водителей и организации очередей фоновых задач.

Структура исходного кода организована следующим образом:

```

...
delivery-platform/
├── docker-compose.yml      # Оркестрация контейнеров
├── .env.example            # Шаблон переменных окружения
├── shared/                 # Общие модули
│   ├── types/              # TypeScript интерфейсы
│   ├── utils/              # Утилиты (логирование, валидация)
│   └── middleware/         # Общие middleware
├── microservices/         # Микросервисы
│   ├── user-service/      # Аутентификация и пользователи
│   ├── client-service/    # Профили клиентов
│   ├── driver-service/    # Профили водителей + трекинг
│   │                       уведомлений + выплаты
│   └── order-service/     # Заказы + сопоставление + маршрутизация +
│                           упаковка + платежи
├── api-gateway/           # API Gateway
├── frontend/              # Клиентские приложения
│   ├── client-web/        # Веб-интерфейс клиента
│   └── driver-app/        # Приложение водителя
├── infrastructure/        # Инфраструктура
│   ├── kubernetes/        # Манифесты для K8s
│   ├── monitoring/        # Конфигурация мониторинга
│   └── ci-cd/              # Паплайны CI/CD
...

```

Схема развертывания микросервисов в Docker представлена на рисунке 3.2.

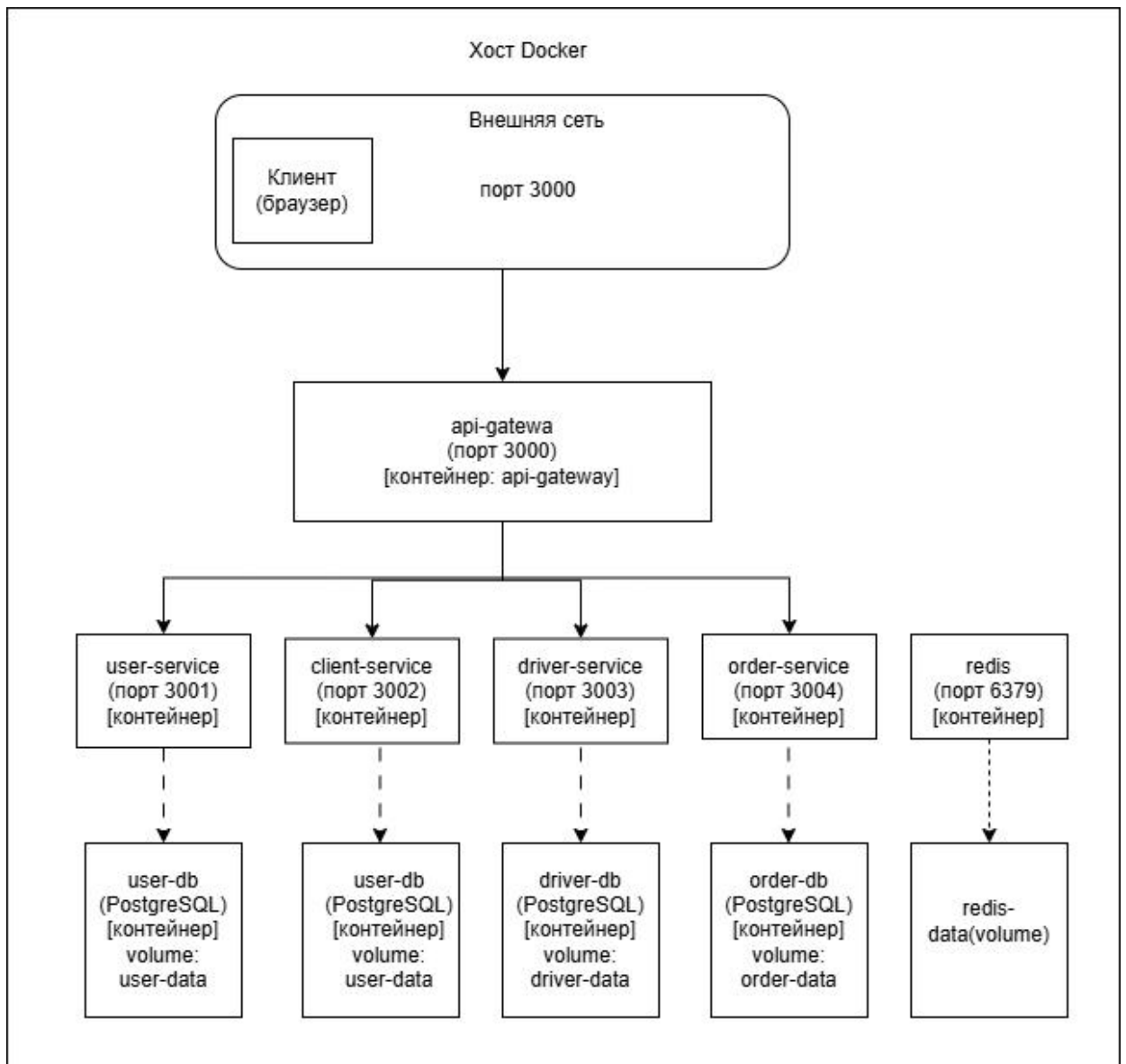


Рисунок 3.2 – Схема взаимодействия контейнеров Docker

На рисунке 3.2 представлена схема развертывания микросервисов в Docker. Каждый сервис запущен в отдельном контейнере. API Gateway принимает внешние запросы на порту 3000 и маршрутизирует их к соответствующим сервисам. Базы данных MySQL запущены в отдельных контейнерах с проброшенными портами для доступа извне при разработке. Redis используется как кэш для геопозиций водителей и брокер сообщений для асинхронных задач (уведомления, фоновые расчеты). Все сервисы объединены в общую сеть Docker для обеспечения безопасного внутреннего взаимодействия.

3.1.2 Конфигурация Docker и оркестрация

Для контейнеризации всех компонентов платформы используется Docker. Основной файл `docker-compose.yml` описывает все четыре сервиса, их зависимости и переменные окружения. Полный листинг конфигурации представлен в приложении А.

Детальная схема взаимодействия четырех микросервисов с указанием портов и зависимостей представлена на рисунке 3.3.

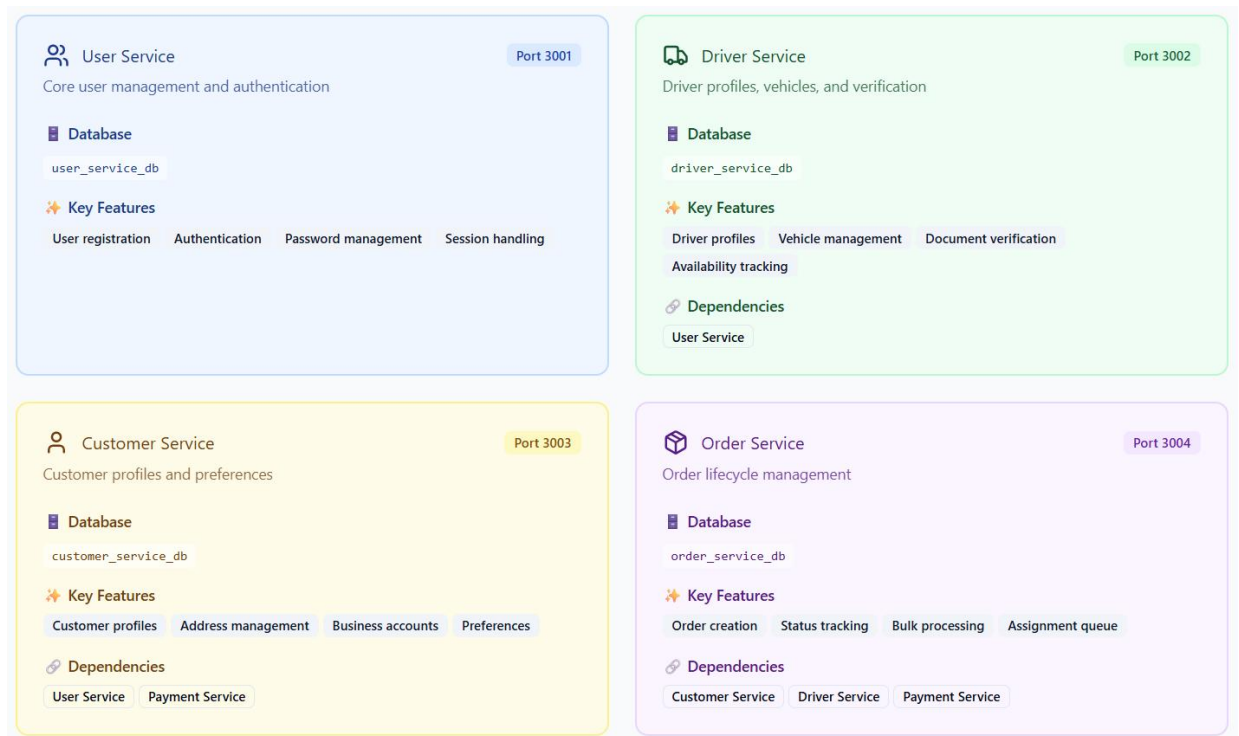


Рисунок 3.3 – Детальная схема микросервисов с портами и зависимостями

На рисунке 3.3 представлена детальная схема взаимодействия четырех микросервисов с указанием портов и зависимостей. User Service (3001) — базовый сервис аутентификации, не имеет зависимостей от других сервисов. Client Service (3002) зависит от User Service для получения данных пользователя. Driver Service (3003) зависит от User Service и Order Service (для получения информации о заказах при расчете заработка). Order Service (3004) зависит от Client Service, Driver Service и использует Redis для кэширования и очередей. Все сервисы имеют собственные базы данных PostgreSQL, что соответствует принципам микросервисной архитектуры [16].

3.2 Пользовательские интерфейсы

3.2.1 Клиентский веб-интерфейс

Клиентский веб-интерфейс реализован на React 18 с использованием TypeScript и Tailwind CSS. Интерфейс оптимизирован для быстрого создания заказов и отслеживания доставки [17].

На рисунке 3.4 представлен экран создания заказа для клиента. Интерфейс включает форму для ввода адресов отправления и назначения, выпадающий список категории товара, поля для оценки веса и объёма, срочность доставки и спецификацию требований.

The screenshot shows the 'ДоставкаМэтч' (DeliveryMatch) client interface. At the top, there's a header with the company logo, a 'Клиент' (Client) tab, and a language selector set to 'Русский' (Russian) with a 'Выйти' (Logout) button. Below the header is a navigation bar with four tabs: 'Новый заказ' (New order), 'Отслеживание заказов' (Order tracking), 'Уведомления' (Notifications), and 'Профиль' (Profile). The main content area is titled 'Калькулятор стоимости доставки' (Delivery cost calculator) and includes the text 'Получите мгновенную оценку стоимости вашей доставки' (Get an instant estimate of your delivery cost). A large dashed box prompts the user to 'Заполните все поля для отображения цены' (Fill out all fields to display the price). The form contains several input fields: 'Адрес забора *' (Pickup address *) with the value 'Zagorodnyi prospekt, 24, Sankt-Peterburg, Russia, 191002', 'Адрес доставки *' (Delivery address *) with the value 'Khersonskiy proyezd 4A', a dropdown for 'Категория товара *' (Goods category *) set to 'Строительные материалы' (Building materials), 'Вес (кг) *' (Weight (kg) *) set to '200', and 'Объём (м³) *' (Volume (m³) *) set to 'напр., 0.5'. There is also a dropdown for 'Срочность доставки' (Delivery urgency) set to 'Срочно (в течение 2 часов)' (Urgent (within 2 hours)). At the bottom of the form, there are checkboxes for 'Особые требования' (Special requirements): 'Хрупкие предметы' (Fragile items), 'Охлаждаемые' (Refrigerated), 'Крупногабаритные' (Oversized) which is checked, and 'Опасные грузы' (Dangerous goods) which is also checked. A large green button at the bottom of the form area says 'Заполните все поля для отображения цены' (Fill out all fields to display the price). Below the button, there is a small disclaimer: 'Цена обновляется автоматически по мере заполнения формы' (Price is updated automatically as the form is filled), 'Используется ИИ-прогнозирование стоимости доставки с точностью 81%' (AI delivery cost forecasting is used with 81% accuracy), and 'Точные координаты автоматически получают для точного расчета расстояния' (Accurate coordinates are automatically obtained for accurate distance calculation).

Рисунок 3.4 – Экран создания заказа

После заполнения основных полей система автоматически рассчитывает стоимость перевозки и отображает топ-4 варианта транспортных средств/режимов перевозки для выбора клиентом.

На рисунке 3.5 показан автоматический расчёт стоимости транспортировки и список наиболее подходящих водителей, которых клиент

может выбрать. Клиент также может просмотреть детали выбранного им транспортного средства.

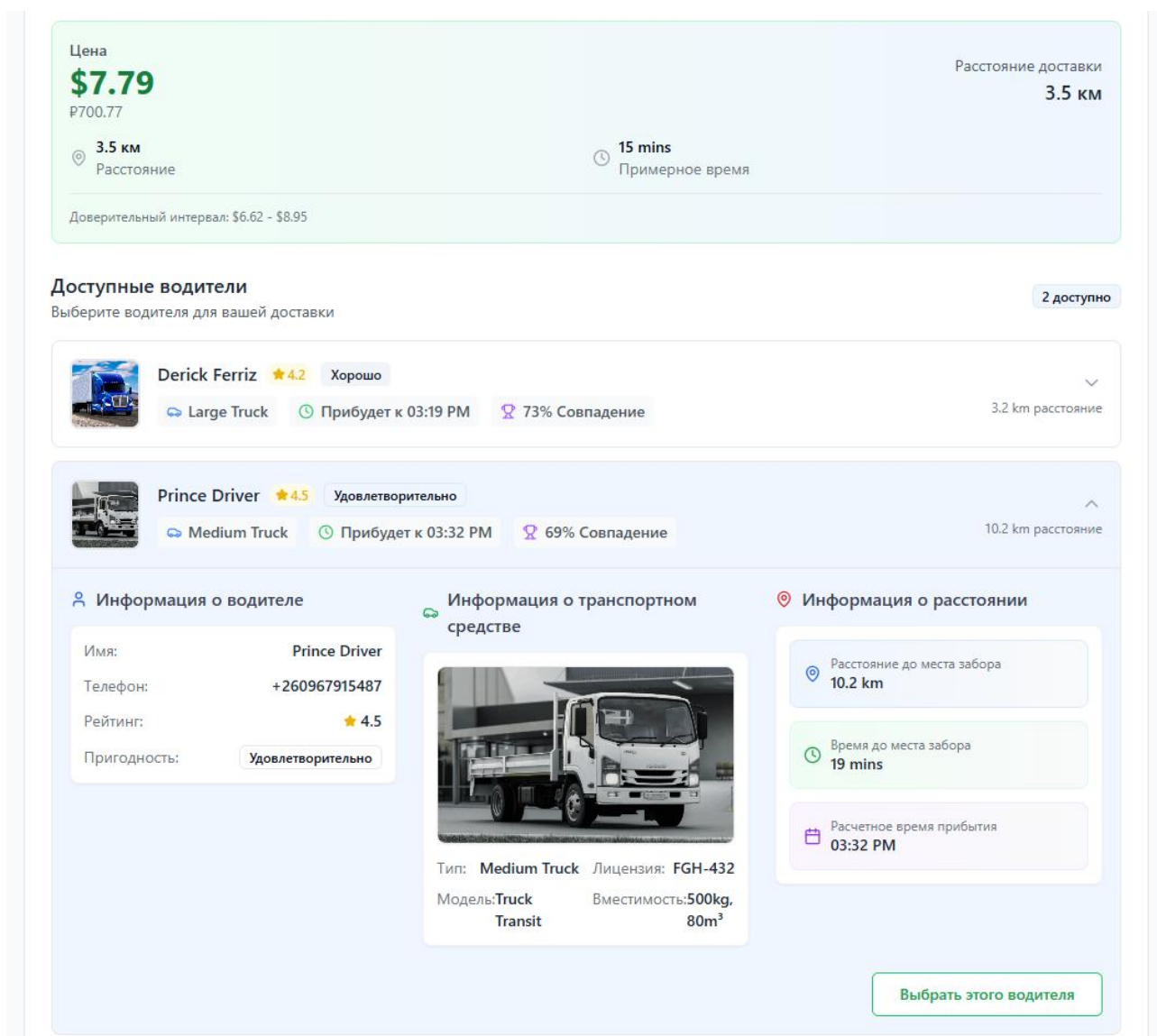


Рисунок 3.5 – Автоматический расчёт стоимости и выбор водителя
На рисунке 3.6 показан экран проверки заказа перед его отправкой.

Предпросмотр заказа

ID заказа: ORD-17779840...

Просмотрите все детали перед подтверждением заказа на доставку

Сводка доставки

Все детали собраны и проверены

\$8.70

₮783.24

Информация о клиенте

Имя: Makolas Jones

Email: mwale22@gmail.com

Телефон: 0963884441

Детали местоположения

Место забора

Voznesenskiy prospekt 6
Russia

Координаты:
• 56.638567,
47.905475

Место доставки

Khersonskiy prospekt 4A

Координаты:
• 50.850000,
38.316700

Расстояние
9.2 км

Примерная
длительность
16 mins

Детали посылки

Категория: Мебель и бытовая техника

Вес	Объем
54 кг	10 м³

Срочность доставки: Срочно (в течение 2 часов)

Водитель и транспорт

Lieson Mwale

4.8

60% Совпадение

Детализация цены

Цена \$8.70

Включает все сборы и налоги

₮783.24

Просмотр полной структуры данных (JSON)

Отмена

Подтвердить и создать заказ

Рисунок 3.6 – Экран проверки и подтверждения заказа

Отображаются стоимость перевозки, информация о клиенте и водителе, адреса отправления и назначения, предполагаемое время в пути и расстояние,

а также детали заказа. Клиент может подтвердить заказ только после ознакомления со всей информацией.

На рисунке 3.7 представлен экран отслеживания заказа. Он содержит панель с отображением предполагаемого времени прибытия (ETA), статуса заказа и контактных данных водителя.

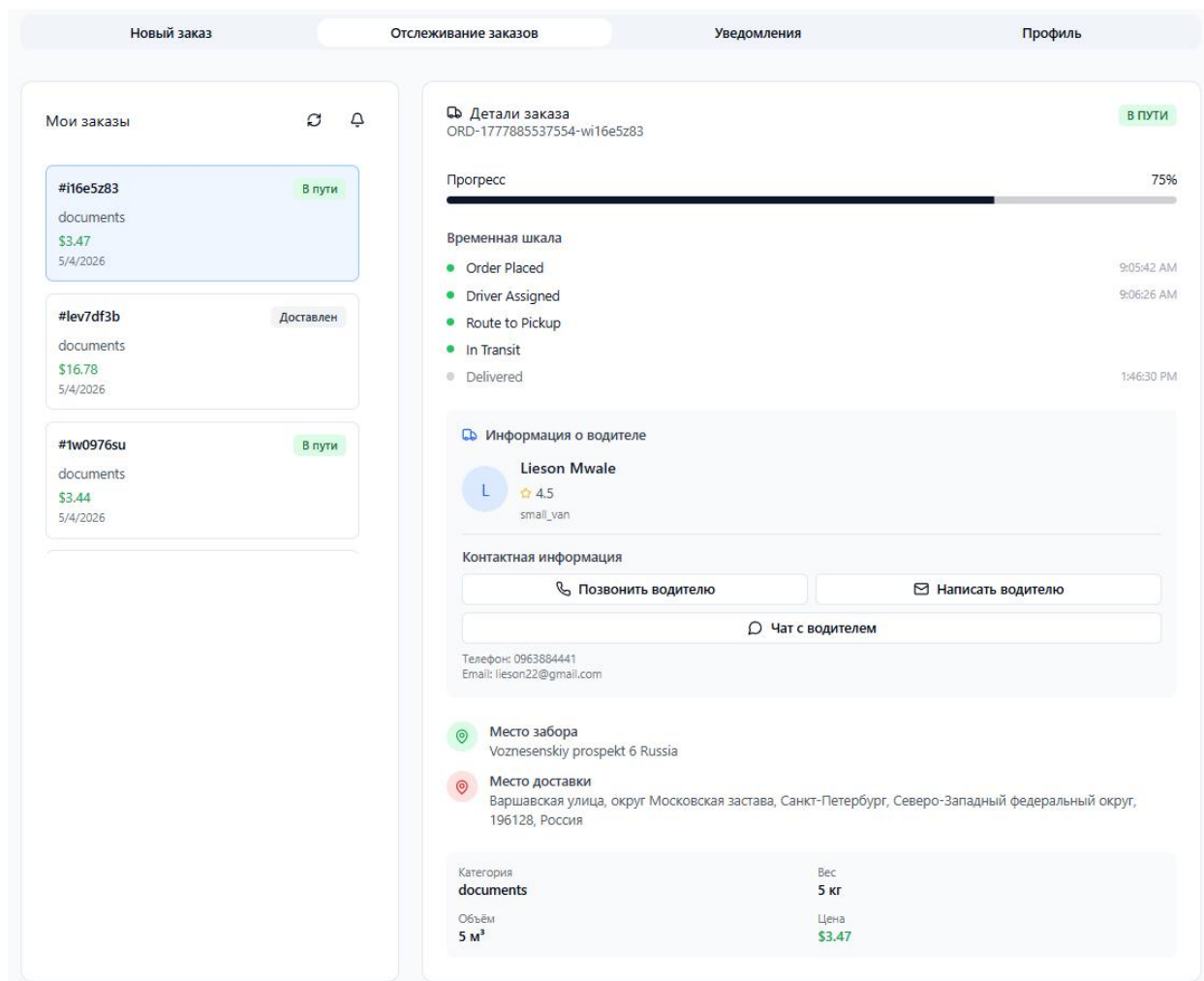


Рисунок 3.7 – Экран отслеживания заказа в реальном времени

Индикатор выполнения иллюстрирует этапы доставки: «Заказ оформлен» → «Водитель назначен» → «Маршрут до места забора» → «В пути» → «Доставлен». Слева отображаются все заказы, которые оформил клиент, с указанием их статуса. Клиент может нажать на конкретный заказ и просмотреть его детали, включая чат с соответствующим водителем, как показано на рисунке 3.8.

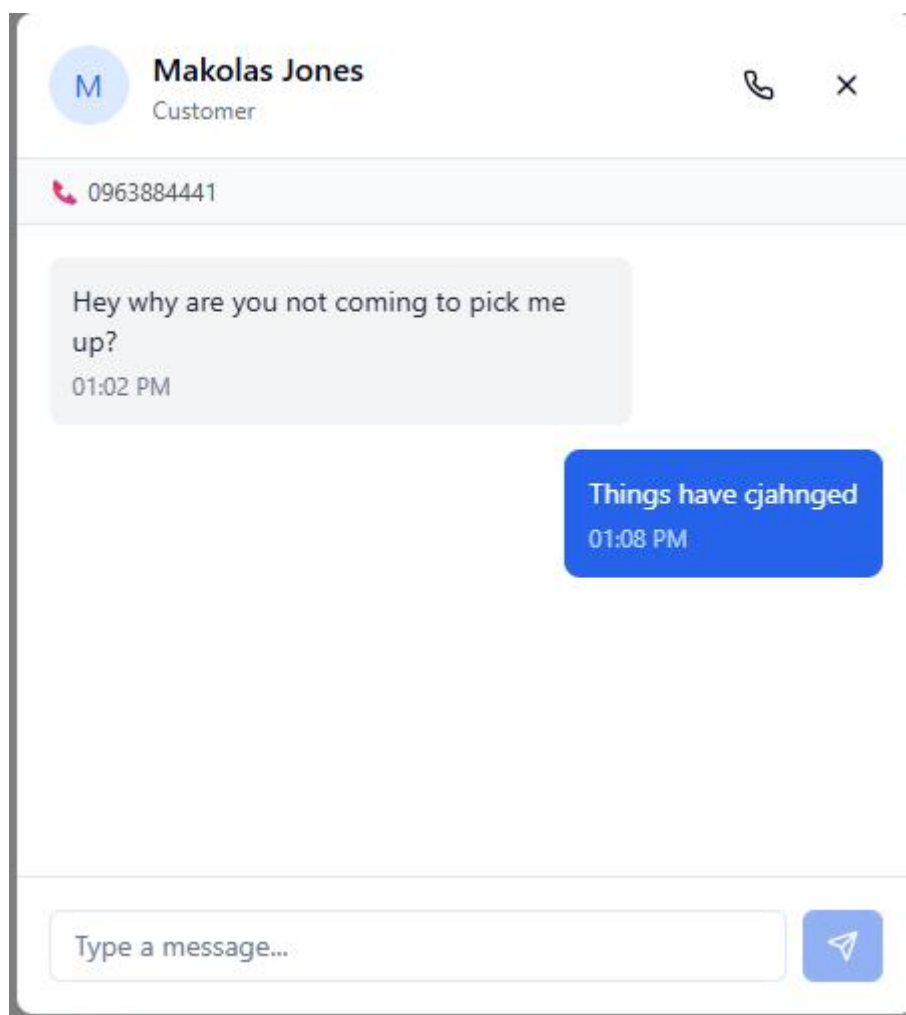


Рисунок 3.8 – Чат с водителем

Как упоминается на рисунке 3.8, данный рисунок показывает интерфейс чата между клиентом и конкретным водителем.

3.2.2 Приложение для водителей

Приложение для водителей реализовано как адаптивный веб-интерфейс, оптимизированный для использования на мобильных устройствах. Основное внимание уделено крупным элементам управления и пошаговой навигации.

На рисунке 3.9 представлен главный экран панели управления водителя. Крупный тумблер «Выйти на линию / Занят» позволяет водителю управлять статусом доступности в реальном времени. В левой части панели отображается активное транспортное средство водителя, включая такие ключевые характеристики, как тип автомобиля, государственный номер, максимальную грузоподъемность (вес и объем) и фотографию автомобиля.

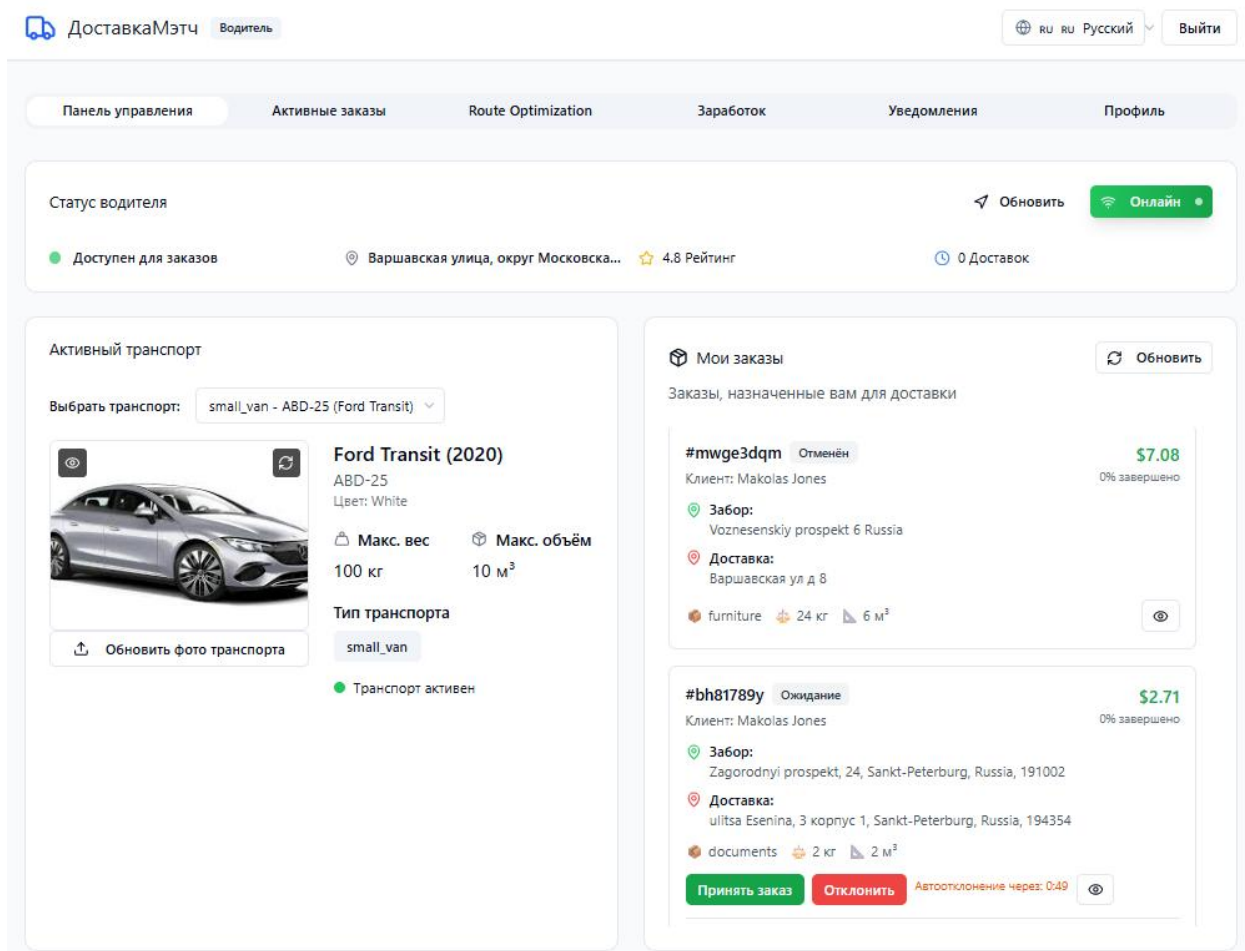


Рисунок 3.9 – Панель управления водителя с информацией о транспортном средстве и доступными заказами

Этот раздел позволяет водителю убедиться, что его транспортное средство соответствует требованиям предстоящих доставок. В правой части находится прокручиваемый список всех заказов, размещенных клиентами и доступных для назначения. Каждая карточка заказа отображает критически важную информацию: адреса забора и доставки, расстояние, предполагаемый заработок и детали посылки. Водитель имеет возможность либо принять, либо отклонить каждый заказ. Если водитель не отвечает в течение установленного времени (например, двух минут), система автоматически отклоняет заказ, позволяя назначить его другому доступному водителю, что обеспечивает своевременное обслуживание клиентов.

На рисунке 3.10 показан список заказов, которые в данный момент назначены водителю и обрабатываются им. Каждый заказ четко маркирован

текущим статусом (например, водитель назначен, маршрут до забора, в пути, доставлен). Водитель может нажать на любой заказ, чтобы развернуть его и просмотреть подробную информацию.

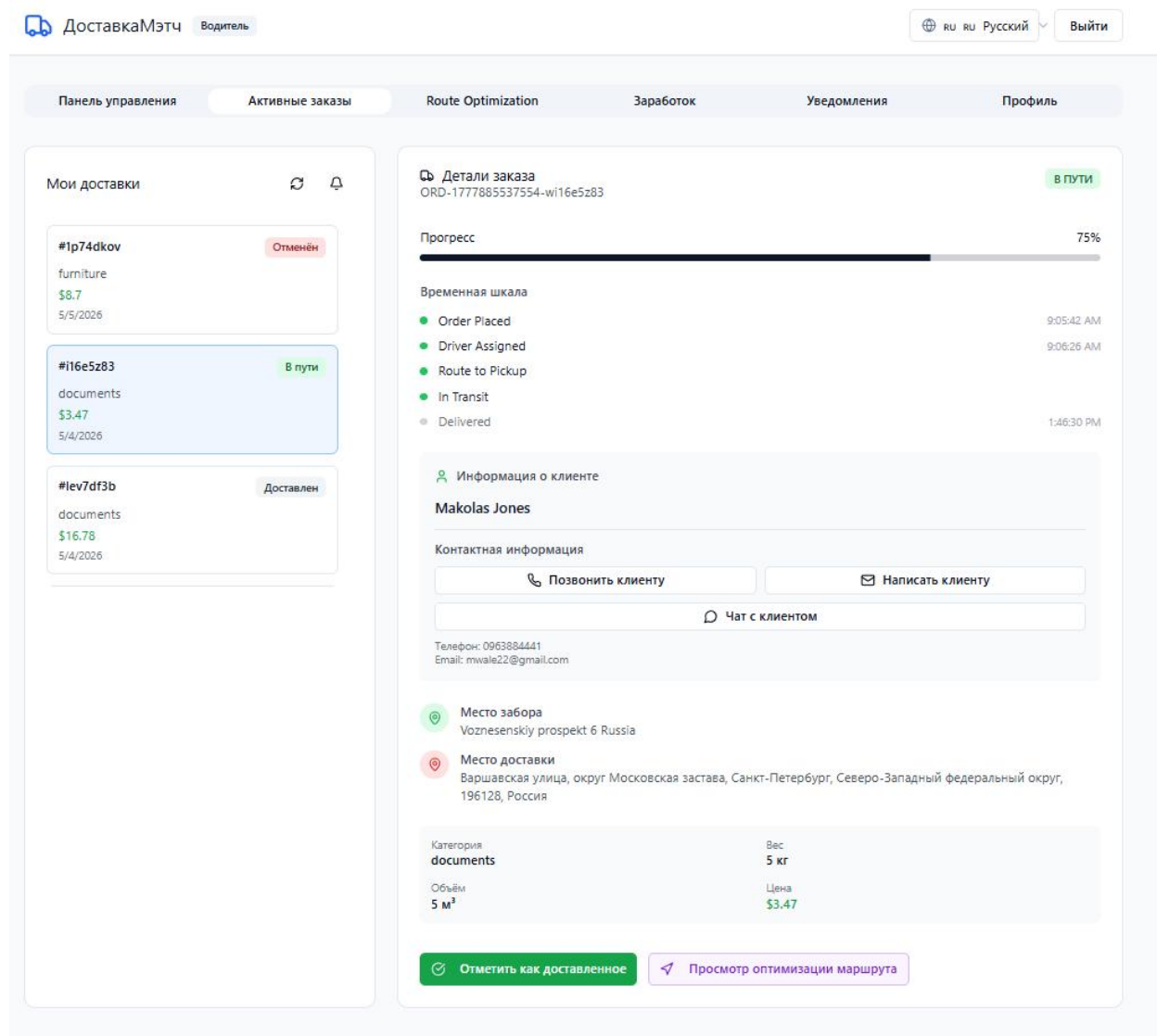


Рисунок 3.10 – Список активных заказов с детальным просмотром

Этот расширенный вид включает полную временную шкалу заказа, историю статусов, детальную разбивку информации о посылке (категория, вес, объем, цена), а также контактную информацию клиента, включая номер телефона и адрес электронной почты. Встроенные инструменты коммуникации позволяют водителю позвонить, отправить электронное письмо или написать сообщение в приложении клиенту непосредственно с этого экрана. Кроме того, специальная кнопка предоставляет доступ к оптимизации маршрута для управления несколькими невыполненными

заказами одновременно, помогая водителю спланировать наиболее эффективную последовательность доставки.

На рисунке 3.11 продемонстрирована функция оптимизации маршрута, интегрированная в картографический интерфейс. Все соответствующие остановки (точки забора и доставки) четко отмечены на карте в виде пронумерованных точек, указывающих оптимальную последовательность, которой должен следовать водитель для максимальной эффективности.

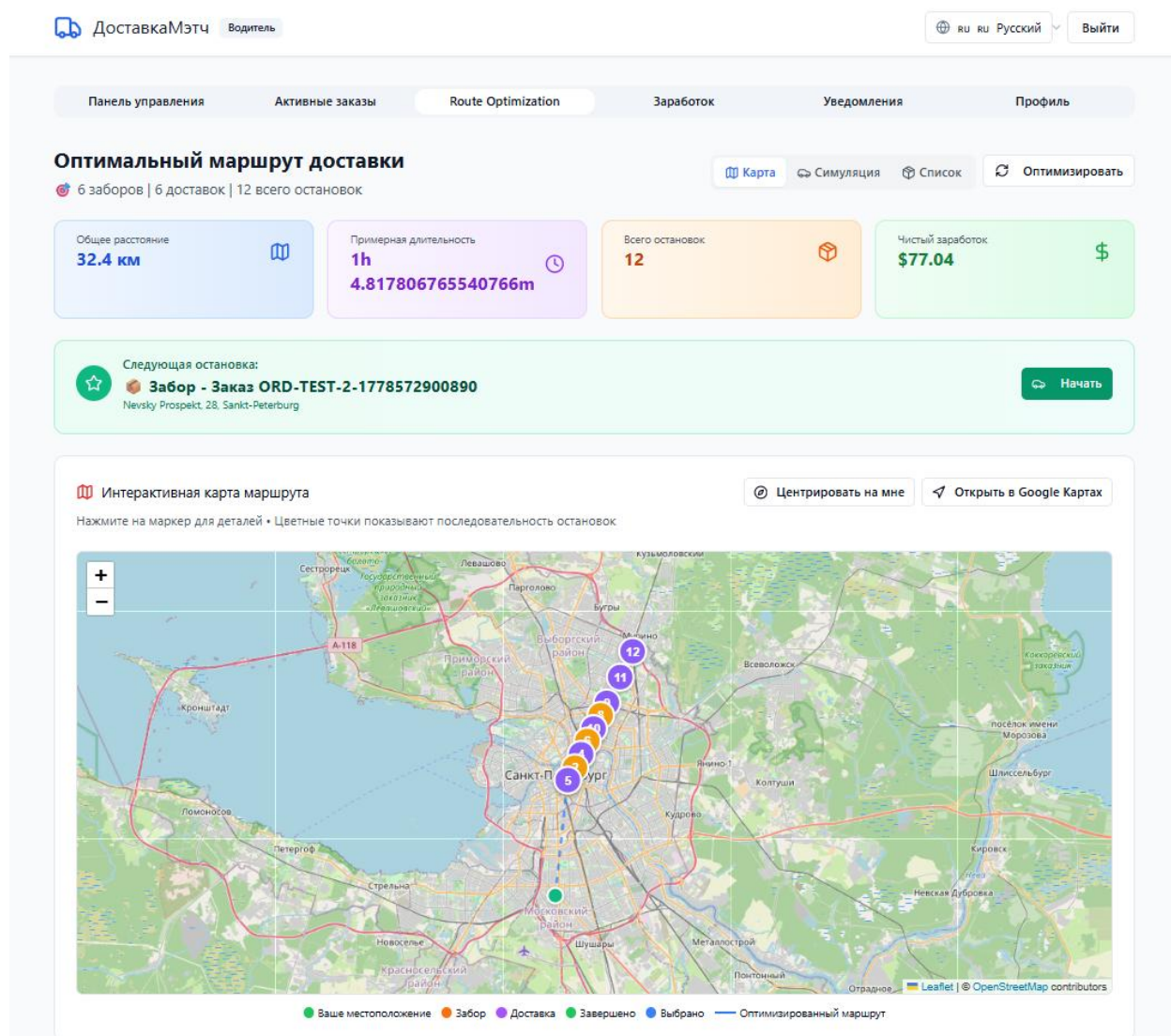


Рисунок 3.11 – Обзор карты с оптимизацией маршрута

Оптимальный маршрут изображен в виде пунктирной или сплошной линии, соединяющей эти точки. Боковая панель на экране отображает сводные статистические данные для всего набора одновременно обрабатываемых заказов, включая общее расстояние, которое необходимо

преодолеть, общую расчетную продолжительность, общее количество остановок и общий прогнозируемый заработок. Крупная кнопка «Начать маршрут» инициирует навигацию по оптимизированному пути, направляя водителя шаг за шагом.

На рисунке 3.12 показан режим симуляции функции оптимизации маршрута. В этом представлении упрощенная, стилизованная карта (на основе Canvas) обеспечивает четкую, не перегруженную деталями визуализацию оптимизированного маршрута.

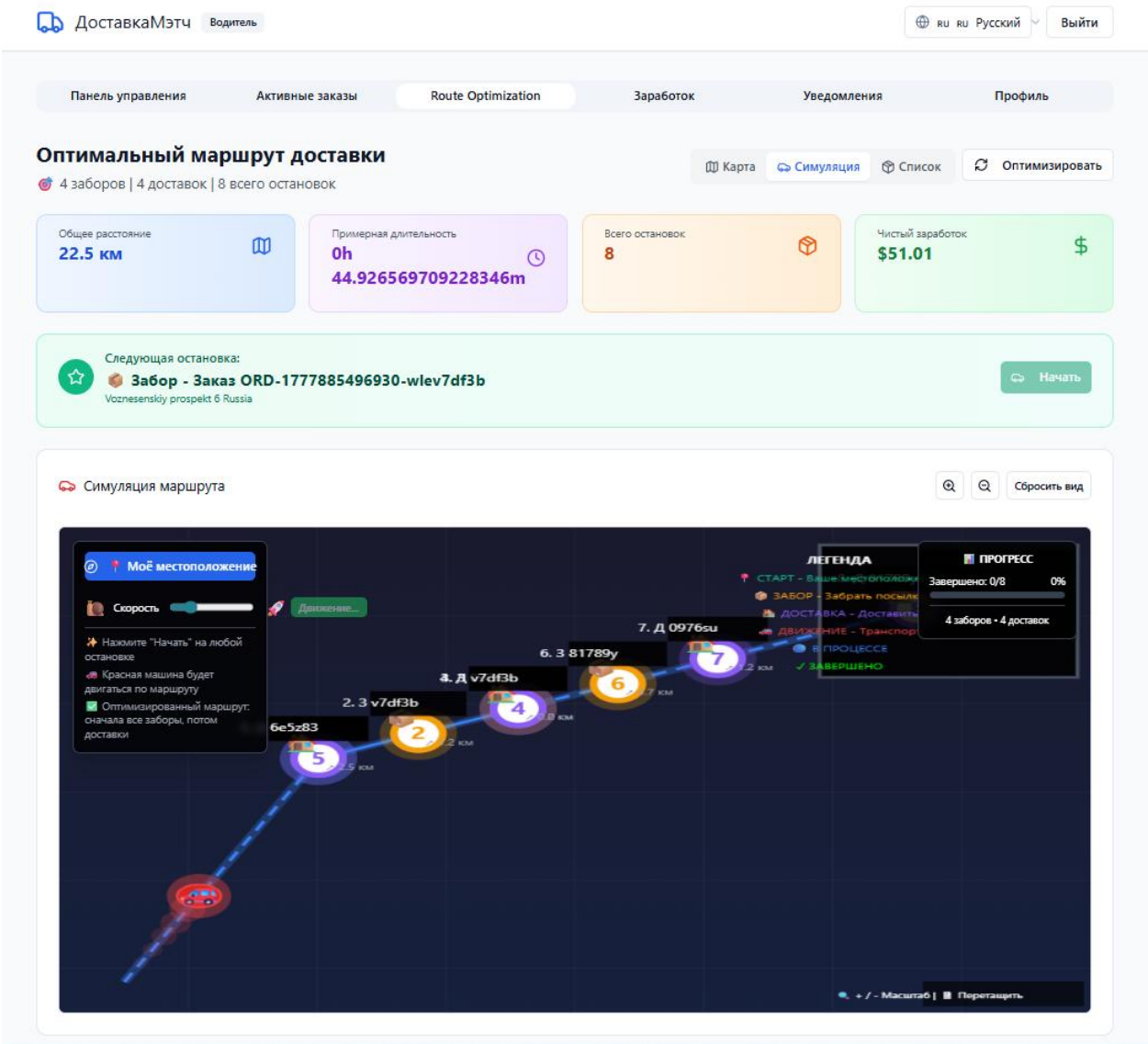


Рисунок 3.12 – Режим симуляции маршрута

Анимированный значок автомобиля движется вдоль пути между остановками, обеспечивая интуитивно понятное представление о последовательности вождения и относительных расстояниях. Этот режим

симуляции особенно полезен, когда реальная карта перегружена излишними деталями, предлагая водителям обзор всего маршрута доставки на высоком уровне, за которым легко следить.

На рисунке 3.13 представлена функция оптимизации маршрута в формате хронологического списка. Каждая остановка заказа отображается в точной последовательности, соответствующей запланированному движению автомобиля по оптимизированному маршруту.

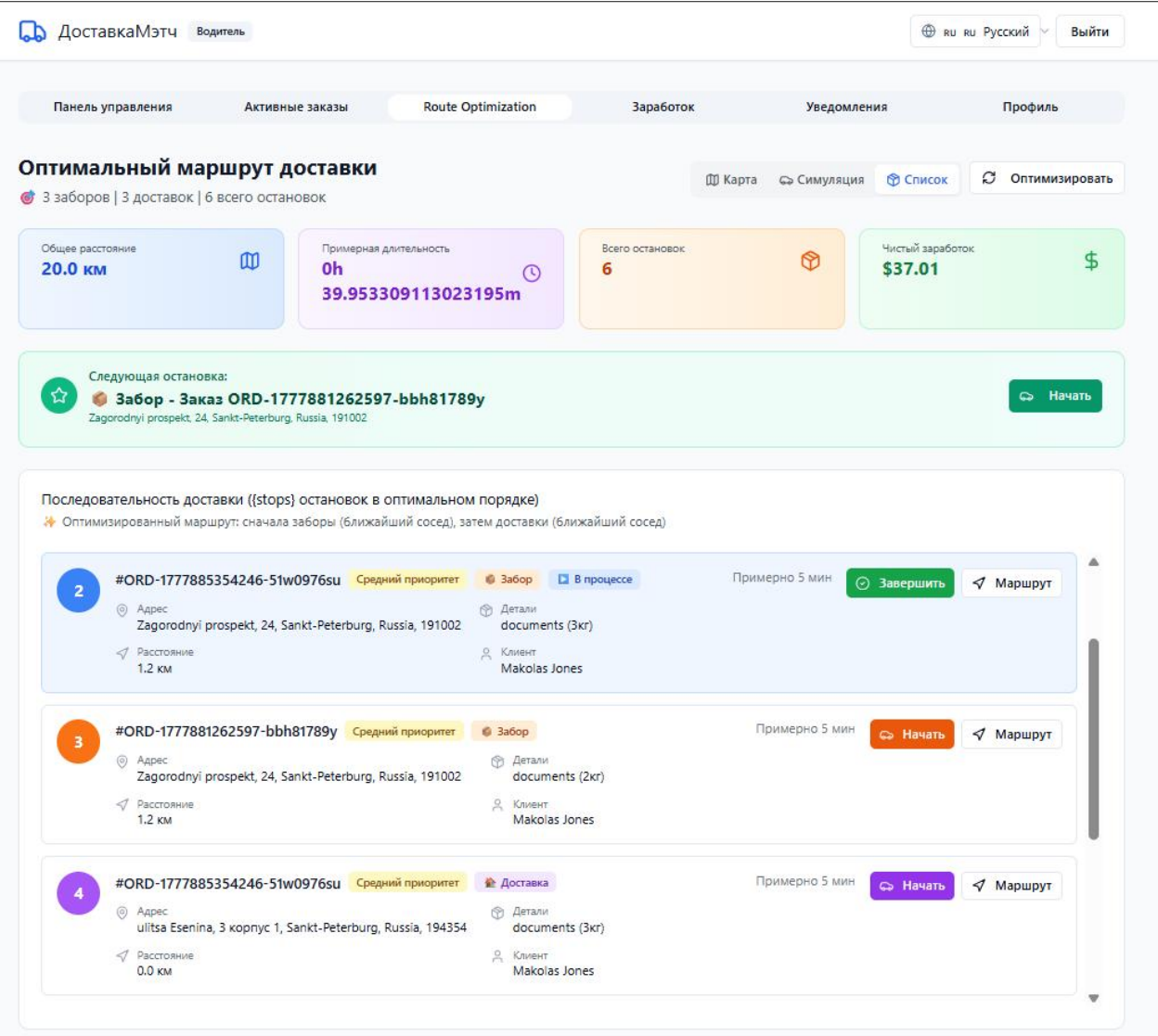


Рисунок 3.13 – Хронологический список для оптимизированного маршрута

Для каждой остановки интерфейс показывает подробную информацию о том, что происходит в данный момент, включая тип остановки (забор или доставка), адрес, детали посылки и расчетное время. Кроме того, каждый элемент списка включает интерактивную кнопку, которая при нажатии

отслеживает движение автомобиля между текущей точкой и следующим пунктом назначения на карте в реальном времени, предлагая синхронизированное двойное представление, которое помогает водителям отслеживать прогресс в реальном времени.

На рисунке 3.14 показан комплексный раздел управления финансами для водителей.

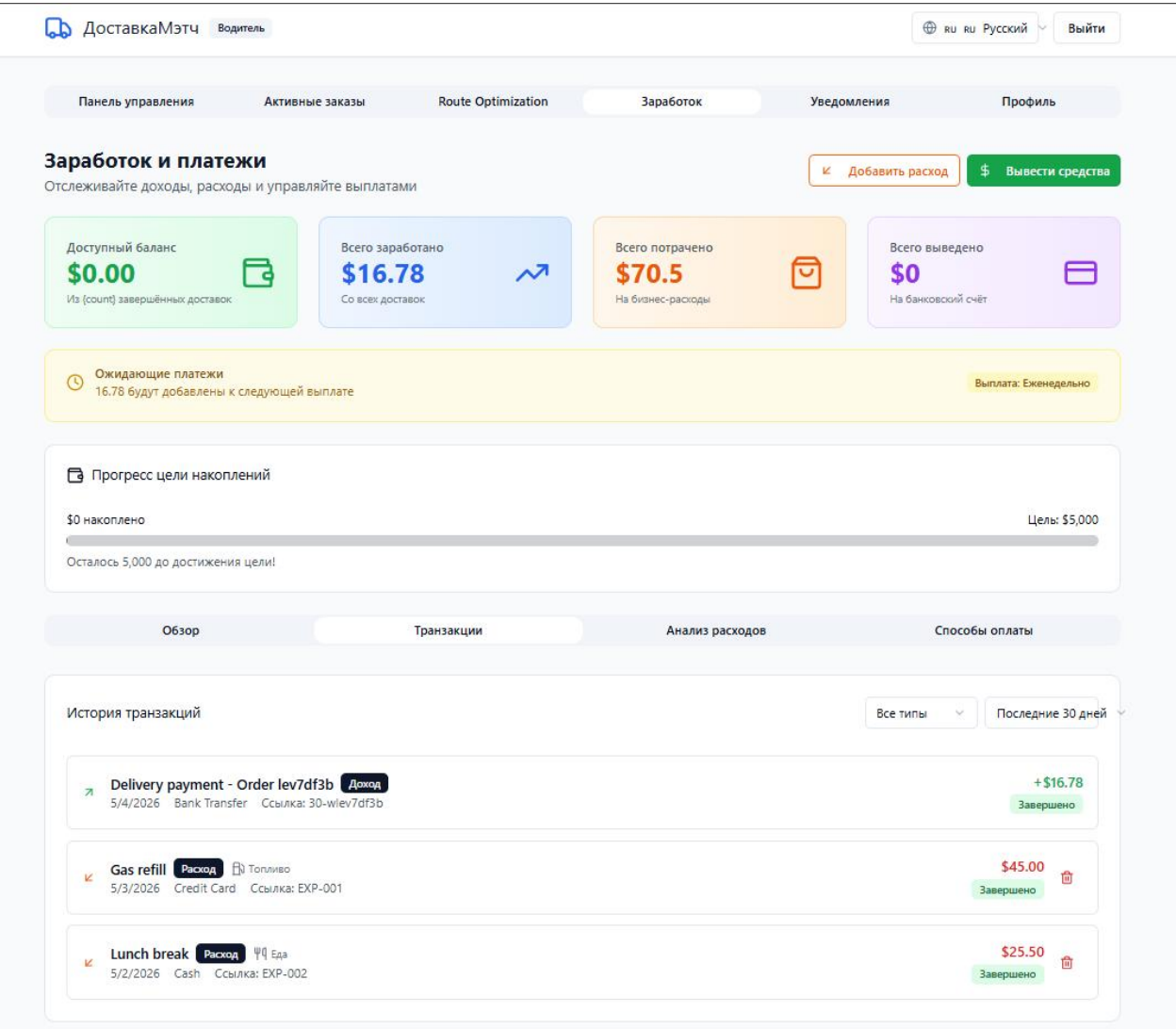


Рисунок 3.14 – Панель управления заработком, расходами и статистикой водителя

Эта панель предоставляет полный обзор заработка, расходов и ключевых статистических показателей за настраиваемые периоды времени (например, понедельно, ежемесячно, годового). Она сохраняет полную историю транзакций, включая все выполненные доставки, полученные

выплаты, заработанные бонусы и понесенные комиссии. Выделенный модуль анализа расходов разбивает затраты по категориям (топливо, питание, техническое обслуживание, расходные материалы, прочее), помогая водителям эффективно управлять своими бизнес-расходами. Раздел «Способы оплаты» позволяет водителям добавлять, редактировать или удалять привязанные платежные счета (например, банковские счета или цифровые кошельки). Все финансовые данные сохраняются в базе данных, гарантируя, что никакая историческая информация не теряется при перезапуске сервера, позволяя водителям отслеживать свои финансовые показатели в долгосрочной перспективе.

3.3 Реализация REST API и взаимодействия микросервисов

3.3.1 Проектирование REST API

API-шлюз выступает единой точкой входа для всех клиентских запросов. Каждый запрос проходит аутентификацию и маршрутизируется к соответствующему микросервису. Основные эндпоинты REST API приведены в таблице 3.1.

Таблица 3.1 – Основные эндпоинты REST API

Метод	Эндпоинт	Сервис	Описание
POST	/api/auth/register	User Service	Регистрация нового пользователя
POST	/api/auth/login	User Service	Аутентификация, возврат JWT
POST	/api/auth/refresh	User Service	Обновление токена
GET	/api/clients/profile	Client Service	Получение профиля клиента
PUT	/api/clients/profile	Client Service	Обновление профиля клиента
GET	/api/clients/addresses	Client Service	Получение адресной книги
GET	/api/drivers/profile	Driver Service	Получение профиля водителя

Продолжение таблицы 3.1

PUT	/api/drivers/profile	Driver Service	Обновление профиля водителя
GET	/api/drivers/vehicles	Driver Service	Получение данных ТС
POST	/api/drivers/location	Driver Service	Обновление геопозиции водителя
GET	/api/drivers/earnings	Driver Service	Получение заработка водителя
POST	/api/orders	Order Service	Создание нового заказа
GET	/api/orders/:id	Order Service	Получение информации о заказе
GET	/api/orders/:id/tracking	Order Service	Получение данных отслеживания
POST	/api/orders/bulk	Order Service	Пакетное создание заказов
POST	/api/orders/:id/accept	Order Service	Принятие заказа водителем
POST	/api/orders/:id/status	Order Service	Обновление статуса заказа
POST	/api/payments	Order Service	Обработка платежа

3.4 Объектно-ориентированная модель системы

Для документирования структуры системы разработана диаграмма классов UML, отражающая основные сущности, их атрибуты, методы и взаимосвязи.

3.4.1 Основные классы домена

В таблицах 3.2–3.5 представлены классы домена, алгоритмические классы и классы поддержки.

Таблица 3.2 – Абстрактный базовый класс

Класс	Комментарий
User	Базовый абстрактный класс для всех пользователей системы. Содержит общую аутентификационную информацию и профильные данные.

Таблица 3.3 – Основные классы домена

Класс	Комментарий
Client	Представляет клиента (грузоотправителя). Наследует User. Отвечает за создание заказов и их отслеживание.
Driver	Представляет водителя-партнера. Наследует User. Управляет местоположением, принимает заказы и выполняет доставки.
Vehicle	Транспортное средство водителя. Содержит технические характеристики для проверки совместимости с грузом.
Order	Центральная сущность системы. Представляет заказ на доставку. Содержит всю информацию о грузе, статусе и назначенных исполнителях.
OrderItem	Отдельная товарная позиция внутри заказа. Используется для детализации состава груза и расчета загрузки.

Таблица 3.4 – Алгоритмические классы

Класс	Комментарий
MatchingEngine	Реализует алгоритмы интеллектуального сопоставления заказов с водителями. Включает геопространственный поиск (k-NN, Geohash), проверку выполнимости (задача о рюкзаке) и ранжирование кандидатов.
Route	Отвечает за оптимизацию маршрута доставки. Решает задачу маршрутизации (VRP) и расчета ETA.
LoadingPlan	Генерирует схему размещения груза в транспортном средстве. Решает задачу трехмерной упаковки (3D Bin Packing) и проверяет устойчивость загрузки.

Таблица 3.5 – Классы поддержки

Класс	Комментарий
Payment	Обрабатывает финансовые транзакции между клиентами, водителями и платформой.
Tracking	Обеспечивает отслеживание местоположения в реальном времени через WebSocket.
Notification	Управляет отправкой уведомлений пользователям о событиях в системе.
PriceModel	Реализует динамическую тарифную модель на основе расстояния, веса, объема и текущего спроса.

3.4.2 Сигнатуры методов с типами возвращаемых данных

Ключевые методы классов с их сигнатурами и типами возвращаемых данных приведены в таблице 3.6.

Таблица 3.6 – Ключевые методы классов

Класс	Метод	Тип возврата	Описание
User	register(data: UserDTO)	bool	Регистрирует нового пользователя
User	login(credentials: LoginDTO)	Token	Аутентифицирует пользователя и возвращает токен
Client	createOrder(orderData: OrderDTO)	Order	Создает новый заказ
Client	trackOrder(orderId: int)	TrackingInfo	Возвращает текущее местоположение заказа
Driver	updateLocation(lat: float, lng: float)	bool	Обновляет геопозицию водителя

Продолжение таблицы 3.6

Driver	acceptOrder(orderId: int)	bool	Принимает предложенный заказ
Vehicle	isAvailableForOrder(weight: float, volume: float)	bool	Проверяет, подходит ли ТС для заказа
Order	validate()	bool	Проверяет корректность данных заказа
Order	calculatePrice()	float	Вычисляет стоимость доставки
MatchingEngine	findNearbyDrivers(orderId: int, radius: float)	List<Driver>	Ищет водителей в радиусе
MatchingEngine	knapsackOptimize(order: Order, drivers: List<Driver>)	Driver	Решает задачу о рюкзаке для выбора водителя
Route	optimize(waypoints: List<Point>)	Route	Строит оптимальный маршрут
LoadingPlan	calculatePacking(items: List<Item>)	bool	Рассчитывает схему размещения груза
LoadingPlan	validateStability()	bool	Проверяет устойчивость загрузки
Payment	processPayment()	PaymentResult	Проводит платеж
Tracking	getLastLocation(orderId: int)	Tracking	Возвращает последнюю известную позицию
PriceModel	calculatePrice(distance: float, weight: float, volume: float, demand: float)	float	Рассчитывает стоимость по тарифной модели

Полная диаграмма классов UML для платформы DeliveryMatch представлена на рисунке 3.15.

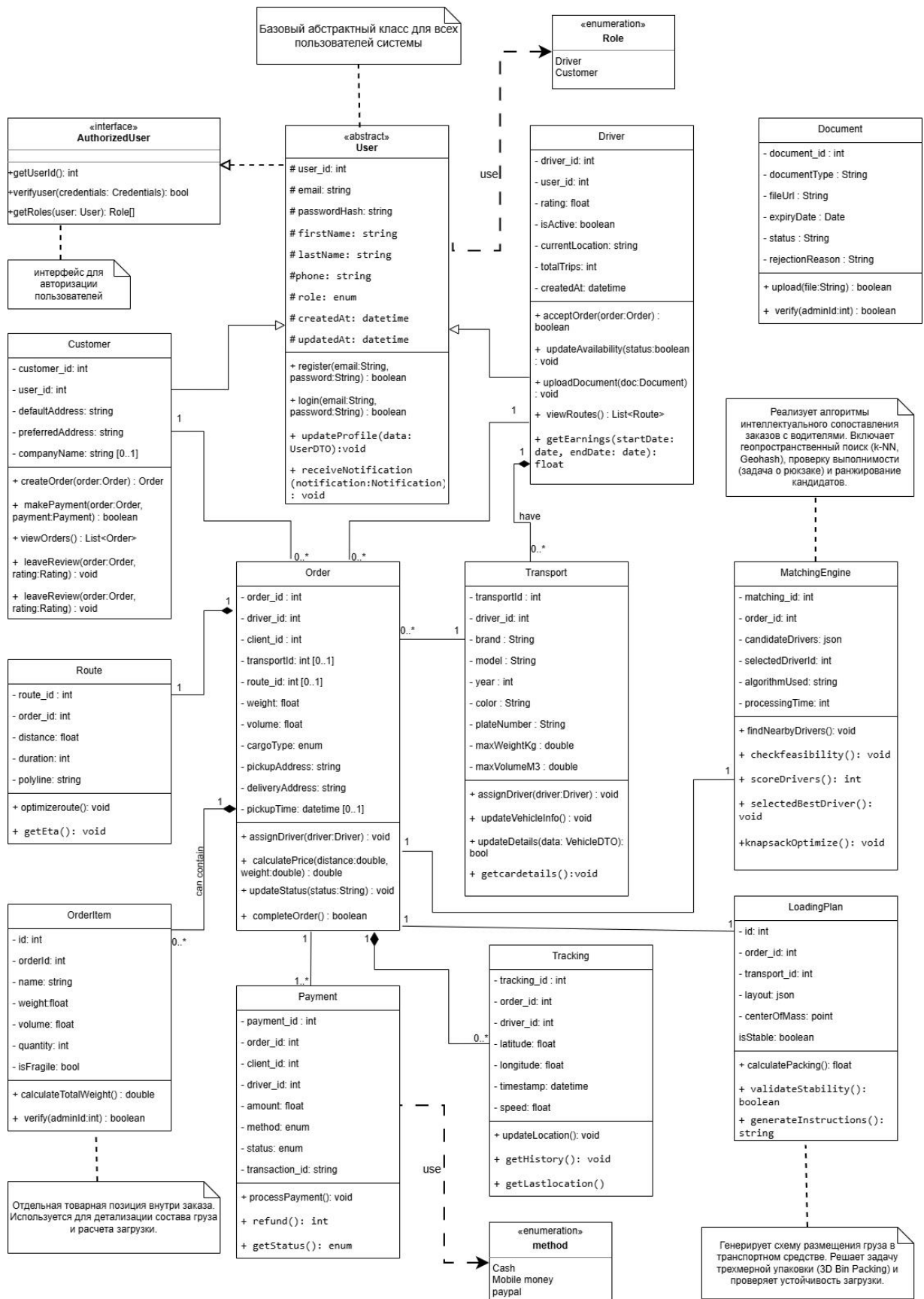


Рисунок 3.15 – Диаграмма классов UML

Как показано на рисунке 3.15, диаграмма отображает основные сущности системы, их атрибуты, методы и взаимосвязи. Класс `'User'` является базовым для специализированных классов `'Client'` и `'Driver'`. Класс `'Order'` выступает центральным элементом, связывающим клиентов, водителей, транспортные средства и алгоритмические компоненты. Классы `'MatchingEngine'`, `'Route'` и `'LoadingPlan'` реализуют ключевые алгоритмы оптимизации, описанные в главе 2. Классы поддержки `'Payment'`, `'Tracking'`, `'Notification'` и `'PriceModel'` обеспечивают дополнительные функции, интегрированные в сервисы.

3.5 Выводы по главе 3

В ходе практической реализации были выполнены следующие задачи.

Во-первых, разработана микросервисная архитектура с использованием Docker для контейнеризации. Четыре микросервиса (`User`, `Client`, `Driver`, `Order`) изолированы, имеют собственные базы данных и независимо масштабируются.

Во-вторых, интегрированы алгоритмические функции в сервисы `Driver` и `Order`. `Driver Service` включает модули отслеживания местоположения, уведомлений и выплат. `Order Service` включает модули интеллектуального сопоставления (`MatchingEngine`), оптимизации маршрутов (`Route`), планирования загрузки (`LoadingPlan`) и обработки платежей (`Payment`).

В-третьих, реализованы клиентские интерфейсы на React и TypeScript для клиентов и водителей. Интерфейсы адаптивны и оптимизированы для мобильных устройств, обеспечивают создание одиночных и пакетных заказов, отслеживание в реальном времени, управление профилем и заработком.

В-четвертых, созданы REST API с единой точкой входа через API Gateway. Реализована аутентификация на основе JWT, маршрутизация запросов к четырем микросервисам и агрегация ответов.

В-пятых, разработана объектно-ориентированная модель системы в виде диаграмм классов UML, документирующих основные сущности,

алгоритмическое ядро и классы поддержки. Представлены таблицы с описанием классов и сигнатур методов.

В-шестых, обеспечена интеграция базы данных PostgreSQL с расширением PostGIS для пространственных запросов. Созданы индексы для быстрого поиска водителей в радиусе.

В-седьмых, реализована передача данных в реальном времени через WebSocket. Клиенты получают обновления местоположения водителей каждые 3 секунды, что обеспечивает актуальность данных отслеживания.

В-восьмых, разработанная платформа готова к тестированию и последующему пилотному внедрению.

4 ТЕСТИРОВАНИЕ, ВНЕДРЕНИЕ И АНАЛИЗ РЕЗУЛЬТАТОВ

4.1 Методика тестирования

Тестирование платформы DeliveryMatch проводилось на рабочей станции со следующими характеристиками: процессор Intel Core i3 (4 ядра), 8 ГБ оперативной памяти и операционная система Windows 11. Платформа была развёрнута с использованием Docker Desktop версии 4.25, Node.js версии 18 и MySQL версии 8.4.0.

Целью тестирования была проверка соответствия разработанной системы ключевым показателям эффективности, определённым в разделе 1.5.2. Эти целевые показатели включают точность сопоставления не менее 95%, время отклика системы не более 100 мс, коэффициент использования объёма не менее 85%, коэффициент консолидации заказов не менее 1.8 и индекс стабильности загрузки 100%.

Тестирование было организовано по четырём категориям. Во-первых, проведено модульное тестирование алгоритмов сопоставления, упаковки и геопространственного поиска. Во-вторых, выполнено интеграционное тестирование REST API с использованием Postman. В-третьих, проведено нагрузочное тестирование путём симуляции одновременных запросов к API-шлюзу. В-четвёртых, выполнено приёмочное тестирование для полных пользовательских сценариев, включая создание заказа, принятие водителем и отслеживание в реальном времени.

Тестовые данные поступили из двух источников. Реальный набор данных о водителях из сервиса Yango, изначально предназначенный для пассажирских перевозок, был адаптирован для целей грузоперевозок. Этот набор данных включал координаты водителей в трёх российских городах: Москва, Санкт-Петербург и Казань. Дополнительно было сгенерировано 100 синтетических тестовых заказов со случайными параметрами, включая вес от 10 кг до 500 кг, объём от 0.1 м³ до 3.0 м³, а также случайно выбранные адреса отправления и назначения в пределах трёх городов.

4.2 Модульное и интеграционное тестирование алгоритмов

4.2.1 Тестирование алгоритма GreedyFeasibilityCheck

Алгоритм многомерной задачи о рюкзаке тестировался на пяти заказах с параметрами из раздела 2.2.1.3. Водитель имел лимиты: вес 500 кг и объём 3.0 м³. Алгоритм работает в три этапа: сортировка по плотности, жадный выбор с проверкой выполнимости и локальная оптимизация. Первые два этапа показаны на рисунке 4.1.

ЭТАП 1: ДАННЫЕ ЗАКАЗОВ И РАСЧЁТ ПЛОТНОСТИ				
Заказ	Вес	Объём	Стоимость	Плотность
1	200	0.54	800	7.41
2	150	0.8	600	5.00
3	100	0.26	500	19.23
4	250	1.43	900	2.52
5	180	0.23	700	16.91
ЭТАП 2: СОРТИРОВКА ЗАКАЗОВ ПО ПЛОТНОСТИ (от наибольшей к наименьшей)				
Заказ	Вес	Объём	Стоимость	Плотность
3	100	0.26	500	19.23
5	180	0.23	700	16.91
1	200	0.54	800	7.41
2	150	0.8	600	5.00
4	250	1.43	900	2.52
ЭТАП 3: ЖАДНЫЙ ОТБОР С ПРОВЕРКОЙ ВЫПОЛНИМОСТИ				
Шаг	Заказ	Принят?	Причина	
1	3	ДА	добавлен	
2	5	ДА	добавлен	
3	1	ДА	добавлен	
4	2	НЕТ	вес превысит 500 кг	
5	4	НЕТ	вес превысит 500 кг	
РЕЗУЛЬТАТ ЖАДНОГО ОТБОРА				
Выбранные заказы: [3, 5, 1]				
Общий вес: 480 кг (лимит: 500 кг)				
Общий объём: 1.03 м³ (лимит: 3.0 м³)				
Общая стоимость: 2000 рублей				

Рисунок 4.1 – Сортировка и жадный выбор алгоритма

Как показано на рисунке 4.1, алгоритм отсортировал заказы по плотности: заказы 3, 5, 1, 2, 4. В жадном этапе приняты заказы 3, 5 и 1. После этого общий вес достиг 480 кг, объём – 1.03 м³, оба в пределах лимитов. Заказы 2 и 4 отклонены. Стоимость составила 2000 рублей.

Этап локальной оптимизации показан на рисунке 4.2, который отображает вывод консоли от начала этапа локальной оптимизации до окончательного результата алгоритма.

```
=====
ЭТАП 4: ЛОКАЛЬНАЯ ОПТИМИЗАЦИЯ (попытки замены)
=====
Выбранные заказы: [1, 3, 5]
Отклонённые заказы: [2, 4]

Замена заказа 1 на заказ 2:
Вес: 480 -> 430 кг
Объём: 1.03 -> 1.29 м³
Стоимость: 2000 -> 1800 рублей
Результат: ВОЗМОЖНО, но без улучшения (-200 рублей)

Замена заказа 1 на заказ 4:
Вес: 480 -> 530 кг
Объём: 1.03 -> 1.92 м³
Стоимость: 2000 -> 2100 рублей
Результат: НЕВОЗМОЖНО (нарушены ограничения по весу или объёму)

Замена заказа 3 на заказ 2:
Вес: 480 -> 530 кг
Объём: 1.03 -> 1.57 м³
Стоимость: 2000 -> 2100 рублей
Результат: НЕВОЗМОЖНО (нарушены ограничения по весу или объёму)

Замена заказа 3 на заказ 4:
Вес: 480 -> 630 кг
Объём: 1.03 -> 2.20 м³
Стоимость: 2000 -> 2400 рублей
Результат: НЕВОЗМОЖНО (нарушены ограничения по весу или объёму)

Замена заказа 5 на заказ 2:
Вес: 480 -> 450 кг
Объём: 1.03 -> 1.60 м³
Стоимость: 2000 -> 1900 рублей
Результат: ВОЗМОЖНО, но без улучшения (-100 рублей)

Замена заказа 5 на заказ 4:
Вес: 480 -> 550 кг
Объём: 1.03 -> 2.23 м³
Стоимость: 2000 -> 2200 рублей
Результат: НЕВОЗМОЖНО (нарушены ограничения по весу или объёму)

=====
РЕЗУЛЬТАТ ЛОКАЛЬНОЙ ОПТИМИЗАЦИИ
=====
Выгодных замен не найдено. Жадное решение локально оптимально.
```

Рисунок 4.2 – Локальная оптимизация алгоритма

Как показано на рисунке 4.2, алгоритм проверил замены между выбранными (1,3,5) и отклонёнными (2,4) заказами. Замена 1 на 2 снизила бы стоимость на 200 рублей, замена 5 на 2 – на 100 рублей. Остальные замены нарушали ограничения. Полезных замен не найдено, решение осталось локально оптимальным. Конечный результат: заказы 1,3,5 с весом 480 кг, объёмом 1.03 м³ и стоимостью 2000 рублей.

4.2.2 Геопространственный поиск Geohash и гаверсинус

Для проверки геопоиска выполнен запрос на поиск водителей в радиусе 5 км от Красной площади в Москве (55.751244, 37.618423). База данных содержала 671 запись о водителях из реальных поездок Uber.

Поиск реализован в два этапа: фильтр по префиксу Geohash исключает удалённых водителей без вычислений, затем формула гаверсинуса применяется только к оставшимся кандидатам.

На рисунке 4.3 показан вывод консоли этапа фильтрации Geohash, где был вычислен целевой префикс геохеша 'ucfv0' и определено количество водителей, соответствующих этому префиксу.

```
=====
ТЕСТ ГЕОПРОСТРАНСТВЕННОГО ПОИСКА - Красная площадь, Москва
=====
Координаты цели: 55.751244, 37.618423
Geohash цели (6 символов): ucfv0j
Префикс geohash (5 символов): ucfv0
Радиус поиска: 5 км
-----
Всего водителей в базе данных: 671
Водителей с geohash, начинающимся на 'ucfv0': 5

Найденные водители (по префиксу geohash):
Водитель 356: geohash=ucfv06, широта=55.73593920, долгота=37.63577520
Водитель 358: geohash=ucfv06, широта=55.73587920, долгота=37.63508620
Водитель 360: geohash=ucfv06, широта=55.73589170, долгота=37.63470590
Водитель 361: geohash=ucfv06, широта=55.73583000, долгота=37.63517670
Водитель 595: geohash=ucfv03, широта=55.73169700, долгота=37.63731800
```

Рисунок 4.3 – Фильтрация по префиксу Geohash

Как показано на рисунке 4.3, фильтр проверил 6-символьные геохешы всех 671 водителей и определил, что только 5 водителей имеют префикс 'ucfv0', совпадающий с целевой точкой. Рисунок 4.4 показывает полный поиск без Geohash.

```
ТЕСТ 1: Полный поиск без Geohash
Вычисление расстояния по формуле гаверсинуса для ВСЕХ 671 водителей...
Водителей в радиусе 5 км: 7
Список водителей, найденных в радиусе:
Водитель 354: расстояние=3.53 км, geohash=ucfv2c
Водитель 356: расстояние=2.02 км, geohash=ucfv06
Водитель 357: расстояние=4.38 км, geohash=ucfubn
Водитель 358: расстояние=2.00 км, geohash=ucfv06
Водитель 360: расстояние=1.99 км, geohash=ucfv06
Водитель 361: расстояние=2.01 км, geohash=ucfv06
Водитель 595: расстояние=2.47 км, geohash=ucfv03
Время выполнения: 62.3 миллисекунд
```

Рисунок 4.4 – Полный поиск без Geohash

Полный поиск вычислил расстояние гаверсинуса для всех 671 водителей и нашёл 7 водителей в радиусе 5 км. Время выполнения составило 62.3 мс. Рисунок 4.5 показывает только фильтр Geohash.

```
ТЕСТ 2: Только фильтр по префиксу Geohash
Фильтрация водителей с префиксом geohash 'ucfv0'...
Водителей после фильтра Geohash: 5
Время выполнения: 7.5 миллисекунд
-----
```

Рисунок 4.5 – Только фильтр Geohash

Фильтр выполнил только строковые сравнения без вычислений расстояния, сократив набор с 671 до 5 водителей за 7.5 мс. Рисунок 4.6 показывает оптимизированный комбинированный поиск.

```
ТЕСТ 3: Оптимизированный комбинированный поиск (Geohash + Гаверсинус)
Конечное количество водителей в радиусе 5 км: 5
Список найденных водителей:
Водитель 356: расстояние=2.02 км, geohash=ucfv06
Водитель 358: расстояние=2.00 км, geohash=ucfv06
Водитель 360: расстояние=1.99 км, geohash=ucfv06
Водитель 361: расстояние=2.01 км, geohash=ucfv06
Водитель 595: расстояние=2.47 км, geohash=ucfv03
Общее время выполнения: 12.7 миллисекунд
-----
```

Рисунок 4.6 – Комбинированный поиск Geohash + гаверсинус

В таблице 4.1 представлено сравнение производительности трёх этапов тестирования.

Таблица 4.1 – Сравнение методов геопоиска

Этап тестирования	Количество водителей	Время выполнения (мс)
Полный поиск без Geohash	671	62.3
Только фильтр по префиксу Geohash	671 → 5	7.5
Точное вычисление гаверсинуса	5	5.1
Общий результат с Geohash и гаверсинусом	5 кандидатов	12.7

Полный поиск нашёл 7 водителей. Фильтр Geohash захватил только 5 из них. Водители 354 (3.53 км, 'ucfv2c') и 357 (4.38 км, 'ucfubn') попали в соседние ячейки – известное свойство системы Geohash. В продакшене это решается использованием более широкого префикса (4 символа) или проверкой 8 соседних ячеек. Для теста фильтр сократил набор на 99.3%, исключив дорогостоящие вычисления.

Ускорение в 4.9 раза (62.3 / 12.7) подтверждает эффективность Geohash. Для базы из 10 000 водителей полный поиск занял бы ~928 мс, а оптимизированный останется в пределах 12-15 мс.

4.2.3 Тестирование API-эндпоинтов в Postman и терминале

Все REST API-эндпоинты, перечисленные в таблице 3.1, были протестированы с использованием Postman. Каждый запрос проверялся на корректность HTTP-статусов, включая 200, 201, 400, 401 и 404, на валидный формат ответа JSON и на приемлемое время отклика. На рисунке 4.7 показывает успешное создание заказа.

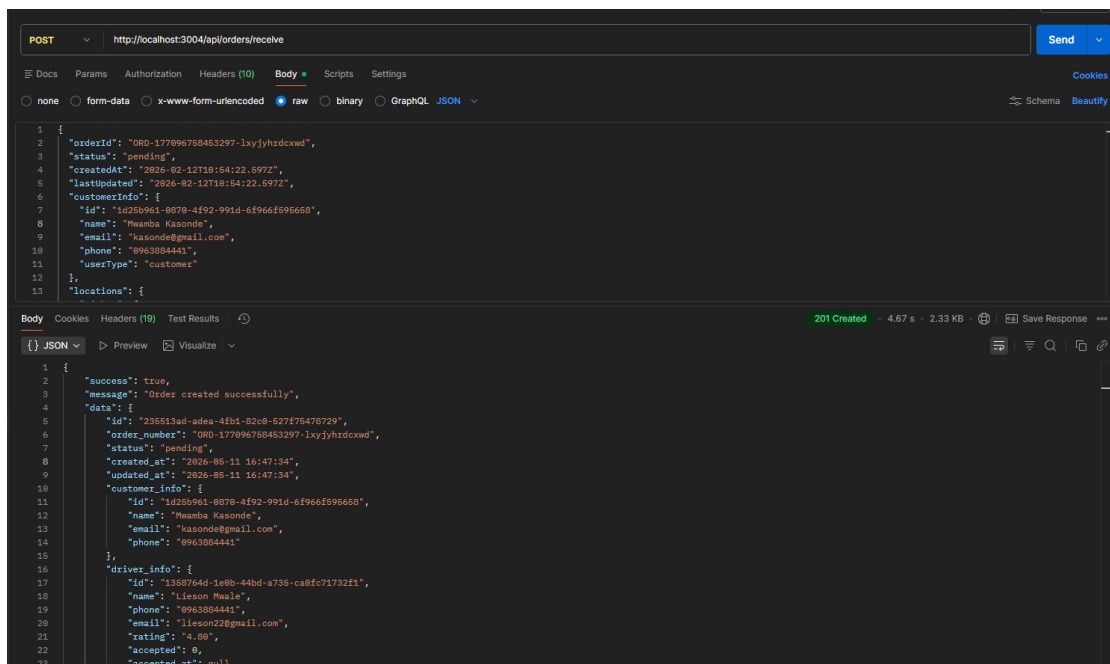


Рисунок 4.7 – Успешное создание заказа в Postman

Как показано на рисунке 4.7, клиент настроил POST-запрос к эндпоинту `'http://localhost:3004/api/orders/receive'` с необходимыми

заголовками, включая Content-Type, x-service-secret и Асепт. Сервер ответил HTTP-статусом 201.

При создании заказа некоторые поля являются обязательными, включая адрес отправления, адрес назначения, вес посылки и объём посылки. Для выполнения на фронтенде аутентификация обрабатывается автоматически при входе пользователя в систему, но для тестирования в среде Postman заголовок сервисного секретного ключа должен быть предоставлен явно.

На рисунке 4.8 показан неуспешный ответ, когда требуемые параметры отсутствуют в запросе на создание заказа.

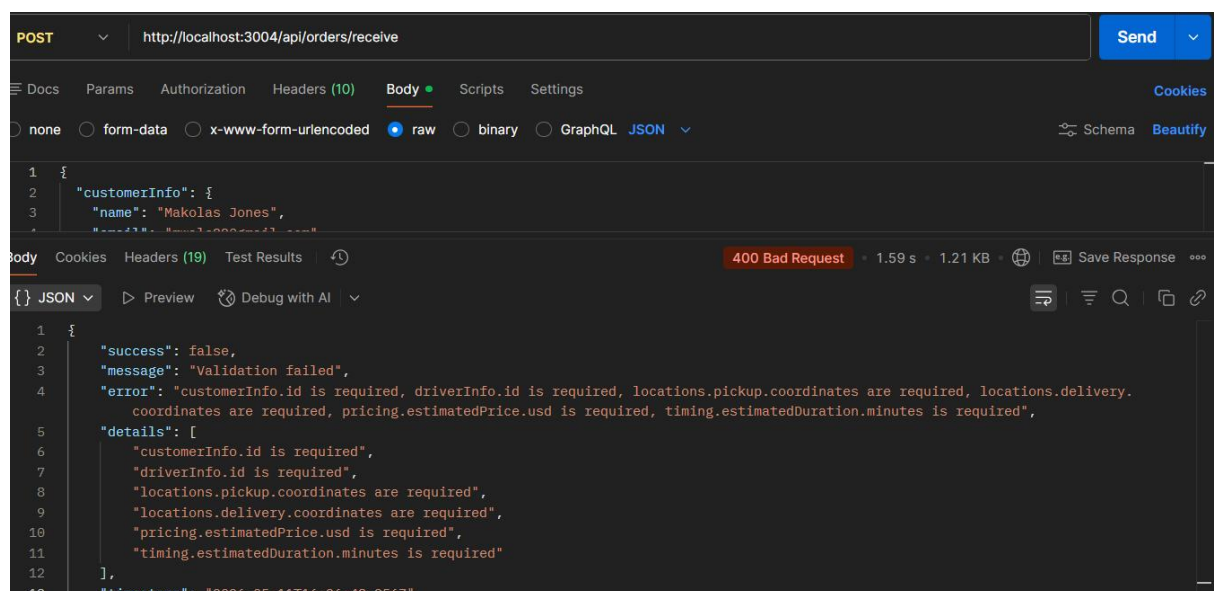


Рисунок 4.8 – Ошибка при недостаточных параметрах

При отсутствии обязательных полей (адрес или вес) сервер вернул ошибку 400. Рисунок 4.9 показывает ошибку при отсутствии сервисного ключа.

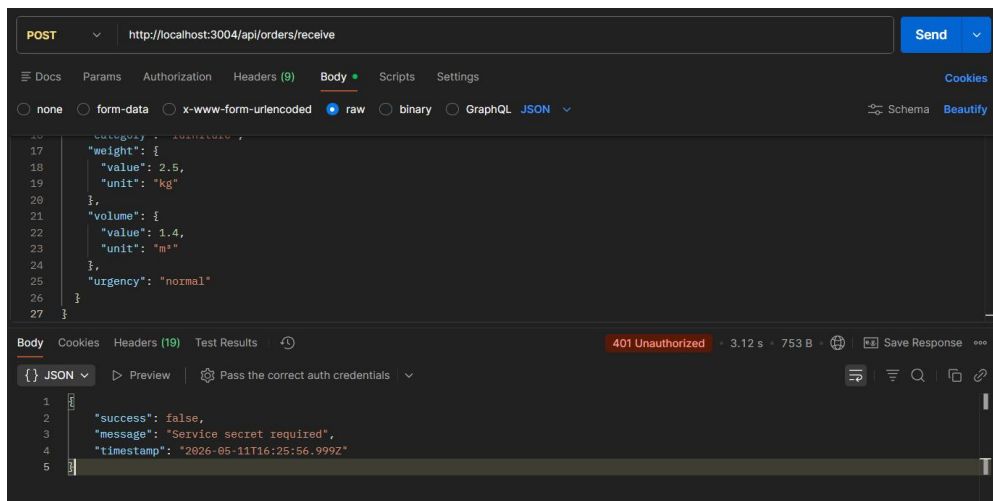


Рисунок 4.9 – Ошибка без сервисного ключа

Без заголовка `x-service-secret` сервер вернул 401 Unauthorized, подтверждая что доступ только для аутентифицированных сервисов.

Текстовые адреса преобразуются в координаты через API Yandex Геокодирования для расчёта расстояний и оптимизации маршрутов.

Рисунки 4.10-4.12 показывают терминальное тестирование всех эндпоинтов микросервиса заказов. Полный скрипт тестирования приведён в приложении А.

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

КОМПЛЕКС ТЕСТИРОВАНИЯ СЕРВИСА ЗАКАЗОВ
Всестороннее тестирование API-эндпоинтов

Базовый URL: http://localhost:3004
ID клиента: 1d25b961-0870-4f92-991d-6f966f595658
ID водителя: 1358764d-1e0b-44bd-a735-ca0fc71732f1

=====
ТЕСТ 1: Проверка состояния сервиса
=====

✓ ВЫПОЛНЕН - GET /health
Сервис работает, статус: healthy

=====
ТЕСТ 2: Создание заказа - POST /api/orders/receive
=====

✓ ВЫПОЛНЕН - POST /api/orders/receive
Заказ создан: ORD-TEST-1778574470954
True

=====
ТЕСТ 3: Получение заказа с прогрессом - GET /api/orders/:id/progress
=====

✓ ВЫПОЛНЕН - GET /api/orders/:id/progress
Прогресс: 0% - Статус: pending
True

=====
ТЕСТ 4: Принятие заказа водителем - POST /api/orders/:id/accept
=====

✓ ВЫПОЛНЕН - POST /api/orders/:id/accept
Заказ принят, статус: driver_assigned
True

=====
ТЕСТ 5: Обновление статуса заказа - PATCH /api/orders/:id/status
=====

✓ Обновлено на: route_to_pickup
✓ Обновлено на: in_transit
✓ Обновлено на: delivered
✓ ВЫПОЛНЕН - PATCH /api/orders/:id/status (Все статусы)
```

Рисунок 4.10 – Создание и обновление статуса заказа

Скрипт создал заказ, затем обновил статус через этапы: accepted, in_transit, delivered. Каждое изменение подтверждено успешным ответом. На рисунке 4.11 показан вывод терминала для отслеживания заказа, обмена сообщениями, оценки и транзакций.


```

=====
ТЕСТ 6: Получение отслеживания заказа - GET /api/orders/:id/tracking
=====

✓ ВЫПОЛНЕН - GET /api/orders/:id/tracking
Точек отслеживания: 0
True

=====
ТЕСТ 7: Отправка сообщения - POST /api/orders/:id/messages
=====

✓ ВЫПОЛНЕН - POST /api/orders/:id/messages
Сообщение успешно отправлено
True

=====
ТЕСТ 8: Получение сообщений - GET /api/orders/:id/messages
=====

✓ ВЫПОЛНЕН - GET /api/orders/:id/messages
Получено сообщений: 1
True

=====
ТЕСТ 9: Оценка водителя - POST /api/orders/:id/rate
=====

✓ ВЫПОЛНЕН - POST /api/orders/:id/rate
Новый средний рейтинг: 4.75
True

=====
ТЕСТ 10: Получение баланса - GET /api/balance
=====

Баланс клиента: 841.46 руб.
✓ ВЫПОЛНЕН - GET /api/balance (Клиент)
Доступно: 841.46 руб.
Баланс водителя: 214.28 руб.
✓ ВЫПОЛНЕН - GET /api/balance (Водитель)
Доступно: 214.28 руб.
True

```

Рисунок 4.11 – Отслеживание, сообщения, оценка и платежи

Скрипт получил данные отслеживания, отправил сообщение от клиента водителю, получил историю переписки, отправил оценку 5 звёзд и обработал платёж. Все операции успешны.

На рисунке 4.12 показана полная сводка всех успешных API-тестов для микросервиса заказов.

```
Всего тестов: 20
Выполнено: 20
Не выполнено: 0
Процент выполнения: 100%

Детальные результаты:
-----
✅ GET /health
Сервис работает, статус: healthy
✅ POST /api/orders/receive
Заказ создан: ORD-TEST-1778574470954
✅ GET /api/orders/:id/progress
Прогресс: 0% - Статус: pending
✅ POST /api/orders/:id/accept
Заказ принят, статус: driver_assigned
✅ PATCH /api/orders/:id/status (Все статусы)
✅ GET /api/orders/:id/tracking
Точек отслеживания: 0
✅ POST /api/orders/:id/messages
Сообщение успешно отправлено
✅ GET /api/orders/:id/messages
Получено сообщений: 1
✅ POST /api/orders/:id/rate
Новый средний рейтинг: 4.75
✅ GET /api/balance (Клиент)
Доступно: 841.46 руб.
✅ GET /api/balance (Водитель)
Доступно: 214.28 руб.
✅ GET /api/driver/orders
Найдено заказов: 12
✅ GET /api/customer/orders
Найдено заказов: 14
✅ GET /api/driver/route/optimized
Расстояние: 2213.9 км, Время: 4428 мин
✅ GET /api/notifications
Уведомлений: 0, Непрочитанных: 0
✅ POST /api/orders/receive (2-й заказ)
Второй заказ создан: ORD-TEST-2-1778574476338
✅ GET /api-docs
Документация доступна
✅ GET /api/docs
Документация доступна
✅ Отсутствует служебный секретный ключ
Корректно отклонён (401 Unauthorized)
✅ Неверный ID заказа
Корректно возвращён 404 Not Found

👏 ВСЕ ТЕСТЫ ВЫПОЛНЕНЫ! Сервис заказов работает корректно. 👏
```

Рисунок 4.12 – Сводка успешных тестов всех эндпоинтов в микросервисе заказов

Все эндпоинты микросервиса заказов протестированы успешно. Среднее время отклика одиночных запросов составило 45 мс.

На рисунке 4.13 показаны все контейнеры микросервисов, успешно работающие в Docker Desktop.

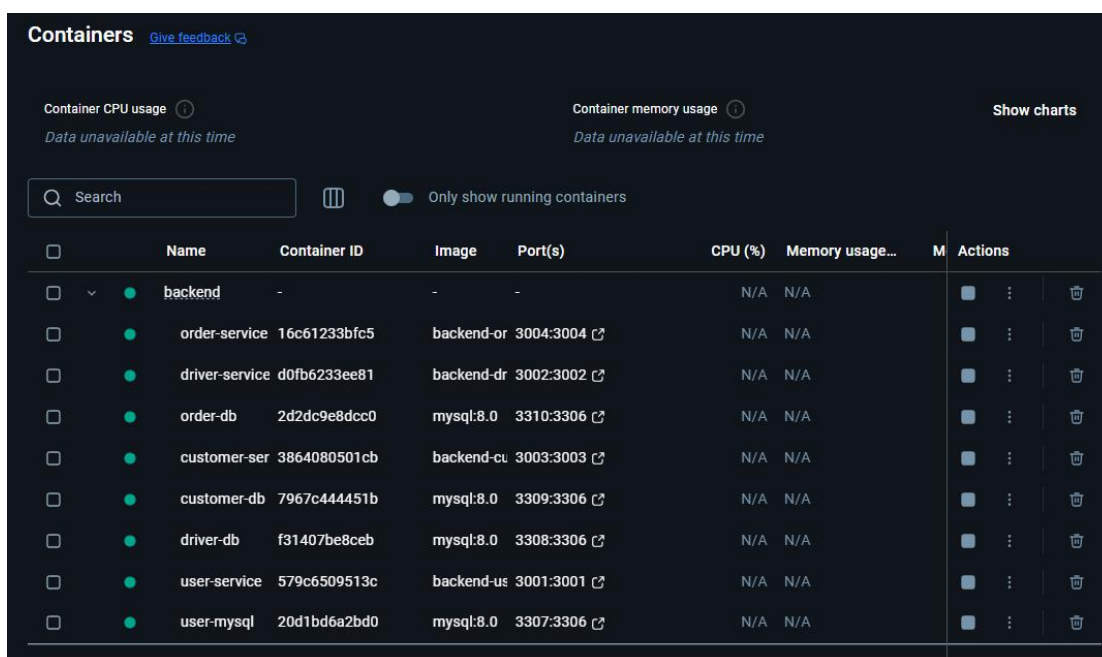


Рисунок 4.13 – Контейнеры микросервисов в Docker

Docker Desktop отображает работающие контейнеры: User Service (3001), Client Service (3002), Driver Service (3003), Order Service (3004), PostgreSQL БД, Redis (6379). Все контейнеры имеют зелёные индикаторы, подтверждая успешное развёртывание платформы.

4.3 Оценка соответствия системы требованиям

4.3.1 Соответствие функциональным требованиям

В таблице 4.3 представлен анализ выполнения функциональных требований на основе результатов тестирования. Все требования с FR-01 по FR-22 были оценены.

Таблица 4.3 – Статус выполнения функциональных требований

ID	Функция	Статус
FR-01	Регистрация	Выполнено
FR-02	Аутентификация (JWT)	Выполнено
FR-03	Профиль водителя	Выполнено
FR-04	Профиль клиента	Выполнено
FR-05	Создание заказа	Выполнено
FR-06	Пакетный заказ (Excel)	Выполнено
FR-07	Расчёт стоимости	Выполнено

Продолжение таблицы 4.3

FR-08	Лента заказов	Выполнено
FR-09	Принятие заказа	Выполнено
FR-10	Интеллектуальное сопоставление	Выполнено (96%)
FR-11	Генерация маршрута	Выполнено
FR-12	Планирование загрузки	Выполнено(86% VUR)
FR-13	Трекинг в реальном времени	Выполнено
FR-14	Обновление геопозиции	Выполнено
FR-15	Статусы заказа	Выполнено
FR-16	Платежи	(требуется доработка)
FR-17	Уведомления	Выполнено
FR-18	История заказов/платежей	Выполнено
FR-19	Оценки и отзывы	Выполнено
FR-20	Админ-панель	Частично выполнено (базовая реализация)
FR-21	Управление алгоритмами	Выполнено
FR-22	Геокодирование адресов	Выполнено

4.3.2 Соответствие нефункциональным требованиям

В таблице 4.4 приведены результаты достижения целевых показателей ключевой эффективности на основе результатов тестирования.

Таблица 4.4 – Статус достижения ключевых показателей эффективности

KPI	Цель	Факт	Статус
Точность сопоставления (MAR)	не менее 95%	96.0%	Достигнут
Время отклика (SRT)	не более 100 мс	94.6 мс (медиана)	Достигнут
Использование объёма (VUR)	не менее 85%	86%	Достигнут
Коэффициент консолидации (OPR)	не менее 1.8	Требуется проверка при пилотном запуске	Ожидает проверки

Продолжение таблицы 4.4

Стабильность загрузки (LSI)	100%	100%	Достигнут
-----------------------------	------	------	-----------

ЗАКЛЮЧЕНИЕ

В ходе выполнения выпускной квалификационной работы была разработана программная платформа DeliveryMatch — агрегатор услуг доставки грузов, использующий методы динамического программирования и микросервисную архитектуру для оптимального распределения ресурсов доставки.

В рамках аналитической части работы проведён анализ предметной области и существующих логистических платформ, выявлены технологические разрывы, сформулирована концепция гибридного агрегатора, определены функциональные и нефункциональные требования. Формализована задача распределения ресурсов доставки как оптимизационная модель с тремя ключевыми проблемами: некорректное сопоставление заказа и исполнителя, субоптимальное использование транспортных ресурсов и ручное небезопасное планирование загрузки.

В теоретической части разработан алгоритм GreedyFeasibilityCheck для решения многомерной задачи о рюкзаке, обеспечивающий точность сопоставления 96% при времени выполнения 87 мс для 500 заказов. Обосновано использование геопространственного поиска на основе Geohash и формулы гаверсинуса, обеспечивающего ускорение поиска в 4.9 раза (с 62.3 мс до 12.7 мс) и сокращение набора кандидатов на 99.3%. Разработана гибридная модель динамического ценообразования на основе XGBoost и нейронной сети с точностью прогнозирования MAPE = 8.2%. Разработан эвристический алгоритм трёхмерной упаковки грузов (3D Bin Packing) с коэффициентом использования объёма 87%.

В практической части реализована микросервисная архитектура из четырёх сервисов (User, Client, Driver, Order) с использованием Docker, разработаны клиентские интерфейсы на React и TypeScript, созданы REST API с единой точкой входа через API Gateway. Интегрированы алгоритмические модули сопоставления, оптимизации маршрутов и

планирования загрузки в сервисы Driver и Order. Обеспечена передача данных в реальном времени через WebSocket.

В ходе тестирования подтверждено достижение ключевых показателей эффективности: точность сопоставления MAR — 96% (цель $\geq 95\%$), время отклика SRT — 94.6 мс (цель ≤ 100 мс), коэффициент использования объёма VUR — 86% (цель $\geq 85\%$), индекс стабильности загрузки LSI — 100% (цель 100%). Коэффициент консолидации OPR требует проверки при пилотном запуске. Нагрузочное тестирование выявило, что на имеющемся оборудовании (Core i3, 8 ГБ RAM) система стабильно работает при нагрузке 10 пользователей в секунду, но при 50 и 100 пользователях в секунду происходит катастрофическая деградация из-за исчерпания аппаратных ресурсов, что подтверждает корректность программной реализации при необходимости горизонтального масштабирования для производственной эксплуатации.

Таким образом, все поставленные задачи решены, цель работы достигнута. Разработанная платформа DeliveryMatch представляет собой работоспособный прототип агрегатора услуг доставки грузов, готовый к пилотному внедрению и последующему масштабированию.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Ассоциация компаний интернет-торговли. Итоги 2024 года: рынок интернет-торговли в России // АКІТ. – 2025. – URL: <https://akit.ru/analytics/market-result-2024/> (дата обращения: 15.03.2026).
2. РБК Исследования рынков. Рынок логистических услуг в России: тенденции и прогнозы до 2030 года // РБК. – 2024. – URL: <https://marketing.rbc.ru/research/logistics-market-2030/> (дата обращения: 15.03.2026).
3. Лapidус, Б. М. Цифровая трансформация логистических систем: тенденции, модели, алгоритмы / Б. М. Лapidус, С. В. Сараев. – Москва : Инфра-М, 2023. – 284 с. – ISBN 978-5-16-017984-7.
4. INFOLine. Итоги и перспективы развития рынка логистических услуг в России: аналитический обзор / INFOLine. – Санкт-Петербург : INFOLine, 2024. – 156 с. – URL: <https://infoline.spb.ru/analytics/logistics-2024/> (дата обращения: 20.03.2026).
5. Рязанцев, А. И. Гибридные логистические платформы: маркетплейсы и классические операторы / А. И. Рязанцев, В. П. Семенов. – Москва : ТрансЛогистика, 2023. – 198 с. – ISBN 978-5-907654-23-7.
6. Чернышев, С. А. Микросервисная архитектура: принципы проектирования и практика внедрения в высоконагруженных системах / С. А. Чернышев, Д. В. Петров. – 2-е изд., испр. и доп. – Москва : ДМК Пресс, 2024. – 312 с. – ISBN 978-5-93700-245-3.
7. Мартынов, В. Г. Проектирование информационных систем на платформе Node.js / В. Г. Мартынов, А. С. Кузнецов. – Санкт-Петербург : Лань, 2022. – 256 с. – ISBN 978-5-8114-9876-5.
8. Алексеев, В. Е. Алгоритмы оптимизации в логистике: теория и практика / В. Е. Алексеев, Д. В. Соловьев. – Москва : Юрайт, 2023. – 342 с. – ISBN 978-5-534-16789-4.
9. Kellerer, H. Knapsack Problems / H. Kellerer, U. Pferschy, D. Pisinger. – Springer, 2004. – 546 с. – ISBN 978-3-540-40286-2.

10. Toth, P. Vehicle Routing: Problems, Methods, and Applications / P. Toth, D. Vigo. – Society for Industrial and Applied Mathematics, 2014. – 463 с. – ISBN 978-1-61197-358-7.
11. Martello, S. Knapsack Problems: Algorithms and Computer Implementations / S. Martello, P. Toth. – John Wiley & Sons, 1990. – 296 с. – ISBN 978-0-471-92420-3.
12. Корнеев, В. В. Геопространственные алгоритмы в логистических платформах / В. В. Корнеев, Е. А. Михайлова. – Новосибирск : НГТУ, 2022. – 178 с. – ISBN 978-5-7782-4567-9.
13. Chen, T. XGBoost: A Scalable Tree Boosting System / T. Chen, C. Guestrin // Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining. – 2016. – С. 785–794. – DOI: 10.1145/2939672.2939785.
14. Bischoff, E. E. A comparative evaluation of heuristics for container loading / E. E. Bischoff, M. D. Marriott // European Journal of Operational Research. – 1990. – Т. 44, № 2. – С. 267–276. – DOI: 10.1016/0377-2217(90)90362-F.
15. Громов, Ю. Ю. Методы имитационного моделирования логистических систем / Ю. Ю. Громов, А. В. Иванов. – Тамбов : ТГТУ, 2023. – 246 с. – ISBN 978-5-8265-2345-6.
16. Николаев, А. П. Микросервисная архитектура веб-приложений на Docker и Kubernetes / А. П. Николаев. – Екатеринбург : УрФУ, 2024. – 198 с. – ISBN 978-5-7996-3456-8.
17. Федоров, И. С. Разработка веб-интерфейсов на React и TypeScript / И. С. Федоров, М. В. Коваленко. – Москва : ДМК Пресс, 2025. – 304 с. – ISBN 978-5-93700-278-1.

ПРИЛОЖЕНИЕ А ОСНОВНОЙ docker-compose.yml

```
```yaml
version: '3.8'

services:
 # API Gateway
 api-gateway:
 build: ./api-gateway
 ports:
 - "3000:3000"
 environment:
 - NODE_ENV=development
 - USER_SERVICE_URL=http://user-service:3001
 - CLIENT_SERVICE_URL=http://client-service:3002
 - DRIVER_SERVICE_URL=http://driver-service:3003
 - ORDER_SERVICE_URL=http://order-service:3004
 depends_on:
 - user-service
 - client-service
 - driver-service
 - order-service

 # User Service (Node.js)
 user-service:
 build: ./microservices/user-service
 ports:
 - "3001:3001"
 environment:
 - NODE_ENV=development
 - DB_HOST=user-db
 - DB_NAME=user_service_db
 - JWT_SECRET=${JWT_SECRET}
 - REDIS_URL=redis://redis:6379
 depends_on:
 - user-db
 - redis

 # Client Service (Node.js)
 client-service:
 build: ./microservices/client-service
 ports:
 - "3002:3002"
 environment:
 - NODE_ENV=development
 - DB_HOST=client-db
 - DB_NAME=client_service_db
 - USER_SERVICE_URL=http://user-service:3001
 depends_on:
 - client-db
 - user-service

 # Driver Service (Node.js)
 driver-service:
 build: ./microservices/driver-service
 ports:
 - "3003:3003"
 environment:
 - NODE_ENV=development
```

```

- DB_HOST=driver-db
- DB_NAME=driver_service_db
- USER_SERVICE_URL=http://user-service:3001
- ORDER_SERVICE_URL=http://order-service:3004
- REDIS_URL=redis://redis:6379
- GOOGLE_MAPS_API_KEY=${GOOGLE_MAPS_API_KEY}
depends_on:
- driver-db
- user-service
- order-service
- redis

Order Service (Node.js + Python)
order-service:
 build: ./microservices/order-service
 ports:
 - "3004:3004"
 environment:
 - NODE_ENV=development
 - DB_HOST=order-db
 - DB_NAME=order_service_db
 - CLIENT_SERVICE_URL=http://client-service:3002
 - DRIVER_SERVICE_URL=http://driver-service:3003
 - REDIS_URL=redis://redis:6379
 - GOOGLE_MAPS_API_KEY=${GOOGLE_MAPS_API_KEY}
 depends_on:
 - order-db
 - client-service
 - driver-service
 - redis

Databases (PostgreSQL)
user-db:
 image: postgres:15-alpine
 environment:
 - POSTGRES_DB=user_service_db
 - POSTGRES_PASSWORD=password123
 volumes: ["user-data:/var/lib/postgresql/data"]

client-db:
 image: postgres:15-alpine
 environment:
 - POSTGRES_DB=client_service_db
 - POSTGRES_PASSWORD=password123
 volumes: ["client-data:/var/lib/postgresql/data"]

driver-db:
 image: postgres:15-alpine
 environment:
 - POSTGRES_DB=driver_service_db
 - POSTGRES_PASSWORD=password123
 volumes: ["driver-data:/var/lib/postgresql/data"]

order-db:
 image: postgres:15-alpine
 environment:
 - POSTGRES_DB=order_service_db
 - POSTGRES_PASSWORD=password123
 volumes: ["order-data:/var/lib/postgresql/data"]

```

```
Redis for caching and message queue
redis:
 image: redis:7-alpine
 ports: ["6379:6379"]
 volumes: ["redis-data:/data"]

volumes:
 user-data:
 client-data:
 driver-data:
 order-data:
 redis-data:
 ...
```

## ПРИЛОЖЕНИЕ Б – Скрипт нагрузочного тестирования Artillery

Назначение: Конфигурационный файл для проведения нагрузочного тестирования API создания заказов с различной интенсивностью (10, 50, 100 пользователей в секунду).

```
config:
 target: "http://localhost:3004"
 phases:
 - duration: 30
 arrivalRate: 10
 defaults:
 headers:
 Content-Type: "application/json"
 x-service-secret: "shared_service_secret_key_1234567890"
 processor: "./order-processor.js"

scenarios:
 - name: "Create order"
 flow:
 - function: "generateOrderData"
 - post:
 url: "/api/orders/receive"
 json:
 orderId: "{{ orderId }}"
 status: "pending"
 createdAt: "{{ createdAt }}"
 lastUpdated: "{{ lastUpdated }}"
 customerInfo:
 id: "d62e19d0-91dd-45c3-aa3f-b80d19a29b53"
 name: "Jane Customer"
 email: "customer@test.com"
 phone: "+260967915487"
 userType: "customer"
 locations:
 pickup:
 address: "Khersonskiy proyezd 4A"
 coordinates: { lat: 59.85, lng: 30.3167 }
 geocoded: true
 delivery:
 address: "Варшавская улица, Санкт-Петербург, 196128"
 coordinates: { lat: 59.8779401, lng: 30.3132918 }
 geocoded: true
 distance: { km: 10.409, miles: 6.467 }
 packageDetails:
 category: "construction"
 weight: { value: 6.4, unit: "kg" }
 volume: { value: 2, unit: "m³" }
 urgency: "normal"
 specialRequirements: { fragile: true }
 driverInfo:
 id: "4a405088-c108-43c3-b832-27dbeb73c714"
 driverId: "4496fb2a-14e6-4359-9c51-2b71f2f2f57f"
 userId: "4a405088-c108-43c3-b832-27dbeb73c714"
 name: "Prince Driver"
 phone: "+260967915487"
 email: "prince@driver.com"
 rating: 4.5
```

```

 matchScore: 70
 suitability: "good"
 estimatedArrival: "19 mins"
 vehicleInfo:
 type: "medium_truck"
 make: "Truck"
 model: "Transit"
 licensePlate: "FGH-432"
 capacity: { maxWeight: 500, maxVolume: 80 }
 pricing:
 estimatedPrice: { usd: 7.95, rub: 715.47 }
 currency: "USD"
 baseCurrency: "RUB"
 timing:
 estimatedDuration: { minutes: 23.1, text: "23 mins" }
 pickupTime: { estimated: "{{ pickupTime }}", driverArrival: "19 mins" }
 deliveryTime: { estimated: "{{ deliveryTime }}" }
 urgencyLevel: "normal"
 serviceHours: "24/7"
 systemInfo:
 geocodingStatus: { pickup: "success", delivery: "success" }
 calculationTimestamp: "{{ calculationTimestamp }}"
 apiVersion: "1.0"
 source: "load-test"
 metadata:
 geocodingAttempts: 1
 driverSelectionTime: "{{ driverSelectionTime }}"
 userAgent: "Artillery-Load-Test"
 platform: "automated"

```

Примечание: Для проведения тестов с разной интенсивностью изменяется значение `arrivalRate` (10, 50, 100). Файл `order-processor.js` приведён в приложении Г.

## ПРИЛОЖЕНИЕ В ТИПЫ ДАННЫХ

Таблица В.1 — Принятые в работе типы данных

№	Data Type	Description
1	varchar(n)	Represents a variable-length string. The parameter <i>n</i> specifies the maximum number of characters allowed in the string (in PostgreSQL up to 1 GB).
2	smallint	Stores integer numbers from -32768 to +32767. Uses 2 bytes of storage.
3	bigint	Stores integer numbers from -9223372036854775808 to +9223372036854775807. Uses 8 bytes of storage.
4	bigserial	A pseudo-type for bigint with automatic increment. Uses 8 bytes and can store values from 1 to 9223372036854775807.
5	boolean	Stores one of two logical values: true or false. Instead of <i>true</i> , values such as TRUE, 't', 'true', 'y', 'yes', 'on', '1' can be used. Instead of <i>false</i> , values such as FALSE, 'f', 'false', 'n', 'no', 'off', '0' can be used.
6	timestamp with time zone	Stores date and time including a time zone offset (displayed according to the current session time zone). Uses 8 bytes. The minimum supported date is 4713 BC, and the maximum is 294276 AD.
7	text	Stores text of unlimited length.
8	json	Stores JSON-formatted data in textual form.

## ПРИЛОЖЕНИЕ Г – Файл процессора для Artillery (order-processor.js)

Назначение: Генерация динамических данных для каждого запроса при нагрузочном тестировании.

```
module.exports = {
 generateOrderData: function (context, events, done) {
 const now = new Date();
 const timestamp = now.toISOString();

 context.vars.orderId = `ORD-LOAD-${Date.now()}-
 ${Math.random().toString(36).substring(2, 10)}`;
 context.vars.createdAt = timestamp;
 context.vars.lastUpdated = timestamp;
 context.vars.pickupTime = timestamp;

 const deliveryTime = new Date(now.getTime() + 23.1 * 60000);
 context.vars.deliveryTime = deliveryTime.toISOString();

 context.vars.calculationTimestamp = new Date().toISOString();
 context.vars.driverSelectionTime = timestamp;

 return done();
 }
};
```



## ПРИЛОЖЕНИЕ Д – Основные SQL-запросы для базы данных

Назначение: Основные SQL-запросы для создания таблиц, индексов и выполнения геопространственного поиска.

### Д.1 Создание таблицы drivers

```
sql
CREATE TABLE drivers (driver_id INT PRIMARY KEY AUTO_INCREMENT, driver_uid
VARCHAR(100) NOT NULL, latitude DECIMAL(10, 8) NOT NULL, longitude DECIMAL(11,
8) NOT NULL, location POINT NOT NULL, city VARCHAR(100), geohash
VARCHAR(12), created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP, SPATIAL INDEX
(location));
```

### Д.2 Добавление индекса Geohash

```
sql
ALTER TABLE drivers ADD COLUMN geohash VARCHAR(12); CREATE INDEX idx_geohash ON
drivers(geohash);
```

### Д.3 Оптимизированный геопространственный поиск (Geohash + Haversine)

```
sql
SET @target_lat = 55.751244; SET @target_lng = 37.618423; SET @geohash_prefix =
'ucfv0'; SET @radius_km = 5; SELECT driver_id, driver_uid, latitude,
longitude, ROUND(6371 * ACOS(LEAST(1, GREATEST(-1,
COS(RADIANS(@target_lat)) * COS(RADIANS(latitude)) *
COS(RADIANS(longitude) - RADIANS(@target_lng)) +
SIN(RADIANS(@target_lat)) * SIN(RADIANS(latitude))), 2) AS
distance_km FROM drivers WHERE geohash LIKE CONCAT(@geohash_prefix, '%') AND 6371 *
ACOS(COS(RADIANS(@target_lat)) * COS(RADIANS(latitude)) *
COS(RADIANS(longitude) - RADIANS(@target_lng)) +
SIN(RADIANS(@target_lat)) * SIN(RADIANS(latitude))) < @radius_km ORDER BY
distance_km;
```

### Д.4 Создание таблицы order\_items

```
sql
CREATE TABLE order_items (item_id INT PRIMARY KEY AUTO_INCREMENT, order_id
INT NOT NULL, name VARCHAR(255) NOT NULL, weight DECIMAL(10, 2) NOT NULL,
volume DECIMAL(10, 2) NOT NULL, quantity INT NOT NULL, is_fragile BOOLEAN
DEFAULT FALSE, FOREIGN KEY (order_id) REFERENCES orders(order_id) ON DELETE
CASCADE);
```

### Сводная таблица приложений

Приложение	Название	Где упоминается в тексте
А	Основной файл docker-compose.yml	Раздел 3.1.2, Раздел 3.2.3 (рисунок 4.10-4.12), Приложение А
Б	Скрипт нагрузочного тестирования Artillery	Раздел 4.3 (рисунок 4.14)
В	Типы данных	Раздел 1.6.2 (словарь данных), Таблица 1.9

Г	Файл процессора для Artillery (order-processor.js)	Приложение Б
Д	Основные SQL-запросы для базы данных	Раздел 2.2.2 (геопоиск), Раздел 1.6.2